MỤC LỤC

PHẦN MỞ ĐẦU **4**
CHƯƠNG 1 MỘT SỐ KIỂU DỮ LIỆU CƠ BẢN **6**
1.1. GIỚI THIỆU CHUNG VỀ NGÔN NGỮ LẬP TRÌNH C VÀ C++ **6**
1.2.2. Từ khoá 7
1.2.4. Biến 7
1.2.5. Cách đặt tên 8
1.2.6. Lời chú thích 9
1.2.7. Hằng 9
1.2.8. Thư viện 10
1.2.9. Lệnh nhập - xuất 10
1.2.10. Cấu trúc của một chương trình C++ 10
1.3. CÁC KIỂU DỮ LIỆU CƠ BẢN VÀ CÁC TOÁN TỬ **11**
1.3.1. Kiểu số nguyên 11
1.3.1.1. Dữ liệu kiểu char, int 11
1.3.1.2. Phương pháp biểu diễn số nguyên 12
1.3.2. Kiểu số thực 13
1.3.2.1. Kiểu float và double 13
1.3.2.2. Phương pháp biểu diễn số thực 14
1.3.3. Các toán tử 15
1.3.3.1. Toán tử gán 15
1.3.3.2. Các toán tử số học 17
1.3.3.3. Toán tử tăng giảm 18
1.3.3.4. Các toán tử quan hệ và các toán tử Logic 19
1.3.3.5. Toán tử ? (toán tử điều kiện) 21
1.3.3.6. Các toán tử Bitwise 21
1.3.3.7. Chuyển đổi kiểu dữ liệu trong các biểu thức 23
1.3.3.8. Ép kiểu 23
1.3.3.9. Cách viết tắt 24
1.3.3.10. Một số hàm toán học 24
CHƯƠNG 2 CÁC CẤU TRÚC ĐIỀU KHIỂN **26**
2.1. CẤU TRÚC CHỌN **26**
2.1.1. Câu lệnh if .. else 26
2.1.1.1. if 26
2.1.1.2. if … else 27
2.1.1.3. Các lệnh if … else lồng 27
2.1.2. Câu lệnh switch .. case 30
2.2. CẤU TRÚC LẶP **32**
2.2.1. Câu lệnh while 32
2.2.2. Câu lệnh do .. while 34
2.2.3. Câu lệnh for 35
2.3. BÀI TẬP MINH HOẠ **37**
CHƯƠNG 3 HÀM **38**
3.1. KIẾN THỨC CƠ BẢN **38**
3.1.1. Khai báo và định nghĩa 38
3.1.2. Vai trò của hàm 41
3.1.3. Giá trị trả về 42
3.1.4. Khai báo prototype 43
3.1.5. Phạm vi của hàm 44

3.2. BIẾN TOÀN CỤC, BIẾN CỤC BỘ, BIẾN TĨNH 45
3.2.1. Biến toàn cục 45
3.2.2. Biến cục bộ 46
3.2.3. Biến tĩnh 47
3.3. TRUYỀN THAM SỐ 48
3.3.1. Truyền tham trị 48
3.3.2. Biến tham chiếu 49
3.3.3. Truyền tham chiếu 50
3.4. ĐỆ QUY 52
3.5. HÀM CHỒNG 54
3.6. HÀM MẪU 56
3.7. BÀI TẬP MINH HOẠ 58
CHƯƠNG 4 KIỂU MẢNG 59
4.1. KIỂU MẢNG 59
4.2. MẢNG MỘT CHIỀU 59
4.2.1. Giới thiệu kiểu mảng một chiều 59
4.2.2. Các kỹ thuật cơ bản 60
4.2.2.1. Khởi tạo 60
4.2.2.2. Cách truy xuất mảng một chiều 61
4.2.2.3. Truyền mảng một chiều vào hàm 62
4.2.2.4. Các bài toán cơ bản 64
Tìm kiếm tuần tự 64
Tìm kiếm nhị phân 65
Sắp xếp 66
4.3. MẢNG NHIỀU CHIỀU 69
4.3.1. Giới thiệu mảng 2 chiều 69
4.3.2. Các kỹ thuật cơ bản 70
4.3.2.1. Khởi tạo 70
4.3.2.2. Cách truy xuất 71
4.3.2.3. Truyền mảng 2 chiều vào hàm 72
4.3.3. Các bài toán cơ bản 74
4.4. Mảng nhiều chiều 75
4.5. KIỂU CON TRỎ 75
4.5.1. Con trỏ và địa chỉ 75
4.5.2. Con trỏ và toán tử new 78
4.5.3. Con trỏ và toán tử delete 79
4.5.4. Mảng động 80
4.6. CON TRỎ VÀ THAM SỐ CỦA HÀM 83
4.6.1. Tham số là kiểu dữ liệu khác kiểu mảng 83
4.6.2. Tham số là kiểu dữ liệu mảng một chiều: 84
4.6.3. Tham số là kiểu dữ liệu mảng hai chiều 85
4.7. CON TRỎ HÀM 86
4.7.1. Khai báo và sử dụng 86
4.7.2. Truyền hàm vào hàm 89
4.8. BÀI TẬP MINH HOẠ 90
CHƯƠNG 5 KIỂU CHUỖI 92
5.1. KIỂU KÝ TỰ 92
5.2. HẰNG KÝ TỰ - HẰNG CHUỖI 92
5.3. KIỂU CHUỖI 93

5.4. CÁC HÀM NHẬP, XUẤT CHUỖI 94
5.4.1. cout và toán tử << 94
5.4.2. cin và toán tử >> 95
5.4.3. cin và getline() 95
5.4.4. cin và get() 96
5.4.5. Nhập số và chuỗi 98
5.4.6. Một số hàm xử lý trên chuỗi 99
5.4.6.1. strlen 99
5.4.6.2. strcpy 100
5.4.6.3. strcat 100
5.4.6.4. strcmp 101
5.5. CON TRỎ VÀ CHUỖI 101
5.6. BÀI TẬP MINH HOẠ 104
CHƯƠNG 6 KIỂU DỮ LIỆU TỰ ĐỊNH NGHĨA 105
6.1. KIỂU CẤU TRÚC 105
6.1.1. Cú pháp khai báo 105
6.1.2. Khởi tạo 107
6.1.3. Cách truy xuất các trường 107
6.1.4. Phép gán trên cấu trúc 108
6.2. CON TRỎ VÀ KIỂU CẤU TRÚC 109
6.3. TRUYỀN THAM SỐ KIỂU CẤU TRÚC 110
6.4. UNION 112
6.5. ENUMERATION 114
6.6. TYPEDEF 117
6.7. BÀI TẬP MINH HOẠ 118
CHƯƠNG 8 TỆP – FILE 119
8.1. KHÁI NIỆM TỆP 119
8.2. CÁC KỸ THUẬT TRÊN TẬP TIN 120
8.2.1. Ghi dữ liệu 120
8.2.2. Đọc dữ liệu 122
8.3. MỘT SỐ PHƯƠNG THỨC CỦA LỚP IFSTREAM VÀ OFSTREAM . 126
8.3.1. int is_open() 126
8.3.2. int eof() 127
8.3.3. Phương thức di chuyển con trỏ tệp trên dòng xuất 128
8.3.4. Một số phương thức khác 129
8.4. ĐỌC GHI ĐỒNG THỜI TRÊN TỆP 130
8.5. BÀI TẬP ỨNG DỤNG 135

PHẦN MỞ ĐẦU

Với ngôn ngữ lập trình Pascal, chúng ta đã có thể xây dựng được rất nhiều ứng dụng với ngôn ngữ lập trình cấp cao này. Tuy nhiên, những ứng dụng này không lớn, không đáp ứng được những nhu cầu của con người vào thời điểm hiện nay. Một ứng dụng hiện nay được gắn kết từ rất nhiều thành phần khác nhau, có thể độc lập với nhau và do nhiều người viết. Các chương trình này hầu hết được viết theo phương pháp lập trình hướng đối tượng. Một trong những ngôn ngữ lập trình hướng đối tượng phổ dụng là C++.

Môn học này nhằm mục đích cung cấp cho người học những kiến thức cơ bản về cách sử dụng ngôn ngữ lập trình C++. Đây là những kiến thức rất cần thiết để sau này người học tiếp cận với phương pháp lập trình hướng đối tượng với công cụ là C++. Nội dung của chương trình học như sau:

CHƯƠNG 1: MỘT SỐ KIỂU DỮ LIỆU CƠ BẢN

1. Giới thiệu chung về ngôn ngữ lập trình C và C++
2. Thành phần cơ bản của ngôn ngữ lập trình C++
3. Các kiểu dữ liệu cơ bản và các toán tử

CHƯƠNG 2: CÁC CẤU TRÚC ĐIỀU KHIỂN

1. Cấu trúc chọn
2. Cấu trúc lặp

CHƯƠNG 3: HÀM

1. Kiến thức cơ bản về hàm
2. Biến cục bộ, biến toàn cục, biến tĩnh
3. Truyền tham số: tham chiếu và tham trị
4. Đệ quy
5. Hàm chồng
6. Hàm mẫu (function template)

CHƯƠNG 4: MẢNG VÀ CON TRỎ

1. Mảng một chiều
2. Mảng nhiều chiều
3. Kiến thức cơ bản về con trỏ

4. Con trỏ và các kiểu dữ liệu phức
5. Con trỏ và tham số của hàm
6. Con trỏ hàm

CHƯƠNG 5: KIỂU CHUỖI

1. Kiểu ký tự
2. Hằng ký tự - hằng chuỗi
3. Kiểu chuỗi
4. Các hàm nhập, xuất chuỗi

CHƯƠNG 6: KIỂU DỮ LIỆU TỰ ĐỊNH NGHĨA

1. Kiểu cấu trúc
2. Union
3. Enumeration
4. Typedef

CHƯƠNG 7: TỆP – FILE

1. Khái niệm tệp
2. Các kỹ thuật trên tập tin
3. Một số phương thức của lớp ifstream và ofstream
4. Đọc ghi đồng thời trên tệp

CHƯƠNG 1 MỘT SỐ KIỂU DỮ LIỆU CƠ BẢN

1.1. GIỚI THIỆU CHUNG VỀ NGÔN NGỮ LẬP TRÌNH C VÀ C++

Ngôn ngữ lập trình C có nguồn gốc từ ngôn ngữ BCPL, được Dennis Ritchie và Brian Kernighan phát triển lần đầu tiên vào năm 1972 tại phòng thí nghiệm Bell thuộc hãng AT&T - Mỹ nhưng nó chính thức ra đời vào năm 1978 khi cuốn "Ngôn ngữ lập trình C" của Dennis Ritchie và Brian Kernighan được xuất bản.

Ngôn ngữ C là ngôn ngữ lập trình cấu trúc có đặc điểm:

- Thể hiện các đặc trưng của ngôn ngữ lập trình bậc cao nhưng có khả năng của các ngôn ngữ lập trình bậc thấp.
- Súc tích và cô đọng.
- Chương trình viết bằng ngôn ngữ lập trình C ngắn gọn, cấu trúc rõ ràng.
- Thư viện các hàm mẫu của ngôn ngữ lập trình C rất phong phú.

Ngôn ngữ lập trình C++ ra đời bởi Bjarne Stroustrup (1983 – 1985) cùng với sự phát triển của phương pháp lập trình hướng đối tượng của thập niên 80. Ngôn ngữ C ++ như là phiên bản mở rộng của C, được thiết kế để hỗ trợ lập trình hướng đối tượng và hỗ trợ toàn bộ ngôn ngữ C.

1.2. CÁC THÀNH PHẦN CƠ BẢN CỦA NGÔN NGỮ LẬP TRÌNH C++

1.2.1. Tập ký tự dùng trong ngôn ngữ lập trình C++

Mọi ngôn ngữ lập trình đều được xây dựng từ một bộ ký tự nào đó. Các ký tự lại được nhóm lại theo nhiều cách khác nhau để tạo nên các từ. Các từ lại được liên kết với nhau theo một quy tắc nào đó tạo nên các câu lệnh. Một chương trình bao gồm nhiều câu lệnh và thể hiện một thuật toán để giải một bài toán nào đó. Ngôn ngữ lập trình C++ được xây dựng trên bộ ký tự sau:

- 26 chữ cái: A B C .. Z
- 26 chữ cái: a b c .. z
- 10 chữ số: 0 1 2 .. 9
- Các ký tự toán học: + - * / ( )
- Ký tự gạch nối: _

- Dấu cách (space)
- Các ký tự khác: . , : ; [ ] { } ! \ | & # % $ …

1.2.2. Từ khoá

Từ khoá là những từ được sử dụng để khai báo các kiểu dữ liệu, để viết các toán tử và các câu lệnh. Dưới đây liệt kê các từ khoá của C++:

break, **char**, **const**, **continue**, **case**, **default**, **do**, **double**, **else**, **enum** , **float**, **for**, **int**, **interrupt**, **if**, **long**, **private**, **public**, **return**, **sizeof**, **static**, **struct**, **switch**, **unsigned**, **while**, **typedef**, **union**, **void** …

Từ khoá không được dùng để đặt tên biến, hằng, hay tên hàm. Ngoài ra khi sử dụng các từ khoá ta phải viết thường.

1.2.3. Kiểu dữ liệu

Có 4 kiểu dữ liệu cơ bản trong C++: **char**, **Int**, **float**, **double**.

- Kiểu **char** (1 byte): biểu diễn 1 ký tự thuộc ASCII hoặc một số nguyên từ 0 đến 255

Ví dụ 1.1

0 → 048

A→ 065

a → 097

- Kiểu **int**: biểu diễn các số nguyên.
- Kiểu **float** : biểu diễn các số thực.
- Kiểu **double** : biểu diễn các số thực.

Kiểu	Phạm vi	Kích thước (byte)
Char	0..255	1
Int	-32768..32767	2
float	3.4e – 38..3.4e + 38	4
double	1.7e – 308 .. 1.7e + 308	8

1.2.4. Biến

Biến là một phần bộ nhớ được đặt tên và được dùng để giữ một giá trị mà có thể được sửa bởi chương trình.

Cú pháp khai báo biến: **dữ_liệu**: <danh sách tên biến>;

Ví dụ 1.2 **int** a; // ta vừa khai báo một biến có tên là a, thuộc kiểu số nguyên

Khi sử dụng biến cần lưu ý những điểm sau:

- Phải khai báo biến trước khi sử dụng
- Tên của những biến khác nhau thì khác nhau
- Dung lượng bộ nhớ dành cho các biến khác nhau tuỳ thuộc vào kiểu dữ liệu của biến đó.
- Các biến có thể được khai báo ở 3 chỗ: trong hàm, trong định nghĩa của các tham số hàm, và bên ngoài tất cả các hàm.

1.2.5. Cách đặt tên

Tên của các biến, hàm và những đối tượng được định nghĩa bởi người sử dụng được gọi là các identifier và có độ dài bất kỳ.

Tên được đặt theo quy tắc sau:

- Tên là một dãy các ký tự bao gồm chữ cái, chữ số hoặc gạch nối. Ký tự đầu tiên của tên phải là chữ cái hoặc gạch nối.
- Tên không được trùng với các từ khoá.
- Tên là duy nhất không được trùng nhau.
- Phân biệt chữ hoa và chữ thường.

Ví dụ 1.3

1ten: không đúng quy tắc

Abc1: đúng quy tắc

_mabc : đúng quy tắc

Mabc và mabc: Là hai tên khác nhau, có thể đặt tên cho hai biến trong cùng một chương trình.

1.2.6. Lời chú thích

Trong chương trình lời chú thích là một thành phần quan trọng, giúp cho đoạn chương trình của chúng ta dễ hiểu hơn, có ích trong quá trình lập trình hoặc bảo trì. Ta sử dụng các ký hiệu // hoặc /* .. */ để ghi lời chú thích. Với ký hiệu // lời chú thích chỉ được viết trên một dòng và bắt đầu với ký hiệu //. Ký hiệu /* để mở đầu đoạn chú thích và kết thúc bằng ký hiệu */.

Ví dụ 1.4

```
/*  lơi chu thich cua chung ta
    Co the trai dai tren nhieu dong */
//  loi chu thich chi o tren mot dong
```

1.2.7. Hằng

Hằng là các giá trị cố định mà chương trình không thể thay đổi. Hằng có thể là số, ký tự, chuỗi.

- Hằng kiểu số:
  - <số >: viết theo hệ 10
  - 0x<số>: viết theo hệ 16

Ví dụ 1.5

```
22
-22.14
0x16 // có giá trị là 22 của hệ thập phân
```

- Hằng kiểu ký tự: Viết trong hai dấu nháy đơn ‘ ‘ Giá trị của hằng chính là mã ASCII của chữ.

Ví dụ 1.6

```
'A' = 65;
'd' = 100,
```

- Hằng kiểu chuỗi: đặt trong dấu nháy kép (" ").

Ví dụ 1.7 "Lơp Hoc"

1.2.8. Thư viện

Một số câu lệnh cơ bản được gom lại theo một tính chất nào đó tạo thành thư viện để người lập trình dễ dàng sử dụng hơn. Để sử dụng được các lệnh đó, ta phải khai báo sử dụng. *Cú pháp* như sau: **#include** <tenthuvien.h>

Ví dụ 1.8 **#include** <iostream.h>

1.2.9. Lệnh nhập - xuất

Lệnh là những hàm của ngôn ngữ lập trình mà thông qua đó người lập trình giao tiếp với máy tính, điều khiển máy tính làm những công việc nhất định. Kết thúc mỗi lệnh trong ngôn ngữ C/C++ phải có dấu **;** .

Hàm nhập và xuất là hai hàm cơ bản nhất để ta có thể lấy thông tin từ bên ngoài vào chương trình và xuất thông tin của chương trình ra bên ngoài. Để nhập hoặc xuất chúng ta dùng toán tử nhập của đối tượng **cin**, toán tử xuất của **cout** trong thư viện iostream.h.

Cú pháp như sau:

xuất: **cout** << (hằng hoặc biến) << (hằng hoặc biến);

nhập: **cin** >> (biến) >> (biến);

Ví dụ 1.9

cout << “ham xuat”; // màn hình sẽ xuất hiện dòng: ham xuat

cin >> a; /* sẽ nhập vào biến a một giá trị nào đó mà chúng ta đã nhập từ bàn phím.*/

Ngoài ra chúng ta vẫn có thể sử dụng hai hàm nhập xuất scanf và printf thuộc thư viện stdio.h kế thừa từ ngôn ngữ lập trình C.

1.2.10. Cấu trúc của một chương trình C++

#include < Thuvien.h> những khai báo, những chỉ thị tiền xử lý.

#define ..../* Các khai báo kiểu dữ liệu, hằng */

```
typedef .... //định nghĩa các kiểu dữ liệu mới

//khi báo các biến toàn cục


// khai báo tiêu đề hàm (prototype)

void main ()
{
   // các dòng lệnh của chương trình chính
}
//định nghĩa cụ thể của các hàm
```

Ví dụ 1.10 Viết chương trình xuất ra dòng "chuong trinh C++".

```
#include <iostream.h>

void main()

{

    cout << "chuong trinh C++";
}
```

1.3. CÁC KIỂU DỮ LIỆU CƠ BẢN VÀ CÁC TOÁN TỬ

1.3.1. Kiểu số nguyên

1.3.1.1. Dữ liệu kiểu char, int

C++ cung cấp các kiểu dữ liệu **char**, **int** để biểu diễn số nguyên. Tuỳ theo trình biên dịch mà kiểu **int** có kích thước thay đổi. Đối với Borland C++, kiểu **int** được biểu diễn bởi 2 byte, có giá trị từ -32768 (-2^{15}) đến 32767 ($2^{15} – 1$). Kiểu **char** được biểu diễn bởi 1 byte, có giá trị từ -128 (-2^7) đến 127 ($2^7 – 1$). Ngoài ra chúng ta có thể thay đổi kiểu số nguyên cơ bản bằng cách thêm các từ khoá sau: **signed**, **unsigned**, **long**, **short**.

Kiểu	Kích thước	Miền giá trị
Int	2	-32768 đến 32767
Unsigned **int**	2	0 đến 65535

Short **int**	2	-32768 đến 32767
Signed **int**	2	-32768 đến 32767
Unsigned short **int**	2	0 đến 65535
Long **int**	4	-2137383648 (-2^{31}) đến 2147483647 ($2^{31} - 1$)
Unsigned long **int**	4	0 đến 4294967295 ($2^{32}-1$)
Char	1	-128 đến 127
unsigned **char**	1	0 đến 255

Kiểu dữ liệu **char** là kiểu dữ liệu không chỉ biểu diễn một số nguyên mà nó còn dùng để biểu diễn một ký tự. Mã các ký tự này trong bảng mã ASII đúng bằng giá trị số nguyên mà nó biểu diễn.

Cú pháp khai báo một biến thuộc kiểu số nguyên như sau:

int *tên_biến_1, tên_biến_2, ..., tên_biến_n;*

char *tên_biến_1, tên_biến_2, ..., tên_biến_n;*

Ví dụ 1.11 Khai báo biến tuổi thuộc kiểu số nguyên.

int tuoi;

char tuoi1;

1.3.1.2. Phương pháp biểu diễn số nguyên

Con người quen sử dung các con số với hệ cơ số thập phân, nhưng đối với máy tính lại sử dụng hệ cơ số nhị phân. *Ví dụ* số 5 trong cơ số 10 chuyển thành số 0101 trong cơ số 2.

Để biểu diễn số nguyên âm người ta có thể biểu diễn dưới dạng dấu lượng, bù 1, bù 2, hoặc quá n. Trong đó số bù 2 được sử dụng phổ biến nhất

Công thức để xây dựng số bù 2 của một số N có n ký số là: $r^n - N$ (r là cơ số của số N).

Đối với số nhị phân, ta có thể lấy số bù 2 của một số bằng cách đảo các bít của số đó, và cộng thêm 1.

Ví dụ 1.12 Lấy bù 2 của số 0101

Cách 1: r^n = 10000 – 0101 = 1011 (2^4 = 16 = 10000)

Số bù 2 của 0101 là 1011

Cách 2: Đảo bít của số 0101 là 1010

1010 + 1 = 1011

Số bù 2 của 0101 là 1011

Chu ý: Nếu số 0 được biểu diễn bởi 4 bit tức 0000 thì số bù 2 của số không cũng là 0000. Vì vậy ta thấy miền giá trị âm của một kiểu dữ liệu bao giờ cũng biểu diễn nhiều hơn miền giá trị dương một con số.

1.3.2. Kiểu số thực

1.3.2.1. Kiểu float và double

C++ cung cấp cho chúng ta hai loại kiểu dữ liệu để biểu diễn số thực đó là **float** và **double.**

Miền giá trị và kích thước của **float** và **double**:

Kiểu	Phạm vi	Kích thước (byte)
Float	3.4e – 38 đến 3.4e + 38	4
double	1.7e – 308 đến 1.7e + 308	8

Các từ khoá **unsigned**, **signed**, **short**, **long** không áp dụng để thay đổi kích thước và miền giá trị của biến kiểu **float** hay **double**. Một vài trường hợp có thể dùng **unsigned** cho kiểu **double** nhưng không được khuyến khích.

Cú pháp khai báo số thực như sau:

float *tên_biến_1, tên_biến_2, ..., tên_biến_n;*

double *tên_biến_1, tên_biến_2, ..., tên_biến_n;*

Ví dụ 1.13

float diemtb; **double** delta;

1.3.2.2. Phương pháp biểu diễn số thực

Để biểu diễn các số rất lớn hoặc rất bé người ta dùng một cách biểu diễn số gọi là số chấm động. Biểu diễn số chấm động gồm 4 phần: dấu, định trị, phần mũ, và cơ số. *Ví dụ* 5.13 được biểu diễn như sau: $+ 0.513 * 10^{-2}$.

Trong máy tính cơ số mặc nhiên là cơ số 2 nên không cần lưu. Chỉ cần 1 bit để lưu dấu (thường 0 biểu diễn số dương, 1 biểu diễn số âm). Ngoài ra còn phần lưu phần định trị và phần mũ. Tuỳ thuộc kiểu dữ liệu mà người ta quy định số lượng bit cho từng phần. Với kiểu dữ liệu 32 bít, người ta thường lưu như sau, bit 31 làm bit dấu, phần mũ dài 8 bit (là một số quá 127 tức biểu diễn giá trị từ -127 đến 128), phần định trị là một số nhị phân chuẩn hoá có dạng 1.F, do đó chỉ cần lưu F.

Ví dụ 1.14

Số thập phân 4.125 tương đương nhị phân là 100.0001. Để chuẩn hoá, dấu chấm dịch về trái 2 vị trí cho số chuẩn hoá là 1.000001 và phần mũ có trị là +2 (tức dãy bit của phần mũ có giá 129 = 127 + 2). Dạng lưu trữ sau cùng như sau:

31	30 23	22 0
0	10000010	00000100000000000000

Số thập phân 209.8125 tương đương nhị phân là 11010001.1101. Để chuẩn hoá, dấu chấm dịch về trái 7 vị trí cho số chuẩn hoá 1.10100011101 và phần mũ có trị là +7 (tức dãy bit của phần mũ có giá trị không dấu là 134 = 127 + 7). Dạng lưu trữ sau cùng như sau:

31	30 23	22 0
0	10000110	10100011101000000000

Lưu ý:

Để chuyển đổi một số thập phân sang nhị phân ta lấy phần nguyên lần lượt chia cho 2 để lấy phần dư. Phần thập phân còn lại lần lượt nhân cho hai và lấy phần nguyên.

Ví dụ 1.15

Chuyển đổi 41.6875 của hệ thập phân sang hệ nhị phân

41	
20	1
10	0
5	0
2	1
1	0
0	1

0.6875
2
1.3750
2
0.7500
2
1.5000
2
1.0000

$(41)_{10} = (101001)_2$ $(0.6875)_{10} = (1011)_2$

$(41.6875)10 = (101001.1011)_2$

1.3.3. Các toán tử

1.3.3.1. Toán tử gán

Dạng tổng quát của toán tử gán là:

Ten_bien = bieu_thuc;

Biểu thức có thể đơn giản là một số hay phức tạp là một biểu thức gồm nhiều phép tính. Ngôn ngữ C++ dùng một dấu = làm toán tử gán. Phần bên trái của toán tử gán không thể là một hằng hay một hàm số.

Ví dụ 1.16

```
int a;
a = 5;
int b;
b = a * 2; // b có giá trị là 10
```

Lưu ý:

Khi thực hiện một phép gán giữa hai biến thuộc hai kiểu dữ liệu khác nhau (dữ liệu kiểu số) thì có sự hoán đổi kiểu xảy ra, giá trị của biến bên trái được gán bằng với giá trị của biến vế phải nhưng kiểu dữ liệu của giá trị này chuyển đổi thành kiểu của vế trái.

Ví dụ 1.17:

```
int  cao;
unsigned char thap;
thap = 0;
cao = 4095; // cao =  0x0fff;     (dòng lệnh 1)
thap = cao;                 (dòng lệnh 2)
cao = thap;                 (dòng lệnh 3)
```

Nhận xét:

Sau dòng lệnh 2, thap có giá trị là 255. Ta thấy thap có giá trị khác với biến cao. Lúc này, khi sử dụng toán tử gán bằng giữa 2 biến cao và thap, trình biên dịch sẽ lấy giá trị của các bit thấp của biến cao gán cho biến thap hay nói cách khác biến thap phản ánh giá trị các bit thấp của biến cao.

Biểu diễn bit của các biến:

cao: 0000 1111 1111 1111 (0x0fff)

thap: 0000 0000 → 1111 1111 (0xff)

Sau dòng lệnh 2 biến thap có giá trị là 255, sau dòng lệnh 3, biến cao có giá trị là 255, không có gì bất thường xảy ra.

Biểu diễn bit của các biến:

thap: 1111 1111 (0xff)

cao: 0000 1111 1111 1111 (0x0fff) → 0000 0000 1111 1111 (0x00ff)

```
int a, c;
double b;
a = 1;        (dòng 1)
b = 2.2; (dòng 2)
c = b;        (dòng 3)
b = a;        (dòng 4)
```

Nhận xét:

Sau dòng lệnh 1: a có giá trị là 1

Sau dòng lệnh 2: b có giá trị là 2.2;

Sau dòng lệnh 3: c có giá trị 2 thay vì 2.2

Sau dòng lệnh 4: b có giá trị 1.0 thay vì 1

C++ cho phép bạn gán nhiều biến cho cùng một giá trị bằng cách sử dụng phép gán chồng trong một phát biểu đơn.

Ví dụ 1.18 x = y = z = 0;

1.3.3.2. Các toán tử số học

Các toán tử +, - , * , / tương tự các phép toán cộng, trừ, nhân và chia trong toán học.

Khi áp dụng toán tử / cho một số nguyên thì toán tử / trở thành phép chia lấy phần nguyên (div). Để thực hiện phép toán chia lấy phần dư ta dùng toán tử %.

Chú ý:

- Toán tử % không thể sử dụng cho kiểu số thực có dấu chấm động.
- Toán tử / được sử dụng với kiểu số thực sẽ thực hiện phép chia bình thường.

Ví dụ 1.19

```
int a,b,c;
float d,e,f;
    a =  5;
    b = 3;
    c =  a / b; // c = 1;
    c = a % b; // c = 2
    d = 5.2;
    e = 2;
```

f = d /e; // f = 2.6

f = d % e; // lổi xãy ra

1.3.3.3. Toán tử tăng giảm

Toán tử tăng và toán tử giảm là 2 toán tử đặc trưng của ngôn ngữ C++. Các ngôn ngữ ra đời sau cũng đưa thêm các toán tử này vào.

Toán tử giảm -- có nghĩa là giảm đi một đơn vị.

Ví dụ 1.20

x = 5;

x--; // x bây giờ có giá trị bằng 4

Toán tử tăng ++ có nghĩa là tăng một đơn vị

Ví dụ 1.21

x = 5;

x ++; // x bây giờ có giá trị bằng 6

Toán tử --, ++ có thể là tiền tố hoặc hậu tố toán hạng. *++x;* hoặc *x++;* đều có *Ý nghĩa* giống nhau. Tuy nhiên, có sự khác nhau giữa dạng tiền tố và hậu tố khi sử dụng các toán tử này trong 1 biểu thức. Khi một toán tử tăng hay giảm đứng trước toán hạng của nó, C++ thực hiện việc tăng hay giảm trước khi lấy giá trị của toán hạng đứng trong biểu thức. Nếu toán tử đi sau toán hạng, C++ lấy giá trị của toán hạng trước khi tăng hay giảm nó.

Ví dụ 1.22

x = 10;

y = ++x;

x thực hiện tăng giá trị trước tức x = 11, y lúc này có giá trị 11

x = 10;

y = x ++;

y có giá trị là 10, sau đó x mới thực hiện tăng lên một đơn vị và bằng 11.

Dưới đây là thứ tự ưu tiên của các toán tử số học:

Cao nhất: ++ --

* / %

Thấp nhất: + -

Các toán tử cùng mức ưu tiên được tính bằng cách biên dịch từ trái sang phải. Dấu ngoặc đơn được sử dụng để thay đổi thứ tự tính toán. Các cặp dấu ngoặc đơn sẽ xác lập lại thứ tự ưu tiên của các toán tử.

Chú ý:

Hầu hết các trình biên dịch C/C++ tạo mã đối tượng rất nhanh và hiệu quả cho các việc tăng và giảm mã lệnh vì thế tốt hơn so với việc tạo ra bởi việc sử dụng biểu thức gán tương đương. Vì nguyên nhân này, chúng ta nên dùng các toán tử tăng, giảm khi có thể.

1.3.3.4. Các toán tử quan hệ và các toán tử Logic

Toán tử quan hệ là toán tử chỉ đến mối quan hệ giữa hai giá trị. Toán tử logic là chỉ đến cách mà các mối quan hệ đó được kết nối.

Ý tưởng chính của các toán tử quan hệ và logic là đúng hoặc sai. Trong Pascal, đúng, sai là hai giá trị tương ứng true, false của kiểu dữ liệu boolean. Trong C/C++ mọi giá trị khác 0 được gọi là đúng, giá trị 0 là sai. Các biểu thức sử dụng các toán tử quan hệ và logic sẽ trả về 0 nếu sai và 1 nếu đúng.

Có 6 toán tử quan hệ như sau:

>	Lớn hơn
<	Nhỏ hơn
>=	Lớn hơn hoặc bằng
<=	Nhỏ hơn hoặc bằng
==	Bằng
!=	Khác (không bằng)

Ví dụ 1.23

10 > (10 + 1) → *Kết quả* là sai

10 >= 10 – 1 → *Kết quả* là đúng

Có 3 toán tử logic như sau:

&&	Và (AND)
||	Hoặc (OR)
!	Phủ định (NOT)

Mặc dù C++ không có toán tử XOR, nhưng XOR có thể được tạo ra bằng cách lấy *Kết quả* của toán tử logic OR and với NOT của kết quả của toán tử logic AND.

Bảng chân trị như sau:

P	Q	!P	P && Q	P || Q	P XOR Q	!(P&&Q)	(P || Q) && !(P&&Q)
0	0	1	0	0	0	1	0
0	1	1	0	1	1	1	1
1	0	0	0	1	1	1	1
1	1	0	1	1	0	0	0

Ví dụ 1.24

10 > 5 && !(5 > 6) || 3 <= 4 cho ra *Kết quả* đúng

Các toán tử quan hệ và logic đều có độ ưu tiên thấp hơn theo thứ tự so với các toán tử số học. Thứ tự ưu tiên của các toán tử quan hệ và logic:

Cao nhất !

> >= < <=

== !=

&&

Thấp nhất ||

Cũng như các biểu thức số học, bạn có thể sử dụng các cặp dấu ngoặc đơn để thay thế thứ tự thông thường của một biểu thức quan hệ và logic.

Ví dụ 1.25

!0 && 0 || 0 cho ra kết quả sai.

!(0 && 0) || 0 cho ra kết quả đúng

1.3.3.5. Toán tử ? (toán tử điều kiện)

Dạng thức chung của toán tử ? là:

<điều kiện>? <giá trị 1>: <giá trị 2>

Nếu biểu thức điều kiện trả về giá trị đúng (tức khác 0), giá trị 1 sẽ là giá trị của biểu thức, ngược lại giá trị 2 sẽ là giá trị của biểu thức.

Ví dụ 1.26

```
X = 10;
Y =  X > 9 ? 100 : 200;
```

Sau đoạn lệnh trên Y có giá trị là 100.

```
X = 10;
Y =  X < 9 ? 100 : 200;
```

Sau đoạn lệnh trên Y có giá trị 200.

1.3.3.6. Các toán tử Bitwise

Không giống như các ngôn ngữ khác, C++ cung cấp đầy đủ các toán tử bitwise. Các toán tử này thực hiện các thao tác kiểm tra, gán hay thay đổi các bit thực sự trong một byte của word hay các kiểu dữ liệu **int**, **char**. Các toán tử này không thể được sử dụng với các kiểu dữ liệu **float**, **double**, hay các kiểu phức tạp khác.

Có 6 toán tử bitwise như sau:

&	Và (and)
|	Hoặc (or)
^	Xor
~	phủ định (not)
>>	Shift trái
<<	Shift phải

Ví dụ 1.27

```
Char a,b,c;
a = 2; //  0000 0010
b = 3; // 0000 0011
c = a & b; // c = 0000 0010
c = a | b ;  // c = 0000 0011
c = ~a ;  // c = 1111 1101
c = a ^ b; // c =  0000 0001
```

Các toán tử >> và << sẽ dịch chuyển tất cả các bit trong một biến qua phải hoặc qua trái. Dạng thức chung của câu lệnh dịch phải hoặc trái là:

Ten_bien << số bit cần dịch

hoặc

Ten_bien >> số bit cần dịch

Ví dụ 1.28

```
a = 132; // 1000 0100
b = 130 ;// 1000 0010
c = a << 2;  // c = 0001 0000
c = b >> 2; // c = 0010 0000
```

Nhận xét:

Sau mỗi phép dịch phải n bit, n bit đầu ở phía phải bị xoá, số lượng bit còn lại được dịch sang phải n bit, và n bit cuối chuyển thành 0. Tương tự cho phép dịch trái. Cần lưu ý là phép dịch bit này sẽ không quay vòng.

Chú ý:

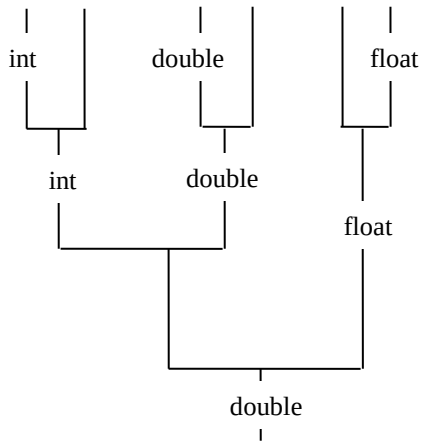
Các toán tử bitwise có thể sinh ra các giá trị khác 0 hay 1, trong khi các toán tử logic luôn sinh ra giá trị 0 hoặc 1.

1.3.3.7. Chuyển đổi kiểu dữ liệu trong các biểu thức

Khi các hằng và các biến của các kiểu dữ liệu khác nhau được trộn lẫn trong một biểu thức, tất cả chúng sẽ đươc chuyển đổi thành một kiểu dữ liệu chung. Chương trình dịch C++ chuyển đổi tất cả các hệ số thành kiểu dữ liệu của hệ số có khả năng biểu diễn lớn nhất gọi là nâng kiểu. Trước tiên, tất cả các giá trị **char** và **short** được chuyển đổi thành **int**. Sau đó các quá trình chuyển đổi khác sẽ được hoàn tất.

Ví dụ 1.29

```
char ch;
int i;
float f;
double b,kq;
kq = (ch / i)   +   (f * d )  –   (f + i);
```



1.3.3.8. Ép kiểu

Một biểu thức có thể bị ép buộc theo một kiểu được chỉ định sẵn.

Cú pháp: (kiểu) <biểu_thức>

Trong đó kiểu là một kiểu dữ liệu hợp lệ của C++.

Ví dụ 1.30

```
int a;
float x = 6.4;
a = 4 % (int) (x / 2);   // a có giá trị bằng 1
```

1.3.3.9. Cách viết tắt

Trong C++ bạn có thể viết tắt khi biểu thức có phép gán.

<biến> = <biến> <toán tử> <biểu thức>

chuyển thành

<biến> <toán tử> = <biểu thức>

Ví dụ 1.31

```
x = x + 10;  //tương đương:      x += 10;
x = 10;
y = 5;
x -= y*2;  // tương đương x = x – y*2
```

1.3.3.10. Một số hàm toán học

Sau đây là các hàm toán học thuộc thư viện math.h.

- int **abs** (int arg);

  Hàm abs() trả về giá trị tuyệt đối của arg.

- double **acos** (double arg);

  Hàm acos() trả về giá trị arccosin của arg. Tham số trong acos() phải nằm trong khoảng -1 đến 1.

- double **asin** (double arg);

  Hàm asin() trả về giá trị arcsin của arg. Tham số trong asin() phải nằm trong khoảng – 1 đến 1.

- double **atan**(double arg);

  Hàm atan() trả về giá trị arctang của arg.

- double **ceil**(double num);

  Hàm ceil() trả về giá trị số nguyên nhỏ nhất không nhỏ hơn num.

- double **cos** (double arg);

  Hàm cos() trả về giá trị cosin của arg. arg có đơn vị tính là radian.

- double **exp** (double arg);

  Hàm exp() trả về giá trị e mủ arg.

- double **floor** (double num);

  Hàm floor() trả về giá trị số nguyên lớn nhất không lớn hơn num.

- double **log**(double num);

  Hàm log() trả về giá trị logarit tự nhiên cơ số e của num.

- double **log10**(double num);

  Hàm log10() trả về giá trị logarit cơ số 10 cho num.

- double **sin** (double arg);

  Hàm sin() trả về giá trị sin của arg.

- double **sqrt**(double arg);

  Hàm sqrt() trả về giá trị căn bậc hai của num. arg phải có giá trị dương.

- double **tan** (double arg);

  Hàm tan() trả về giá trị tang của arg.

1.5. BÀI TẬP MINH HOẠ

1. Nhập số học sinh của 4 lớp, tính tổng số học sinh và số học sinh trung bình của từng lớp.

2. Nhập vào số tiền gửi tiết kiệm, thời gian gữi (tháng), lãi suất (%) trên một tháng, tính tiền lãi và tổng số tiền có được sau thời gian gữi đó (Không cộng lãi suất tháng trước với tiền gốc để tính lãi của các tháng sau).

3. Nhập vào 2 cạnh của hình chữ nhật, tính diện tích chu vi của hình đó.

4. Nhập vào 2 số, tìm số lớn hơn.

5. Nhập vào điểm của 4 học sinh, tìm điểm lớn nhất.

6. Nhập vào một số, kiểm tra số đó là số chẳn hay lẻ.

7. Nhập vào một số có 2 chữ số ab, xuất ra màn hình số ba.

8. Nhập vào diện tích của hình tròn, tính chu vi của hình đó.

CHƯƠNG 2 CÁC CẤU TRÚC ĐIỀU KHIỂN

Ngôn ngữ lập trình C++ có 3 cấu trúc điều khiển là cấu trúc tuần tự, cấu trúc chọn lựa, cấu trúc lặp. Cấu trúc tuần tự là cấu trúc mà ta thường sử dụng nhất, không cần sử dụng một câu lệnh điều khiển nào của C++, mọi dòng lệnh được tuần tự thực hiện theo thứ tự từ trên xuống dưới. Muốn sử dụng được cấu trúc chọn và cấu trúc lặp ta cần phải sử dụng các câu lệnh điều khiển riêng.

2.1. CẤU TRÚC CHỌN

2.1.1. Câu lệnh if .. else

2.1.1.1. if

Cú pháp

if (< biểu thức điều kiện>)

<lệnh>;

Ý nghĩa

Nếu <biểu thức điều kiện> trả về giá trị đúng (trả về giá trị khác 0), thì <lệnh> sẽ được thực hiện, đồng nghĩa với nếu <biểu thức điều kiện> trả về giá trị sai (trả về giá trị bằng 0), thì <lệnh> sẽ không được thực hiện.

Ví dụ 2.1

```
int a,b;
a = 5;
b = 10;
if ( a == b)
    cout << " a  bang  b";
if (a < b - 2)
    cout << "a nho hon b - 2";
```

<lệnh> có thể là một lệnh đơn hoặc là một khối lệnh. Nếu là một khối lệnh, ta phải dùng 2 dấu { } ở đầu và cuối khối lệnh.

Ví dụ 2.2

```
int a,b,c;
a = 5;
b = 10;
if ( a < b)
{
```

```
        c = a;
        cout << "a co gia tri lon hon b"
    }
```

2.1.1.2. if … else

Cú pháp

```
if (<biểu thức điều kiện>)
    <lệnh1>;
else
    <lệnh2>;
```

Ý nghĩa

Nếu <biểu thức điều kiện> trả về giá trị đúng (trả về giá trị khác 0), thì <lệnh1> sẽ được thực hiện, ngược lại nếu <biểu thức điều kiện> trả về giá trị sai (trả về giá trị bằng 0), thì <lệnh2> sẽ được thực hiện. <lệnh1> và <lệnh2> đều có thể là 1 lệnh đơn hoặc lệnh ghép.

Ví dụ 2.3

```
int a,b;
a  = 4;
b = 5;
if ( a < b)
    cout << "gia tri cua a nho hon b";
else
{
    b -= a;
    cout << " b – a =" << b;
}
```

2.1.1.3. Các lệnh if … else lồng

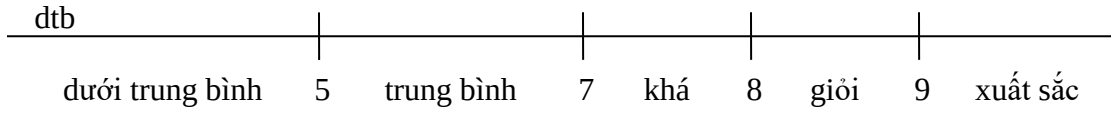
Dạng 1

```
if (<điều kiện 1>)
    <lệnh 1> ;
else
    if (< điều kiện 2>)
        <lệnh 2>;
    else
    …
        if (<điều kiện n>)
            <lệnh n>
        else
            <lệnh n + 1>
```

Ý nghĩa

- Nếu không có biểu thức <điều kiện> nào đúng thì <lệnh n + 1> thực hiện.
- Nếu biểu thức <điều kiện> thứ k đúng thì <lệnh k> được thực hiện.

Ví dụ 2.4




```
float  dtb;
int  toan, ly, hoa;
toan  = 5;
ly = 6;
hoa = 7;
dtb = (float) (toan + ly + hoa)/3;
if ( dtb >= 9)
    cout << " dat loai xuat sac";
else
    if (dtb >= 8)
        cout << "dat loai gioi";
    else
        if (dtb >= 7)
            cout << "dat loai kha";
        else
            if (dtb >= 5)
                cout << "dat loai trung binh";
            else
                cout << "duoi trung binh";
```

Dạng 2

```
If (<điều kiện 1>)
    If (<điều kiện 2>)
        …
        If(<điều kiện n>)
            <lệnh n + 1>;
        Else
            <lênh n >;
    Else
        <lệnh 2>;
```

```
Else
    <lệnh 1>;
```

Ý nghĩa

Nếu <điều kiện 1> đến <điều kiện> thứ k đúng thì thực hiện lệnh thứ k + 1 được thực hiện.

Ví dụ 2.5

```
int toan, ly, hoa;
float dtb;
toan = 5;
ly = 6;
hoa = 7;
dtb = (float) (toan + ly + hoa)/3;
if ( dtb < 9)
    If( dtb <8)
        If (dtb < 7)
            If(dtb < 5)
                cout << "duoi trung binh";
            else
                cout << "dat loai tb";
        else
            cout << "dat loai kha";
    else
        cout << "dat loai gioi";
else
    cout << "dat loai xuat sac";
```

Ví dụ 2.6 Giải phương trình ax + b = 0

- Phân tích:

- Nếu a và b đồng thời bằng 0 thì x có vô số nghiệm.
- Nếu a bằng 0 và b khác không thì x vô nghiệm.
- Nếu a khác không thì x có nghiệm là x = -b/a .

- Chương trình

```
int a,b;
float x;
// đoạn chương trình nhập a
cout << "nhap a:" ;
cin >> a;
```

```
cout <<"\nnhap b:";
cin >> b;
cout << '\n';
if ( a == 0 )
    if ( b == 0)
        cout << "pt vo so nghiem";
    else
        cout << "pt vo so nghiem";
else
{
        x = (float) -b /a;
        cout << "pt co nghiem x =" << x;
}
```

2.1.2. Câu lệnh switch .. case

Cú pháp

```
switch (<mệnh đề>)
{
    case  giá_trị_1:
            <lệnh 1>
            break;
    case giá_trị_2:
            <lệnh 2>
            break;
    …
    case giá_trị_n:
            <lệnh n>
            break;
    default
        <lệnh default>
}
```

Ý nghĩa

- Nếu <mệnh đề> có giá trị bằng *giá_trị_k* thì <lệnh k> được thực hiện
- Nếu <mệnh đề> có giá trị khác với tất cả các giá trị từ *giá_ trị_1* đến *giá_trị_n* thì <lệnh default> được thực hiện.
- Lệnh **break** là lệnh nhảy, có tác dụng nhảy ra khỏi lệnh **switch**, hay kết thúc lệnh **switch**.

Chú ý

- Các *giá_trị* phải là những hằng số thuộc kiểu dữ liệu nguyên (rời rạc), biến trong <biểu thức> phải thuộc kiểu dữ liệu số nguyên.
- <lệnh> có thể là một lệnh đơn hay một khối lệnh.

Ví dụ 2.7

```
int sodinh;
sodinh = 0;
cout << "nhap so dinh :";
cin >> sodinh;
switch (sodinh)
{
    case 1:
        cout << "la mot diem";
        break;
    case 2:
        cout << "la mot doan thang";
        break;
    case 3:
        cout << "la mot tam giac";
        break;
    default:
        cout << "la da giac co" << sodinh << " dinh";
}
```

Khi giá trị <mệnh đề> bằng với một <giá trị k> nào đó thì <lệnh k> được thực hiện. Nếu sau <lệnh k> không có lệnh **break** thì các lệnh phía sau (<lệnh k+1>, <lệnh k+2> vẫn được thực hiện cho đến khi gặp lệnh **break** hoăc đến cuối lệnh **switch**. Vì vậy, nếu ta muốn ứng với nhiều <giá trị> cùng làm một <lệnh> thì ta chỉ đặt lệnh **break** ở phần <lệnh> ứng với giá trị cuối cùng.

Ví dụ 2.8

```
int ngay;
ngay = 0;
cout << "nhap ngay thu may trong tuan :";
cin >> ngay;
switch (ngay)
{
    case 2:
```

```
            cout << "ngay dau tuan";
            break;
        case 3:
        case 4:
        case 5:
        case 6:
            cout << "ngay giua tuan";
            break;
        case 7:
            cout << "ngay nghi thu nhat";
            break;
        case 8:
            cout << "ngay nghi thu hai";
            break;
        default:
            cout << "so ngay khong hop le!";
    }
```

Chú ý

Lệnh **switch** có thể là một phần của dãy lệnh của một lệnh **switch** ngoài. Dù các *giá_trị* của **switch** trong và ngoài giống nhau cũng không phát sinh lỗi.

2.2. CẤU TRÚC LẶP

Các lệnh lặp (còn được gọi là vòng lặp) cho phép lặp đi lặp lại một đoạn lệnh nào đó cho đến khi một điều kiện nào đó xảy ra. Điều kiện này có thể được đưa ra trước hoặc sau. Trong ngôn ngữ C++ có 3 loại vòng lặp.

2.2.1. Câu lệnh while

Cú pháp

while <điều kiện >

<lệnh>;

Ý nghĩa

- Trong khi thoả mãn <điều kiện> hay biểu thức điều kiện trả về giá trị đúng thì <lệnh> được thực hiện lặp đi lặp lại. Khi biểu thức điều kiện trả về giá trị sai, thì <lệnh> không được thực hiện nữa, chương trình ra khỏi vòng lặp, tiếp tục thực hiện các lệnh tiếp sau vòng lặp **while**.
- <lệnh> có thể là một lệnh đơn, một khối lệnh, hoặc không có lệnh nào.

Ví dụ 2.9

```
#include <iostream.h>
void main()
{
    int n,s;
    n = 10;
    s = 0;
    while ( n > 0)
    {
        s+= n;
        n --;
    }
    cout << "tong cac so tu 1 den 10: " << s;
}
```

Chú ý

Vòng lặp **while** chỉ dừng khi biểu thức <điều kiện> trả về giá trị sai, hay nói cách khác chỉ khi giá trị của biểu thức <điều kiện> thay đổi. Vì vậy trong khối lệnh được lặp lại phải có một lệnh làm thay đổi giá trị của biểu thức <điều kiện>. Trong *Ví dụ* 1 lệnh **n--;** làm thay đổi giá trị n, dẫn đến làm thay đổi biểu thức điều kiện là **n > 0**.

Ví dụ 2.10

```
int n;
n = 2;
while ( n > 1)
    cout << “n lon hon 1”;
```

Nhận xét

- Đoạn chương trình này sẽ lặp vô hạn vì trong đoạn lệnh được lặp lại, không có lệnh làm thay đổi giá trị biểu thức **n > 1.**
- Tuy nhiên, trong thực tế, một số trường hợp đặc biệt người lập trình có thể sử dụng lệnh **break** trong đoạn lệnh được lặp lại để thoát khỏi vòng lặp **while** mà không cần giá trị biểu thức <điều kiện> thay đổi.

Ví dụ 2.11

```
while ( 2 < 3)
{
    cout << "lap lai mot lan";
    break;
}
```

Nhận xét

- Lệnh **cout** << "lap lai mot lan"; chỉ thực hiện một lần duy nhất, mặc dù điều kiện 2 < 3 luôn luôn đúng.
- Trường hợp <lệnh> không có lệnh nào chỉ được sử dụng khi lập trình multithread (đa tiến trình). Trong những trường hợp khác bắt buộc <lệnh> phải có ít nhất một lệnh.

2.2.2. Câu lệnh do .. while

Cú pháp

```
do {
    <lệnh >
}while ( <điều kiện> );
```

Ý nghĩa

Thực hiện <lênh>, sau đó kiểm tra điều kiện. Nếu điều kiện cho giá trị đúng, tiếp tục thực hiện lệnh cho đến khi biểu thức <điều kiện> trả về giá trị sai. Như vậy, <lệnh> ít nhất phải được thực hiện một lần.

Ví dụ 2.12

Tính tổng các số nguyên dương nhỏ hơn 10.

```
int s,n;
s = 0;
n = 10;
do
{
    s += n;
    n = n --;
} while ( n >= 0);
```

Ví dụ 2.13

```
void main()
{
    int  so = 0, du = 0;
    cout << "nhap vao mot so nguyen duong: \n";
    cin >> so;
    cout << "so dao nguoc cua so vua nhap: \n";
    do{
        du = so % 10;
        so = so / 10;
        cout >> du;
    }while (so > 0);
}
```

2.2.3. Câu lệnh for

Cú pháp

for (<biểu thức bắt đầu>; <điều kiện> ; <biểu thức tăng>)

<lệnh>;

Giải thích

- <biểu thức bắt đầu>: Là một lệnh gán để tạo giá trị ban đầu cho biến điều khiển.
- <điều kiện>: Là một biểu thức quan hệ xác định khi nào kết thúc vòng lặp.
- <biểu thức tăng>: Là biểu thức làm thay đổi giá trị biến điều khiển sau mỗi lần lặp lại của vòng lặp.
- <lệnh> có thể là một lệnh hoặc là một khối lệnh.

Ý nghĩa

Câu lệnh trên sẽ thực hiện các bước như sau:

- Bước 1: Thực hiện <biểu thức bắt đầu> để gán giá trị đầu cho biến điều khiển.
- Bước 2: Thực hiện biểu thức điều kiện. Nếu biểu thức <điều kiện> trả về giá trị sai, chương trình sẽ ra khỏi vòng lặp **for**, ngược lại sẽ thực hiện <lệnh>.
- Bước 3: Tính giá trị <biểu thức tăng>, sau đó quay lại bước 2.

Ví dụ 2.14

Tính tổng các số từ 0 đến 10

```
int k,s;
s = 0;
for (k = 0; k <=10; k++ )
    s += k;
cout << "tong cac so tu 0 den 10 la:" << s;
```

Chương trình trên có thể viết lại như sau:

```
int k,s;
s = 0;
for (k = 10; k >=0 ; k--)
    s+= k;
cout << "tong cac so tu 0 den 10 la:"<< s;
```

Các biểu thức <biểu thức ban đầu>, <điều kiện>, <biểu thức tăng> có thể vắng mặt. Nếu <biểu thức ban đầu> vắng mặt thì biến điều khiển sẽ được lấy giá trị của biến đó trước khi vào vòng lặp **for**.

Ví dụ 2.15

```
int k,s;
s = 0;
k = 10;
for ( ;  k >=0 ; k--)
    s+= k;
cout << "tong cac so tu 0 den 10 la:"<< s;
```

Nếu <điều kiện> vắng mặt thì coi như biểu thức điều kiện của vòng lặp **for** luôn trả về giá trị đúng, vòng lặp sẽ lặp vô hạn.

Ví dụ 2.16

```
int k,s;
s = 0;
k = 10;
for ( ;  ; k--)
    s+= k;
cout << "tong cac so tu 0 den 10 la:"<< s;
```

Nếu <biểu thức tăng> vắng mặt thì biến điều khiển không được thay đổi giá trị sau mỗi lần <lệnh> được thực hiện xong. Vì vậy nếu <lệnh> không làm thay đổi biến điều khiển thì vòng lặp có thể lặp vô hạn.

Ví dụ 2.17

```
int k,s;
s = 0;
for (k = 10; k >=0 ; )
    s+= k;
cout << "tong cac so tu 0 den 10 la:"<< s;
```

Biến điều khiển có thể là một hoặc nhiều biến ngăn cách bằng toán tử dấu phẩy (,).

Ví dụ 2.18

```
int x, y,dem ;
dem = 0;
for ( x =0, y=0; x + y < 10, x < 3; x ++, y=y+2)
    dem ++;
cout <<" x = "<<x <<"y = "<< y<< "dem = "<< dem;
```

Kết quả

x = 3, y = 6 và dem = 3;

2.3. BÀI TẬP MINH HOẠ

1. Nhập vào 2 số a,b, kiểm tra a có chia hết cho b.
2. Nhập vào 3 cạnh a,b,c của một tam giác ABC, kiểm tra ABC có phải là tam giác vuông.
3. Kiểm tra xem vị trí tương đối của 2 đường tròn khi biết tâm và bán kính của 2 đường tròn đó.
4. Viết chương trình tính cộng trừ nhân chia của 2 sô, cho phép người dùng lưa chọn phép tính.
5. Viết chương trình đổi giây thành giờ, phút, giây.
6. Viết chương trình tính cước adsl biết đơn giá như sau:
   - < 100 MB - 100 đồng / MB
   - < 200 MB - 50 đồng/ MB
   - < 300 MB - 30 đồng /MB
   - >= 300 MB - 20 đồng /MB
7. Viết chương trình xác định điểm ưu tiên biết:
   - đối tượng 1: 3 điểm
   - đối tượng 2: 1.5 điểm
   - đối tương 3: 1 điểm
8. Viết chương trình đổi một số thập phân sang số nhị phân
9. Viết chương trình tính n!
10. Viết chương trình tính tổng, tích các số chẳn nhỏ hơn số nguyên dương n.
11. Viết chương trình kiểm tra một số là số chính phương.
12. Viết chương trình kiểm tra một số là số nguyên tố.

CHƯƠNG 3 HÀM

3.1. KIẾN THỨC CƠ BẢN

3.1.1. Khai báo và định nghĩa

Hàm (chương trình con) là một đoạn chương trình tương đối hoàn chỉnh độc lập có thể thực hiện trọn vẹn một công việc, chức năng nào đó.

Cú pháp

```
dữ_ liệu   Tên_hàm (<danh sách tham số>)
{
    <thân chương trình>
}
```

Giải thích

- <dữ liệu>: Là kiểu dữ liệu mà hàm sẽ trả về. Hàm có thể trả về bất cứ kiểu dữ liệu gì ngoại trừ mảng. .
- <tên hàm>: Là một nhóm các chữ cái dùng để đặt tên cho hàm. Cách đặt tên cho hàm cũng tương tự như cách đặt tên cho biến.
- <danh sách tham số>: là một danh sách các biến và sẽ nhận được giá trị của đối số khi hàm được gọi.
- <thân chương trình>: Là đoạn lệnh của hàm để xử lý một công việc nào đó.

Ví dụ 3.1

```
int binhphuong (int x)
{
    int kq;
    kq = x * x;
    return kq;
}
```

Nếu không có kiểu dữ liệu nào được chỉ định, trình biên dịch sẽ ngầm định kết quả trả về của hàm thuộc kiểu **int.**

Ví dụ 3.2

```
binhphuong (int x)
{
    int kq;
    kq = x * x;
    return kq;
}
```

Nhận xét Hàm trên vẫn đúng và trả về là *Kết quả* một số nguyên.

Giá trị trả về thông qua lệnh **return** phải thuộc kiểu dữ liệu trùng hoặc có thể thực hiện chuyển đổi kiểu dữ liệu sang kiểu trả về đã được khai báo trước.

Ví dụ 3.3

```
int binhphuong (int x)
{
    float kq;
    kq = x * x;
    return kq;
}
```

Hàm sẽ kết thúc việc thực thi và trở về hàm đã gọi nó khi thực hiện lệnh cuối cùng của hàm hoặc gặp lệnh **return**.

Ví dụ 3.4

```
void chao ( )
{
    cout << "Chao ban, chuc ban co mot ngay tot lanh";
}
int binhphuong (int x)
{
    int kq;
    return kq;
    kq = x * x;
}
```

Nếu hàm không trả về kết quả nào, từ khoá **void** được thay thế cho kiểu dữ liệu và không cần lệnh **return** ở cuối thân hàm.

Ví dụ 3.5

```
void binhphuong ( int x )
{
    x = x * x;
    cout << "x binh phuong là:"<< x;
}
```

Danh sách tham số này có thể có 1 biến, nhiều biến hoặc không có biến nào.Các biến này được phân cách bởi dấu phẩy (,) <danh sách tham số> có dạng như sau:

<kiểu dữ liệu1> <biến1>, <kiểu dữ liệu 2> <biến2> ,…,<kiểu dữ liệu n> <biến n>

Ví dụ 3.6

```
void chao ( )
{
```

```
        cout << "Xin chao!";
    }
    int tong (int a, int b)
    {
        int kq;
        kq = a + b;
        return kq;
    }
```

Hàm **main** cũng là một hàm tương tự như các hàm khác được người lập trình định nghĩa. Tuy nhiên hàm **main** là hàm được trình biên dịch gọi thực hiện đầu tiên. Mọi hàm khác do người lập trình định nghĩa được gọi thông qua tên hàm của chính nó.

Chỉ những hàm được định nghĩa sau có thể gọi được các hàm định nghĩa trước,ngược lại, những hàm định nghĩa trước không thể gọi các hàm định nghĩa sau.

Ví dụ 3.7

```
    int binhphuong (int x)
    {
        int kq;
        kq = x * x;
        return kq;
    }
    void main()
    {
        int a,b;
        cout << "nhap a:";
        cin >> a;
        b = binhphuong(a);
        cout<< "\n" << a << "binh phuong bang" << b;
    }
```

Các hàm phải được định nghĩa độc lập không được lồng nhau.

Ví dụ 3.8

```
int binhphuong (int x)
{
    void chao ( )
{
        cout << "Chao ban, chuc ban co mot ngay tot lanh";
}
```

```
int kq;
    kq = x * x;
    return kq;
}
```

3.1.2. Vai trò của hàm

Chương trình con giúp phân chia một công việc lớn thành nhiều việc nhỏ, giúp cho chương trình ngắn gọn, rõ ràng, có cấu trúc và dễ bảo trì.

Ví dụ 3.10:

Nhập vào 3 số nguyên, kiểm tra nếu 3 số là 3 số nguyên tố thì tổng của chúng có là một số nguyên tố không.

```
#include <iostream.h>
int nguyento (int x )
{
    int i,kq;
    kq = 1;
    for (i = 2; i < x - 1; i++)
        if (x % i == 0)
            kq = 0;
    return kq;
}
void main()
{
    int a,b,c,s;
    s = 0;
    cout << "nhap a: ";
    cin >> a;
    cout <<"\nnhap b:";
    cin >> b;
    cout <<"\nnhap c:";
    cin >> c;
    if (nguyento(a)==1 && nguyento(b)==1 && nguyento(c)==1)
    {
        cout << "3 so vua nhap la so nguyen to";
        s = a + b + c;
        if(nguyento(s) == 1)
            cout << "tong 3 so la so nguyen to";
        else
            cout <<"tong 3 so khong la so nguyen to";
    }
    else
```

```
        cout <<"3 so vua nhap khong phai la so nguyen to";
    }
```

3.1.3. Giá trị trả về

Một hàm chỉ có thể trả về một giá trị. Hàm có 3 loại giá trị trả về.

Thứ nhất, hàm có thể không trả về giá trị nào. Những hàm này được thiết kế thực hiện một công việc nào đó ví dụ như nhập xuất mà không cần có giá trị trả về.

Ví dụ 3.11:

```
void thongbao( )
{
    cout << "nhap du lieu khong dung quy cach";
}
```

Thư hai, hàm trả về giá trị 0 hoặc 1 (hoặc khác 0). Những hàm này được thiết kế thực hiện một công việc kiểm tra nào đó, tương tự như những biểu thức logic, và trả về giá trị đúng (tương đương 1) hoặc giá trị sai (tương đương với 0).

Ví dụ 3.12:

```
int nguyento (int x )
{
    int i,kq;
    kq = 1;
    for (i = 2; i < x - 1; i++)
        if (x % i == 0)
            kq = 0;
    return kq;
}
```

Thứ ba, hàm trả về một giá trị thuộc các kiểu dữ liệu nào đó đã được khai báo. Những hàm này được thiết kế để thực hiện một chức năng tính toán nào đó.

Ví dụ 3.13:

```
int binhphuong (int x)
{
    int kq;
    kq = x * x;
    return kq;
}
int giaithua( int x)
{
```

```
    int kq = 1;
    for (int i = 1; i <= x; i++)
        kq *= i;
    return kq;
}
```

Có thể gọi hàm mà không cần lấy giá trị trả về của hàm đó. Giá trị này sẽ được bỏ qua.

Ví dụ 3.14:

```
int giaithua( int x)
{
    int kq = 1;
    for (int i = 1; i <= x; i++)
        kq *= i;
    cout << "da tinh giai thua xong";
    return kq;
}
void main()
{
    int x;
    cout << “nhap gia tri x:”;
    cin >> x;
    giaithua(x);
}
```

3.1.4. Khai báo prototype

Ngôn ngữ C++ cung cấp cách khai báo prototype để tách biệt phần khai báo hàm và định nghĩa hàm.

Cú pháp khai báo prototype:

dữ_liệu Tên_hàm (**dữ _liệu_1** biến_1,**dữ _liệu_2** biến_2,…,**dữ_liệu_n** biến_n);

Ví dụ 3.15:

```
int binhphuong ( int x);
```

Phần khai báo hàm này có thể có hoặc không phần khai báo tên biến.

Ví dụ 3.16:

```
int binhphuong ( int);
```

Phần khai báo hàm thường ở phần đầu của chương trình. Sau đó là hàm main() và phần định nghĩa của các hàm.

Cú pháp định nghĩa các hàm tương tự như đã giới thiệu ở phần 1.

Ví dụ 3.17:

```
#include <iostream.h>
int binhphuong (int x);
void main()
{
    int a,b;
    cout << "nhap a:";
    cin >> a;
    cout<< "\n" << a << "binh phuong bang" << binhphuong(a);
}
int binhphuong (int x)
{
    int kq;
    kq = x * x;
    return kq;
}
```

Vai trò của khai báo prototype:

- Trình biên dịch kiểm soát được giá trị trả về của hàm, số lượng tham số và kiểu tham số mà người lập trình sử dụng.
- Giúp các hàm có thể gọi nhau mà không bị ảnh hưởng bởi thứ tự định nghĩa các hàm.
- Giúp người lập trình dễ dàng nhận biết cách sử dụng hàm như giá trị trả về, các đối số của hàm mà không cần quan tâm đến các lệnh chi tiết của hàm.

3.1.5. Phạm vi của hàm

Trong C++, mỗi một hàm là một khối mã lệnh riêng biệt. Mã của một hàm chỉ thuộc riêng của hàm đó và các câu lệnh trong bất cứ một hàm nào khác đều không thể truy cập đến nó ngoại trừ phương thức gọi hàm. Nói theo cách khác, các mã và dữ liệu được định nghĩa trong một hàm thì không thể ảnh hưởng đến mã và dữ liệu được định nghĩa trong một hàm khác, bởi vì hai hàm có hai phạm vi khác nhau.

Ví dụ 3.18:

```
int binhphuong (int x)
{
    float kq;
    kq = x*x;
    return kq;
}
```

```
int giaithua( int y)
{
    int kq = 1;
    for (int i = 1; i <= y; i++)
        kq *= i;
    return kq;
}
int nguyento (int x )
{
    int i,kq;
    kq = 1;
    for (i = 2; i < x - 1; i++)
        if (x % i == 0)
            kq = 0;
    return kq;
}
```

Nhận xét:

Ba hàm trên có khai báo biến có cùng tên kq, i. Khi chạy chương trình kết quả cho ra vẫn chính xác, chứng tỏ các biến này không có mối liên hệ với nhau bởi vì các biến này thuộc các hàm khác nhau.

3.2. BIẾN TOÀN CỤC, BIẾN CỤC BỘ, BIẾN TĨNH

3.2.1. Biến toàn cục

Các biến được khai báo ngoài phạm vi của các hàm được gọi là biến toàn cục. Biến toàn cục có hiệu lực đối với tất cả các hàm từ khi chương trình bắt đầu thực thi cho đến lúc kết thúc.

Ví dụ 3.19:

```
void thongbao();
int global;
void main()
{
    global = 9;
    thongbao();
    cout <<"gia tri global: " << global;
}
void thongbao()
{
    cout << "ban dang xem chuong trinh vi du";
```

```
        cout << "\ngia tri cua global trong ham: "<<global;
    }
```

Nhận xét:

Ta thấy kết quả in ra hai giá trị của biến global đều bằng 9, chứng tỏ biến global có hiệu lực trong hàm **thongbao** và cả trong hàm **main** cho đến khi chương trình hoàn toàn kết thúc.

3.2.2. Biến cục bộ

Các biến được khai báo trong phạm vi một hàm được gọi là biến cục bộ. Biến cục bộ sẽ có hiệu lực khi hàm được nạp vào và sẽ mất hiệu lực khi thoát hàm. Biến cục bộ không thể lưu trữ giá trị giữa các lần gọi hàm.

Ví dụ 3.20:

```
void thongbao();
void main()
{
    int a = 0;
    thongbao();
    thongbao();
    cout <<"\ngia tri a trong ham main: " << global;
}
void thongbao()
{
    int a;
    cout << "\ngia tri a truoc khoi tao: "<<a;
    a = 2;
    cout << "\ngia tri a sau khi khoi tao: "<<a;
}
```

Nhận xét:

- Chương trình sẽ xuất ra 5 câu thông báo cho ta 5 *Kết quả* của a. Trong câu thông báo thứ ba là các giá trị của **a** chưa khởi tạo, chứng tỏ biến **a** trong hàm **thongbao** không lưu giữ giá trị 2 do lệnh **a = 2** trong hàm sau khi kết thúc lần gọi hàm lần thứ nhất.
- Trong câu thông báo thứ 5, giá trị **a** là 0, chứng tỏ rằng **a** trong hàm **main** có giá trị khác và không liên quan đến biến **a** trong hàm **thongbao**.

Nếu biến toàn cục và biến cục bộ có trùng tên thì trong phạm vi của biến cục bộ được khai báo, trình biên dịch sẽ sử dụng biến cục bộ và xem như biến toàn cục đó không tồn tại, mọi sự thay đổi giá trị không ảnh hưởng đến biến toàn cục.

Ví dụ 3.21:

```
void thongbao();
int a;
void main()
{
    a = 0;
    thongbao();
    cout <<"\ngia tri a trong ham main: " << global;
}
void thongbao()
{
    int a;
    a = 3;
    cout << "\ngia tri a sau khi khoi tao: "<<a;
}
```

Nhận xét:

Giá trị đầu của a là 3, giá trị thứ 2 của a là 0, chứng tỏ biến a toàn cục không bị thay đổi giá trị trong hàm **thongbao**.

3.2.3. Biến tĩnh

Biến tĩnh là biến cục bộ của một hàm có thể lưu trữ giá trị của biến qua các lần gọi hàm. Trình biên dịch sẽ xem các biến tĩnh này như là các biến toàn cục khi lưu trữ nhưng vẫn giới hạn phạm vi của nó trong hàm mà nó được khai báo.

Cú pháp :

```
static  dữ_liệu   tên_biến;
```

Ví dụ 3.22:

```
void thongbao();
void main()
{
    int a = 0;
    thongbao();
    thongbao();
    cout <<"\ngia tri a trong ham main: " << a;
}
void thongbao()
{
    static int a;
    cout << "\ngia tri a truoc khoi tao: "<<a;
```

```
        a = 3;
        cout << "\ngia tri a sau khi khoi tao: "<<a;
        a ++;
    }
```

Nhận xét:

Giá trị của a được in ra 5 lần. Lần thứ nhất a có giá trị bất kỳ. Lần thứ hai a có giá trị 3. Lần thứ ba a có giá trị là 4. Chứng tỏ biến a được lưu giữ giá trị sau lần gọi thứ nhất. Lần thứ tư a có giá trị là 3. Lần thứ 5 a có giá trị là 0. Chứng tỏ biến a của hàm main và biến a của hàm thongbao không liên quan đến nhau.

3.3. TRUYỀN THAM SỐ

3.3.1. Truyền tham trị

Cú pháp khai báo prototype:

dữ_liệu tên_hàm (**dữ_liệu** biến , …,**dữ_liệu_n** biến_n);

Cú pháp định nghĩa hàm:

dữ_liệu tên_hàm (**dữ_liệu** biến, …,**dữ_liệu_n** biến_n)

{

<lệnh>

}

Ví dụ 3.23:

```
#include <iostream.h>
int tong(int a,int b);
void main()
{
    int a,b,c;
    a = 5;
    b = 6;
    c = 0;
    c = tong(a,b);
    cout << "a = " << a << '\n';
    cout << "b = " << b << '\n';
    cout << "c = " << c << '\n';
}
int tong( int x, int y)
{
    x = x + y;
```

```
        return x;
    }
```

Kết quả:

```
a = 5
b = 6
c = 11
```

Nhận xét:

- Giá trị của a không bị thay đổi vì x không phải là biến a.
- Khi đối số được truyền vào một hàm theo kiểu tham trị, trình biên dịch sẽ tạo ra một biến cục bộ có tên tương ứng với các tham số trong phần định nghĩa hàm và có giá trị bằng giá trị của đối số truyền vào. Khi biến cục bộ này có giá trị thay đổi thì không ảnh hưởng đến giá trị của đối số.

3.3.2. Biến tham chiếu

Biến tham chiếu của một biến đã được khai báo trước như là một tên khác của biến đó. Cả hai biến này chia sẽ với nhau cùng một vùng nhớ và cùng kiểu nên có cùng một giá trị. Nếu một trong hai biến thay đổi giá trị thì biến còn lại cũng bị thay đổi.

Cú pháp :

dữ_liệu_A biến_1;

dữ _liệu_A & biến_2 = biến_1;

Lưu ý: Hai biến là tham chiếu của nhau chỉ khi có cùng kiểu dữ liệu.

Phải chỉ định biến được tham chiếu đến cho biến tham chiếu ngay sau khi khai báo biến tham chiếu.

Ví dụ 3.24:

```
#include <iostream.h>
void main()
{
        int  den;
        int & mun = den;
        den = 10;
        cout << "den =" << den <<"\n";
        cout << "mun =" << mun << "\n";
        mun --;
        cout << "den = " << den << "\n";
        cout << "mun =" << mun << "\n";
        cout << "dia chi den:" << &den << "\n";
```

```
        cout << "dia chi mun:" << &mun << "\n";
        int trang = 2;
        mun = trang;
        cout << "den = " << den << "\n";
        cout << "mun =" << mun << "\n";
        cout << "trang =" <<trang << "\n";
        cout << "dia chi den:" << &den << "\n";
        cout << "dia chi mun:" << &mun << "\n";
        cout << "dia chi trang:" << & trang << "\n";
    }
```

Kết quả:

```
den=10
mun=10
den=9
mun=9
dia chi den:0x0012ff28
dia chi mun:0x0012ff28
den = 2
mun = 2
trang = 2
dia chi den:0x0012ff28
dia chi mun:0x0012ff28
dia chi trang:0x0012ff20
```

3.3.3. Truyền tham chiếu

Cú pháp khai báo prototype:

dữ_liệu tên_hàm (**dữ_liệu** biến , …,**dữ_liệu_n** biến_n);

Cú pháp định nghĩa hàm:

dữ_liệu tên_hàm (**dữ_liệu** biến, …,**dữ_liệu_n** biến_n)

```
{
    <lệnh>
}
```

Ví dụ 3.25:

```
#include <iostrream.h>
void tang( int & x);
void main()
```

```
{
    int a;
    a = 5;
    tang(a);
    cout << "a = " << a;
}
void tang(int & x)
{
    x = x + 5;
}
```

Kết quả:

```
a =  10
```

Nhận xét:

- Khi một đối số được truyền vào hàm theo kiểu tham chiếu, trình biên dịch sẽ tạo ra một biến tham chiếu đến đối số được truyền vào. Vì vậy mọi thao tác xử lý trên biến tham chiếu cũng là những thao tác xử lý trên đối số.
- Khi gọi sử dụng hàm có truyền tham số là tham chiếu, thì đối số tương ứng phải là một biến.

Trong cùng một hàm ta có thể truyền các đối số vào theo hai kiểu tham trị và tham biến cùng với nhau.

Ví dụ 3.26:

```
#include <iostream.h>
int tong(int & a,int b);
void main()
{
    int a,b,c;
    a = 5;
    b = 6;
    c = 0;
    c = tong(a,b);
    cout << "a = " << a << '\n';
    cout << "b = " << b << '\n';
    cout << "c = " << c << '\n';
}
int tong( int & x, int y)
{
    x = x + y;
```

```
        return x;
    }
```

Kết quả:

```
    a = 11
    b = 6
    c = 11
```

3.4. ĐỆ QUY

Trong C++, một hàm có thể gọi chính nó, một hàm được gọi là đệ quy nếu một phát biểu trong thân hàm gọi lại chính nó.

Cú pháp:

```
dữ_liệu   hàm_A ( <danh_sách_tham_số>)
{
        <lệnh 1>
        if ( <điều kiện> )
            hàm_A (danh_sach_tham_số);
        <lệnh 2>
}
```

Ý nghĩa:

Khi hàm_A được gọi, <lệnh 1> sẽ được thực hiện. Nếu <điều kiện> trả về giá trị đúng thì sẽ gọi thực hiện hàm_A mới mà không thực hiện <lệnh 2>. Nếu <điều kiện> của hàm_A thứ n trả về giá trị sai hàm_A thứ n+1 sẽ không được gọi và <lệnh 2> được thực hiện. Thực hiện xong <lệnh 2>, chương trình quay trở về hàm_A n – 1 và thực hiện <lệnh 2>. Tiếp tục như thế cho đến quay trở về hàm_A đầu tiên, thực hiện <lệnh 2> và kết thúc ra khỏi hàm_A.

Ví dụ 3.27: Tính tổng S = 1+2+ … +n

```
#include <iostream.h>
void demlui( int n);
void main()
{
    int  a = 3;
    demlui(a);
}
void demlui(int n)
{cout<<"demlui … "<<n<<"\n";//lệnh 1
    If(n>0)
```

```
        demlui(n-1);
    cout<<"n="<<n<<"\n";//lệnh 2
    }
```

Kết quả:

```
demlui … 3
demlui … 2
demlui … 1
demlui … 0
n=0
n=1
n=2
n=3
```

Ví dụ 3.28:

```
#include <iostream.h>
void demlui( int &n);
void main()
{
    int  a = 3;
    demlui(a);
}
void demlui(int &n)
{
    cout << "dem lui ... "<<n<<"\n"; //lenh 1
    if( n>0 )
    {
        n--;
        demlui (n);
    }
    cout << "n = " << n << "\n";   // lenh 2
}
```

Kết quả:

```
dem lui … 3
dem lui … 2
dem lui … 1
dem lui … 0
```

```
n = 0
n = 0
n = 0
n = 0
```

Nhận xét:

Tham số truyền vào hàm đệ quy là theo kiểu tham chiếu, nên khi giá trị của n ở hàm demlui cuối cùng bị thay đổi bao nhiêu, giá trị đó vẫn giữ nguyên khi trở về các hàm đếm lùi phía trước.

Chú ý:

- Khi sử dụng đệ quy chúng ta phải xây dựng điều kiện dừng.
- Phải xem xét cách sử dụng biến sao cho phù hợp, tránh bị tràn bộ nhớ: khai báo biến cục bộ, Sử dụng tham số truyền vào theo kiểu tham trị hay tham chiếu.

3.5. HÀM CHỒNG

Hàm chồng cho phép chúng ta sử dụng nhiều hàm khác nhau với cùng một tên hàm. Chúng ta thường sử dụng hàm chồng đối với những hàm có cùng một *ý nghĩa* nào đó nhưng có tham số truyền vào khác nhau (về số lượng hoặc kiểu dữ liệu) và cách xử lý có thể khác nhau.

Để khai báo hàm chồng, chúng ta vẫn sử dụng cách khai báo prototype, cách định nghĩa hàm như cách thông thường. Nhưng để phân biệt giữa các hàm chồng có cùng tên thì danh sách đối số của chúng phải khác nhau về số lượng hoặc kiểu dữ liệu.

Ví dụ 3.29:

```
float dientich ( float a, float h);
float dientich (float a, float b, float c);
…
float dientich (float a, float h)
{
    float dt;
    dt = 0.5 * a * h;
    return dt;
}
float dientich(float a, float b,float c)
{
    float dt, p;
    p = (a + b + c)/2;
    dt = sqrt(p*(p-a)*(p-b)*(p-c));
```

```
    return dt;
}
```

Nhận xét:

- Hai hàm trên có cùng chung ý *nghĩa* là tính diện tích, nên ta đặt tên là dientich.
- Hàm thứ nhất truyền vào 2 tham số kiểu **float**. Hàm thứ hai truyền vào 3 tham số kiểu **float**. Hai hàm trên gọi là hai hàm chồng.

Đối với hai hàm có cùng tên, có danh sách tham số cùng kiểu dữ liệu,mặc dù tên biến và kiểu dữ liệu trả về khác nhau thì hai hàm này không phải là hàm chồng của nhau và trình biên dịch sẽ báo lỗi.

Ví dụ 3.26:

```
float dientich (float a, float b);
float dientich(float c, float d);
int dientich (float a, float b);
```

Những hàm sau là những hàm chồng của nhau:

```
float dientich (float a, float b);
float dientich (int a, float b);
float dientich (float a, int b);
float dientich (int a, int b);
```

Khi một trong số các hàm chồng được gọi, trình biên dịch sẽ so sánh danh sách đối số với danh sách tham số của các hàm chồng đã được định nghĩa. Trình biên dịch chọn ra hàm nào phù hợp nhất tức có danh sách tham số phù hợp với danh sách đối số thì sẽ gọi thực hiện hàm đó. Hai danh sách tham số và đối số được coi là phù hợp khi có cùng số lượng và kiểu dữ liệu.

Ví dụ 3.30:

```
#include <iostream.h>
#include <math.h>
float dientich(float a, float h);
float dientich (float a, float b, float c);
void main()
{
    float a = 3, b = 4, c = 5;
    float d = 4, h = 5;
    float dt1, dt2;
    dt1 = dientich(a,b,c);
    dt2 = dientich(d,h);
    cout <<"dien tich tam giac: "<< dt1;
    cout <<"\ndien tich hinh chu nhat: "<<dt2;
```

```
}
float dientich (float a, float h)
{
    float dt;
    dt = 0.5 * a * h;
    return dt;
}
float dientich(float a, float b,float c)
{
    float dt, p;
    p = (a + b + c)/2;
    dt = sqrt(p*(p-a)*(p-b)*(p-c));
    return dt;
}
```

Kết quả:

dien tich tam giac: 6

dien tich hinh chu nhat: 20

Chú ý:

Nếu kiểu dữ liệu không phù hợp, trình biên dịch sẽ chọn hàm mà danh sách tham số của nó có kiểu dữ liệu mà kiểu dữ liệu của danh sách đối có thể chuyển đổi tốt nhất. Tuy vậy để lựa chọn chính xác hàm chồng mà chúng ta muốn sử dụng, ta nên đưa các đối số có cùng kiểu dữ liệu với tham số hàm đã khai báo và định nghĩa.

3.6. HÀM MẪU

Hàm template là chức năng mới mà C++ thêm vào. Hàm này được dùng để mô tả chung từng bước thực hiện của một công việc nào đó với một kiểu dữ liệu chung. Kiểu dữ liệu này có thể được thay thế bởi các kiểu dữ liệu khác như **int**, **double** … Ứng với nhiều kiểu dữ liệu khác nhau này, hàm vẫn hoạt động theo một cách thức chung mà hàm template đã mô tả. Kiểu dữ liệu này sẽ thay đổi phụ thuộc theo kiểu dữ liệu của đối số truyền vào. Trình biên dịch sẽ tạo ra một hàm có cách thức hoạt động như ở hàm template đã định nghĩa và thay thế dữ liệu chung thành kiểu dữ liệu của đối số.

Cú pháp:

Cách 1:

Khai báo prototype:

template <typename tên_chung>

Dữ_liệu Tên_hàm (tên_chung biến_1, …,tên_chung biến_n);

Định nghĩa hàm:

```
template  <typename     tên_chung>
Dữ_liệu   Tên_hàm ( tên_chung      biến_1, …,tên_chung  biến_n)
{
    <lệnh>
}
```

Cách 2:

Khai báo prototype:

```
template  <class  tên_chung>
Dữ_liệu   Tên_hàm ( tên_chung      biến_1, …,tên_chung  biến_n);
```

Định nghĩa hàm:

```
template  <class  tên_chung>
Dữ_liệu   Tên_hàm ( tên_chung      biến_1, …,tên_chung  biến_n)
{
    <lệnh>
}
```

Ví dụ 3.31:

Ta muốn tạo ra các hàm đổi giá trị cho hai biến kiểu **int**, **double**, kiểu **char** …

```
#include <iostream.h>
template <typename tam>
void doi( tam &a, tam &b);
void main()
{
    int a; int b;
    a = 5;
    b = 6;
    cout << "a = " << a;
    cout <<"\n b = " << b;
    float c, d;
    c = 0.5;
    d = 4.2;
    doi (c,d);
    cout <<"\nc = "<<c;
    cout <<"\nd = "<<d;
}
template <typename tam>
void doi( tam &a, tam &b)
{
    tam c;
```

```
        c = a;
        a = b;
        b = c;
    }
```

Kết quả:

```
    a = 5
    b = 6
    c = 4.2
    d = 0.5
```

3.7. BÀI TẬP MINH HOẠ

1. Viết chương trình nhập vào 4 số, tìm giá trị lơn nhất, giá trị nhỏ nhất của 4 số đó với các hàm sau:
   - Hàm nhập vào 4 số nguyên.
   - Hàm tìm max 2 số
   - Hàm tìm min 2 số
   - Hàm main
2. Viết chương các hàm tính n!, x^y để thực hiện các công thức sau:

   $1 + 2 + 3 \ldots + n$

   $1! + 2! + 3! + \ldots + n!$

   $1^n + 2^{n-1} + \ldots. + n^1$
4. Viết hàm kiểm tra một số là số nguyên tố, số chính phương. Áp dụng tính tổng của các số nguyên tố, số chính phương nhỏ hơn 1000.
5. Viết hàm kiểm tra một số là số chính phương. Áp dụng tính tổng của các số chính phương nhỏ hơn 1000.
6. Viết hàm tính diện tích tam giác, hình tròn, hình chữ nhật, hình thang.
7. Viết hàm kiểm tra năm nhuận.
8. Viết hàm đệ quy và không đệ quy để tính: $f_n = f_{n-1} + f_{n-2}$ với $f_0 = f_1 = 1$.
9. Viết hàm tìm ước số chung lớn nhất của hai số nguyên dương.

CHƯƠNG 4 KIỂU MẢNG

6.1. KIỂU MẢNG

Mảng là một loại dữ liệu có thể lưu trữ nhiều giá trị cùng một loại dữ liệu nào đó, bởi những phần tử riêng biệt của mảng, và máy tính lưu giữ các phần tử này trên những vùng nhớ liên tục.

Mảng là kiểu dữ liệu phức hợp, vì nó được xây dựng từ các loại dữ liệu khác. Không một biến nào có thể khai báo là một mảng đơn thuần, mà luôn khai báo biến đó là mảng của kiểu dữ liệu nhất định như **char**, **int**, **float,double,** …

6.2. MẢNG MỘT CHIỀU

6.2.1. Giới thiệu kiểu mảng một chiều

Mảng một chiều là mảng của kiểu dữ khác kiểu mảng.

Cú pháp:

dữ_liệu Tên_Mảng [*số_lượng_phần_tử*];

Giải thích:

- **dữ_liệu** : Là kiểu dữ liệu các phần tử trong mảng.
- Tên_mảng: Tên của mảng.
- *số_lượng_phần_tử*: Số lượng các phần tử của mảng.

Ví dụ 4.1: **int** thang[12];

float diem[60];

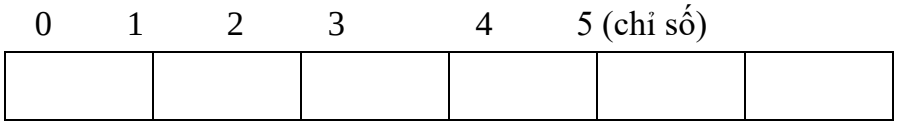
Mỗi phần tử của mảng có thể truy xuất một cách riêng lẻ thông qua chỉ số và tên của mảng. Chỉ số là số nguyên dương. Phần tử đầu tiên của mảng có chỉ số là 0, lần lượt tăng dần đến phần tử cuối cùng có chỉ số bằng số lượng phần tử của mảng trừ đi 1.

Cú pháp:

Tên_mang [*chỉ_số*]

Ví dụ 4.2:

char mangsonguyen[6];



mangsonguyen

Cách truy xuất:

mangsonguyen[0] = 2;

```
mangsonguyen[5] = 48;
mangsonguyen[1] = mangkytu[5] + 2;
```

Ví dụ 4.3:

```
#include <iostream.h>
void main()
{
    float diem[3];
    float dtb;
    diem[0]= 4;
    diem [1]= 5.5;
    diem [2] = 6;
    dtb = (diem[0] + diem[1] + diem[2])/3;
    cout << "diem trung binh: " << dtb;
}
```

Kết quả:

```
diem trung binh 5.16667
```

Nhận xét:

Các phần tử của mảng diem như là một biến riêng kiểu **float**. Chúng ta có thể thực hiện các phép toán cơ bản, hay phép gán …

Chú ý:

Trình biên dịch không kiểm tra người lập trình có sử dụng chỉ số hợp lệ hay không. *Ví dụ* mảng diem chỉ có 3 phần tử, trình biên dịch không báo lỗi nếu bắt gặp câu lệnh diem[101] = 1. Nhưng khi chương trình thực thi có thể bị lỗi.

4.2.2. Các kỹ thuật cơ bản

4.2.2.1. Khởi tạo

Các biến kiểu mảng có thể khởi tạo các giá trị đầu của phần tử ngay khi khai báo. Có 2 cách khởi tạo:

Cú pháp cách 1:

dữ_liêu Tên_mang[số_phần_tử] = {giá_trị_1,giá_trị_2, …,giá_trị_n};

Ví dụ 4.4:

```
int songuyen[3] = {1,2,5};
```

Số lượng giá trị liệt kê phải nhỏ hơn hoặc bằng số phần tử của mảng. Nếu số giá trị liệt kê nhỏ hơn số phần tử của mảng, các phần tử đầu của mảng lần lượt mang giá trị liệt kê, các phần tử còn lại có giá trị 0.

Ví dụ 4.5:

```
float   sothuc[20] = {1.2, 2.5} ;
```

Cú pháp cách 2:

```
dữ_liệu   Tên_mảng[ ] = {giá_trị_1, giá_trị_2, …, giá_trị_n};
```

Chúng ta không khai báo số phần tử cho mảng trước, nhưng trình biên dịch sẽ đếm có bao nhiêu giá trị được liệt kê sẽ tạo ra bấy nhiêu phần tử và lần lượt mang các giá trị đã được liệt kê.

Ví dụ 4.6:

```
int songuyen [] = { 1,2,6,8,5};
```

Chú ý:

- Chúng ta chỉ có thể khởi tạo giá trị ngay khi khai báo.
- Chúng ta không thể gán giá trị của toàn bộ một mảng này sang một mảng khác.

Ví dụ 4.7:

```
int A[4];
A = {1,2,3,4};    // Không hợp lệ
hoặc
int A[4] = {1,2,3,4};
int B[4];
B = A;          //Không hợp lệ
```

4.2.2.2. Cách truy xuất mảng một chiều

Các phần tử của mảng được truy xuất thông qua tên mảng và chỉ số của phần tử. Những chỉ số này có thể là một hằng, hoặc là giá trị của một biến.

Ví dụ 4.8:

```
#include <iostream.h>
void main()
{
    int songuyen = {1,2,3,4};
    int i = 1;
    cout << "gia tri phan tu thu 2: " << songuyen[1] << "\n";
    cout << "gia tri phan tu thu " << i+1 << ": " << songuyen[i] << "\n";
}
```

Kết quả:

```
gia trị phan tu thu 2: 2
gia trị phan tu thu 2: 2
```

Biến lưu giữ chỉ số phải là biến thuộc kiểu số nguyên vì các chỉ số luôn là các số nguyên. Việc sử dụng biến lưu giữ chỉ số các phần tử của mảng là một giải pháp rất hiệu quả nếu mảng có số lương phần tử lớn. Để duyệt qua tất cả các phần tử của mảng, ta có thể sử dụng các cấu trúc vòng lặp như **for**, **while**, hoặc **do** … **while**.

Ví dụ 4.9:

```
#include <iostream.h>
void main()
{
    int songuyen [5] = {1,3,5,7};
    int i = 0;
    for ( i = 0; i < 5; i++)
        cout << songuyen[i] << " ";
    int tong = 0;
    i = 0;
    while ( i < 5)
    {
        tong += songuyen[i];
        i++;
    }
    cout <<"\n"<<"tong cac phan tu mang songuyen: "<<tong;
}
```

Kết quả:

```
1 3 5 7 0
tong cac phan tu mang songuyen: 16
```

4.2.2.3. Truyền mảng một chiều vào hàm

Ngoài những kiểu dữ liệu cơ bản, mảng cũng có thể là tham số truyền vào của một hàm.

Cú pháp:

```
dữ_liệu   Tên_hàm ( dữ_liệu_mảng tên_mảng [])
{
    <lệnh >
}
```

Theo cú pháp trên ta chỉ truyền vào hàm tên của mảng và kiểu dữ liệu của phần tử mảng, chúng ta không truyền vào kích thước của mảng. Vậy để sử dụng được mảng, chúng ta cần truyền vào thêm kích thước của mảng.

Cú pháp đề nghị:

```
dữ_liệu   Tên_hàm ( dữ_liệu_mảng  tên_mảng[], int  n)
{
    <lệnh>
}
```

Với n là tham số lưu giữ số phần tử của mảng.

Chú ý:

Chúng ta có thể truyền một tham số kiểu mảng có 12 phần tử như sau:

```
dữ_liệu   Tên_hàm ( dữ_liệu  tên_mảng [12])
{
    <lệnh >
}
```

Trình biên dịch thực sự không kiểm tra số phần tử của mảng thông qua cách khai báo như trên. Ở trong thân hàm, trình biên dịch hoàn toàn không biết mảng có bao nhiêu phần tử.

Ví dụ 4.10:

```
#include <iostream.h>
int tongmang(int a[], int n);
void main()
{
    int banh[7]= {1,2,3,3,2,4,5};
    int tong;
    tong = tongmang(banh,7);
    cout << "tong banh an trong mot tuan la: "<< tong;
}
int tongmang (int a[],int n)
{
    int i,s;
    s = 0;
    for (i= 0; i< n; i++)
        s += a[i];
    return s;
}
```

Kết quả:

```
tong banh an trong mot tuan la: 20
```

Chú ý:

Khi truyền tên mảng vào hàm, ta đã truyền địa chỉ của mảng hay đia chị của phần tử đầu tiên của mảng. Việc truyền tham số này tương tự như truyền tham số theo kiểu tham chiếu.

4.2.2.4. Các bài toán cơ bản

Trên mảng chúng ta có 2 bài toán đặc trưng đó là sắp xếp và tìm kiếm.

Tìm kiếm tuần tự

Nội dung bài toán: Xác định vị trí (chỉ số) của giá trị x trong mảng a có n phần tử. Nếu a có phần tử mang giá trị x thì trả ra kết quả chỉ số của phần tử đó, ngược lại trả ra -1.

Thuật toán: Lần lượt so sánh giá trị của phần tử đầu tiên đến phần tử cuối cùng của mảng. Nếu có một phần tử nào bằng giá trị x chúng ta lưu giữ giá trị chỉ số của phần tử đó. Nếu không có giá trị chỉ số nào được lưu giữ thì trả ra *Kết quả* -1, ngược lại trả về giá trị đó.

Chương trình:

```
int timmang( int a[],int n, int x)
{
    int i,kq;
    kq = -1;
    for (i = 0; i< n; i++)
        if ( a[i] == x )
            kq = i;
    return kq;
}
```

Nhận xét:

Nếu ta duyệt các phần tử của mảng từ các phần tử đầu đến các phần tử cuối thì giá trị trả về sẽ là chỉ số của phần tử cuối cùng bằng x. Ngược lại, nếu ta duyệt các phần tử mảng từ các phần tử cuối đến phần tử đầu thì giá trị trả về sẽ là chỉ số của phần tử đầu tiên trong mảng bằng x.

Hàm tìm vị trí đầu:

```
int timmangcuoi(int a[],int n ,int x)
{
    int i,kq;
    kq = -1;
    for (i = n- 1; i >= 0; i--)
        if ( a[i] == x )
            kq = i;
    return kq;
```

```
}
```

Nhận xét:

Với thuật toán này, ta phải kiểm tra hết tất cả các phần tử của mảng. Nếu trong trường hợp phần tử có giá trị x là phần tử đầu tiên thì thuật toán này không trả về giá trị ngay mà vẫn tiếp tục so sánh với các phần tử còn lại. Để cải tiến thuật toán này, ta cần một biến làm hiệu cho chương trình biết sau mỗi lần so sánh đã tìm ra phần tử x chưa, nếu tìm thấy rồi chương trình không cần tiếp tục kiểm tra nữa.

Chương trình:

```
int timmang( int a[], int n , int x)
{
    int i,kq;
    i = 0;
    kq = 0;
    while (i < n && kq == 0)
    {
        if (a[i] == x)
            kq = 1;
        else
            i++;
    }
    if ( kq == 0 )
        return -1;
    else
        return i;
}
```

Tìm kiếm nhị phân

Nội dung bài toán: Tìm kiếm một phần tử có giá trị x trên một dãy số n phần tử đã được sắp xếp tăng dần.

Ý tưởng: Khoanh từng vùng của dãy số so sánh với x để biết x thuộc nhóm nào. Vì mảng đã sắp xếp nên lấy phần tử ở giữa dãy số so sánh với x, nếu phần tử này bằng x thì kết thúc việc tìm kiếm. Nếu x có giá trị nhỏ hơn thì x thuộc nửa dãy số phía bên trái, ngược lại ở nửa dãy số phía bên phải. Tiếp tục lấy phần tử ở giữa dãy số mà x thuộc về so sánh với x để xác định được vị trí của x. Nếu dãy số cuối cùng là một phần tử, không thể chia đôi được nữa mà vẫn không tìm thấy phần tử bằng x thì kết luận không có phần tử nào bằng x.

Ví dụ 4.11:

x = 23;

dãy số: 1 4 6 8 9 12 23 45

cách tìm:

0	1	2	3	4	5	6	7
1	4	6	8	9	12	23	45

phần tử ở giữa : phần tử 4
x < 9 → x thuộc dãy số từ phần tử thứ 5 đến 8

0	1	2	3	4	5	6	7
1	4	6	8	9	12	23	45

phần tử ở giữa: phần tử 6
x = 23 → phần tử thứ 6 có giá trị bằng x

Chương trình:

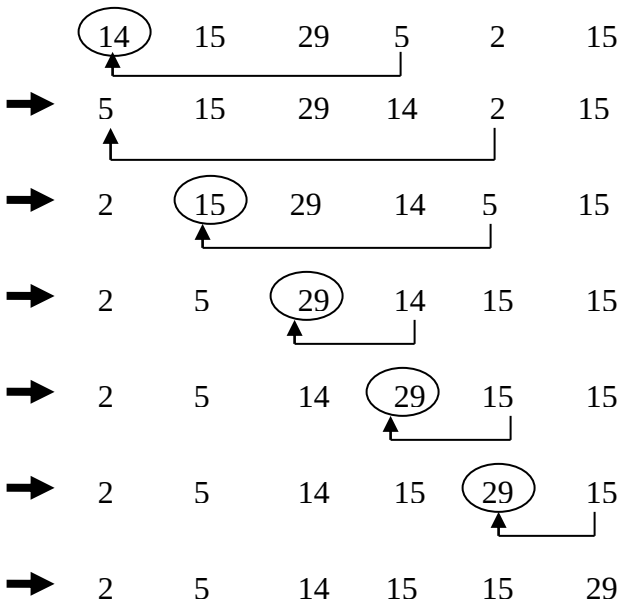
```
int timkiemnhiphan(int a[],int n,int x)
{
    int trai , phai ,giua;
    trai = 0;
    phai = n - 1;
    while (phai >= trai)
    {
        giua = (trai + phai)/2;
        if ( a[giua] == x)
            return giua;
        else
        {
            if ( x < a[giua] )
                phai = giua - 1;
            else
                trai = giua + 1;
        }
    }
    return - 1;
}
```

Sắp xếp

Nội dung bài toán: Cho một mảng a có n phần tử. Sắp xếp sao cho giá trị của mảng a có giá trị tăng dần.

Thuật toán: Đổi chỗ các phần tử cho nhau, các phần tử nhỏ về đầu mảng, các phần tử lớn về cuối mảng. Lần lượt lấy các phần tử từ vị trí đầu đến vị trí cuối so sánh với các phần tử còn lại ở phía phải, nếu gặp phần tử nào nhỏ hơn chính nó thì đổi chỗ cho phần tử ấy.

Ví dụ:



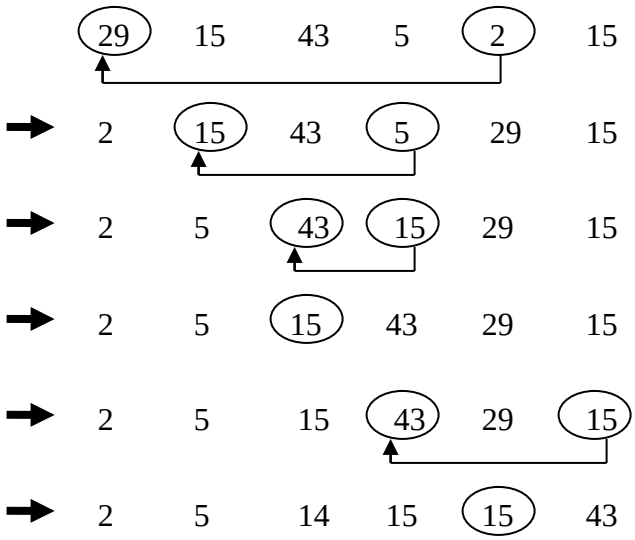
Chương trình:

```
void sapxepxen(int a[],int n )
{
    int i,j,t;
    for (i = 0; i < n - 1; i++)
        for (j = i+ 1; j< n ; j++)
            if ( a[j] < a[i] )
            {
                t = a[j];
                a[j] = a[i];
                a[i] = t;
            }
}
```

Nhận xét:

Nếu bên phải phần tử đang xét có n phần tử lớn hơn thì xảy ra n lần đổi chỗ giữa các phần tử. Phần tử cuối cùng được đổi chỗ là phần tử nhỏ nhất ở bên phải

tính từ phần tử đang xét. Vậy nếu chúng ta tìm ra phần tử cuối cùng phải đổi chỗ thì với mỗi phần tử chỉ cần đổi chỗ một lần.



Chương trình:

```
void sapxepchon(int a[], int n)
{
    int i,j,t,k;
    for (i = 0; i< n - 1; i++)
    {
        k = i;
        for (j = i + 1; j< n; j++)
        {
            if ( a[j] < a[k] )
                k = j;
        }
        if ( k != i)
        {
            t = a[k];
            a[k] = a[i];
            a[i] = t;
        }
    }
}
```

Chú ý:

Bài toán tương tự với sắp xếp giảm dần.

4.3. MẢNG NHIỀU CHIỀU

4.3.1. Giới thiệu mảng 2 chiều

Nếu mảng một chiều như là một dãy các dữ liệu thì mảng 2 chiều như là một bảng các dữ liệu, có các dòng và các cột dữ liệu. Ví dụ:

1 2 3

4 5 6

C++ không cung cấp một kiểu dữ liệu mảng hai chiều riêng biệt, mà mảng hai chiều được tạo ra bằng cách tạo ra mảng một chiều mà mỗi phần tử là một mảng cũng là mảng một chiều.

Cú pháp:

dữ_liệu Tên_mảng[*số_phần_tử_hang*][*số_phần_tử_cột*];

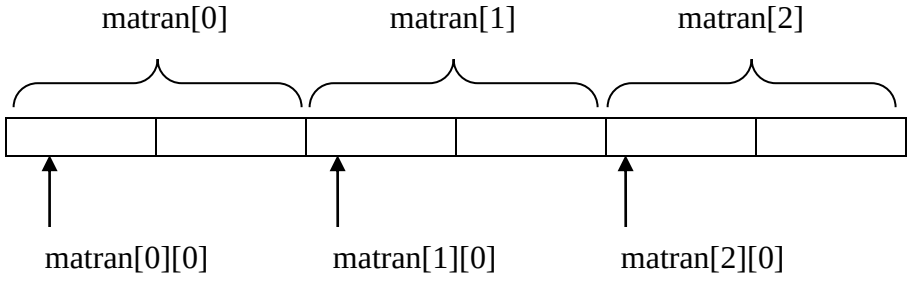
Giải thích:

- **dữ_liệu**: Là kiểu dữ liệu các phần tử trong mảng.
- Tên_mảng: Tên của mảng.
- *số_lượng_phần_tử_hàng*: Số lượng hàng các phần tử của mảng.
- *số_lượng_phần_tử_cột*: Số lượng cột các phần tử của mảng.

Ví dụ 4.13: **int** matran[3][2];

Nhận xét:

matran được khai báo như trên là mảng có 2 phần tử matran[0] và matran[1] mà mỗi phần tử là một mảng số nguyên một chiều có 3 phần tử. Phần tử đầu của matran[0] là matran[0][0].



Muốn truy xuất một phần tử của mảng 2 chiều phải thông qua hai chỉ số. Ta có thể xem chỉ số thứ nhất là hàng, chỉ số thứ hai là cột. matran có thể xem như có 3x2=6 phần tử số nguyên, có 3 hàng và 2 cột.

Cú pháp truy xuất phần tử mảng 2 chiều:

Tên_mang [*chỉ_số_dong*][*chỉ_số_cột*]

(chỉ số cột)

		cột 1	cột 2
(chỉ số dòng) ↓	dòng 1	matran[0][0]	matran[0][1]
	dòng 2	matran[1][0]	matran[1][1]
	dòng 3	matran[2][0]	matran[2][1]

Ví dụ 4.14:

```
#include <iostream.h>
void main()
{
    int matran[2][2];
    matran[0][0] = 0;
    matran[0][1] = 2;
    matran[1][0] = 6;
    matran[1][1] = 8;
}
```

Nhận xét:

Mảng trên có 2 dòng và 3 cột. Giá trị của mảng như sau:

0	2
6	8

4.3.2. Các kỹ thuật cơ bản

4.3.2.1. Khởi tạo

Các biến kiểu mảng có thể khởi tạo các giá trị đầu của phần tử ngay khi khai báo.

Cú pháp1 :

dữ_liêu Tên_mang[*số_phần_tử_hàng*][*số_phần_tử_cột*] =

```
    {
        {giá_trị1_hang1,giá_trị2_hang1, …,gia_triM_hang1},
        {giá_trị1_hang2,giá_trị2_hang2, …,gia_triM_hang2},
        …
        {giá_trị1_hangN,giá_trị2_hangN, …,gia_triM_hangN}
};
```

Ví dụ 4.15:

```
int a[2][3] =
    {
```

```
        {1,2},
        {3,4}
    };
```

Nhận xét:

Giá trị các phần tử của mảng như sau:

1 2

3 4

Cú pháp 2:

dữ_liêu Tên_mang[][*số_phần_tử_cột*] =

```
    {
        {giá_trị1_hang1,giá_trị2_hang1, …,gia_triM_hang1},
        {giá_trị1_hang2,giá_trị2_hang2, …,gia_triM_hang2},
        …
        {giá_trị1_hangN,giá_trị2_hangN, …,gia_triM_hangN}
  };
```

Ví dụ 4.15:

```
    int a[][3] =
        {
            {1,2},
            {3,4},
            {1,3,5}
        };
```

Nhận xét:

Giá trị các phần tử của mảng như sau:

1 2 0

3 4 0

1 3 5

4.3.2.2. Cách truy xuất

Các phần tử của mảng được truy xuất thông qua tên mảng và chỉ số hàng và cột của phần tử. Những chỉ số này có thể là một hằng, hoặc là giá trị của một biến. Với tính chất của chỉ số hàng và cột của mảng 2 chiều ta có thể dùng 2 vòng lặp **for** để truy xuất từng phần tử của mảng.

Ví dụ 4.16:

```
    #include <iostream.h>
    void main()
```

```
{
    int a[2][3] =
    {
        {1,2},
        {3,4}
    };
    for (int i = 0; i< 2; i++)
    {
        for (int j = 0; j< 3; j++)
            cout << &a[i][j] << "\t";
        cout << "\n";
    }
}
```

Kết quả:

```
1    2
3    4
```

Nhận xét:

Chúng ta sử dụng 2 biến điều khiển vòng lặp **for** i,j làm chỉ số hàng và cột. Ngoài sử dụng vòng lặp **for** ra ta có thể sử dụng các loại vòng lặp khác tuỳ theo điều kiện của bài toán.

4.3.2.3. Truyền mảng 2 chiều vào hàm

C++ xem mảng 2 chiều là mảng 1 chiều mà mỗi phần tử là mảng một chiều. Vì vậy khi truyền mảng 2 chiều vào hàm tương tự như ta truyền mảng một chiều mà mỗi phần tử là một mảng có số phần tử cố định.

Cú pháp khai báo prototype:

dữ_liệu Tên_hàm (**dữ_liệu_mảng** tên_mảng[][*số_cột*] , **int** so_hàng);

Cú pháp định nghĩa hàm:

dữ_liệu Tên_hàm (**dữ_liệu_mảng** tên_mảng [][*số_cột*], **int** so_hàng)

```
{
    <lệnh >
}
```

Nhận xét:

- *số_cột* là một hằng số.
- Số cột của mảng hai chiều là cố định, bằng với số_cột mà ta đưa vào. Vì vậy chúng ta chỉ truyền thêm một số nguyên so_hàng để truyền vào hàm số dòng của mảng.

Ví dụ 4.17:

```
#include <iostream.h>
void xuatmang2chieu(int a[][7], int n);
int tongmang2chieu(int a[][7], int n);
int timvitrimax(int a[], int n);
void main()
{
    int a[2][7] =
    {
        {132,112,312,156,126,289,244},
        {121,145,178,167,213,324,201}
    };
    cout << "bang thong ke ban hoa trong tuan: \n";
    xuatmang2chieu(a,2);
    cout << "\ntong so bong hoa da ban: " << tongmang2chieu(a,2);
    cout << "\nngay nu mua hoa nhieu nhat: "<< 2 + timvitrimax(a[0],7);
    cout << "\nngay nam mua hoa nhieu nhat: "<< 2 + timvitrimax(a[1],7);
}
int tongmang2chieu(int a[][7], int n)
{
    int tong = 0;
    for (int i = 0; i< n; i++)
        for (int j = 0; j< 7; j++)
            tong += a[i][j];
    return tong;
}
void xuatmang2chieu(int a[][7], int n)
{
    for (int i = 0; i < n; i++)
    {
        for (int j =0; j< 7; j++)
            cout << a[i][j] << "\t";
        cout << "\n";
    }
}
int timvitrimax(int a[], int n)
{
    int i,vitri;
```

```
        vitri = 0;
        for ( i = 1; i< n; i++)
            if( a[vitri] < a[i])
                vitri = i;
        return vitri;
    }
```

Kết quả:

```
bang thong ke ban hoa trong tuan:
132     112     312     156     126     289     244
121     145     178     167     213     324     201
tong so bong hoa da ban: 2720
ngay nu mua hoa nhieu nhat: 4
ngay nam mua hoa nhieu nhat: 7
```

4.3.3. Các bài toán cơ bản

- **Bài toán:** Sử dụng mảng 2 chiều để biểu diễn ma trân. Xuất ma trận tam giác.

Phân tích: Các phần tử ở ma trận tam giác trên có chỉ số cột lớn hơn chỉ số hàng. Các phần tử ở ma trân tam giác dưới có chỉ số hàng lớn hơn chỉ số cột.

- **Chương trình:**

```
void xuattamgiacduoi( int a[][5], int n)
{
    int i,j;
    for( i =0; i< n; i++)
    {
        for( j =0; j<= i;j++)
            cout << a[i][j] << '\t';
        cout << '\n';
    }
}
void xuattamgiactren( int a[][5], int n)
{
    int i,j;
    for( i =0; i< n; i++)
    {
        for( j =0; j< 5;j++)
            if( j >= i )
                cout << a[i][j] << '\t';
            else
```

```
                cout << " " << '\t';

            cout << '\n';
        }
    }
```

Bài toán sắp xếp

Ứng dụng các thuật toán sắp xếp trên mảng một chiều sắp xếp mảng hai chiều.

4.4. Mảng nhiều chiều

C++ cho phép bạn khai báo và sử dụng mảng nhiều chiều tuỳ theo trình biên dịch. Cú pháp khai báo mảng nhiều chiều:

dữ_liệu Tên_mảng *[số_phân_tử_1*][*số_phân_tử_2*]…[*số_phân_tử_n*]

Các mảng ba hay nhiều chiều hơn ít khi được sử dụng vì tổng số bộ nhớ chúng yêu cầu khá lớn. Ngoài ra việc truy cập một phần tử là phức tạp cho người sử dụng và chiếm nhiều thời gian hơn trên mảng một chiều.

Khi chuyển mảng nhiều chiều vào một hàm, tương tự như mảng 2 chiều, chúng ta cũng phải khai báo hết tất cả kích thước bên trái.

4.5. KIỂU CON TRỎ

4.5.1. Con trỏ và địa chỉ

Con trỏ là kiểu dữ liệu lưu trữ địa chỉ của một vùng nhớ. Nói cách khác, nội dung của biến con trỏ chỉ đến vị trí vùng nhớ nhất định nào đó.

Kích thước của con trỏ tuỳ theo cách biểu diễn bộ nhớ. Khi chúng ta khai báo một con trỏ, ta phải khai báo con trỏ sẽ trỏ đến vùng nhớ thuộc kiểu dữ liệu gì để thông báo cho trình biên dịch biết kiểu dữ liệu của giá trị mà vùng nhớ lưu trữ mà con trỏ chỉ đến. Một con trỏ thuộc kiểu dữ liệu A thì ta gọi là con trỏ A.

Cú pháp khai báo biến con trỏ:

dữ_liêu * tên_con_tro;

Chúng ta sử dụng toán tử * ở phía trước tên của con trỏ để thông báo biến này là biến con trỏ chỉ đến một vùng nhớ lưu trữ giá trị thuộc kiểu **dữ_liệu**.

Ví dụ 7.3:

```
int * pt;
float * ptf, *ptff2;
int *pti, so;
```

Nhận xét:

pt, ptf, ptf2, pti là những biến con trỏ. so chỉ là biến thuộc kiểu số nguyên.

Chúng ta muốn chỉ định biến con trỏ này chỉ đến một vùng nhớ của một biến nào đó, ta gán giá trị biến con trỏ là địa chỉ của biến đó.

Ví dụ 7.4:

```
int *pt;
int hoa = 5;
pt = &hoa;
```

Để truy xuất tới nội dung mà con trỏ chỉ đến ta dùng toán tử *. Nếu pt là con trỏ, *pt là giá trị của vùng nhớ mà pt trỏ tới.

Ví dụ 7.5:

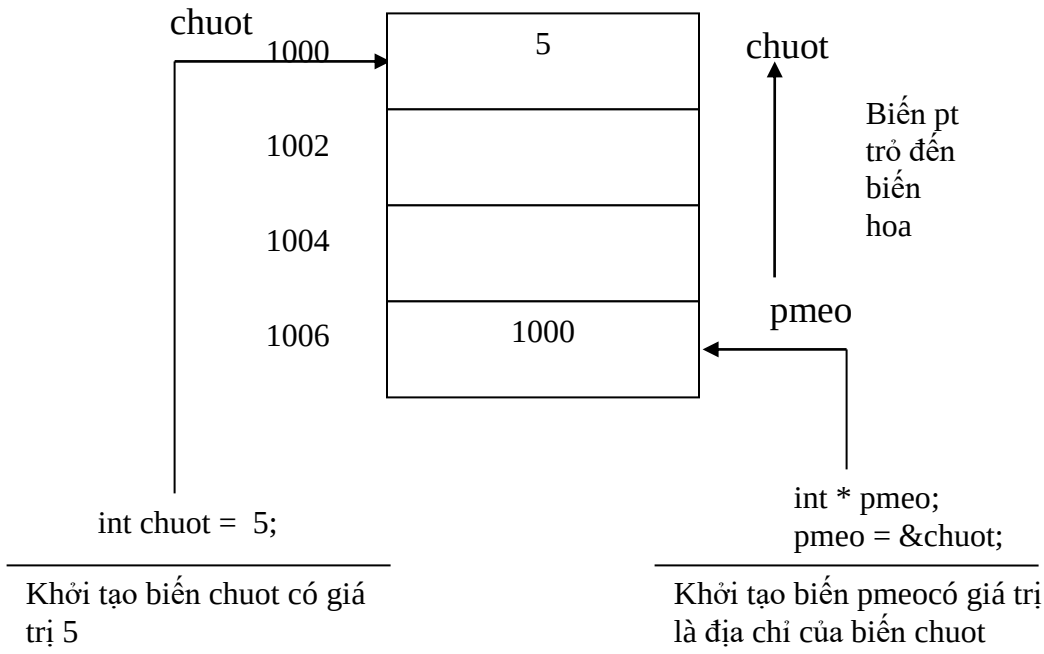
```
#include <iostream.h>
void main()
{
    int *pmeo;
    int chuot = 5;
    pmeo = &chuot;
    cout << "gia tri chuot: " << chuot;
    cout << "\ndia chi chuot: "<< &chuot;
    cout << "\ngia tri *pmeo: "<<*pmeo;
    cout << "\ngia tri pmeo: " << pmeo;
}
```

Kết quả:

```
gia tri chuot: 5
dia chi chuot: 0x0012FF78
gia tri *pmeo: 5
gia tri pmeo: 0x0012FF78
```

Nhận xét:

pmeo được chỉ định trỏ tới biến chuot nên giá trị của pmeo là địa chỉ của biến chuot, *pmeo là giá trị của biến chuot.



Khởi tạo biến chuot có giá trị 5

Khởi tạo biến pmeocó giá trị là địa chỉ của biến chuot

Nếu thay đổi giá trị *pmeo thì giá trị của chuot cũng thay đổi và ngược lại.

Con trỏ có thể thay đổi địa chỉ mà nó trỏ tới bằng cách chỉ định địa chỉ mới mà nó trỏ đến.

Một vùng nhớ có thể có một hoặc nhiều con trỏ chỉ đến.

Ví dụ 7.6:

```
#include <iostream.h>
void main()
{
    int *pmeo,*pcumeo;
    int chuot = 5;
    int ca = 12;
    pmeo = &chuot;
    pcumeo = &chuot;
    cout << "gia tri chuot: " << chuot;
    cout << "\ndia chi chuot: "<< &chuot;
    cout << "\ngia tri *pmeo: "<<*pmeo;
    cout << "\ngia tri pmeo: " << pmeo;
    cout << "\ndia chi pmeo: "<< & pmeo;
    cout << "\ngia tri *pcumeo: "<<*pcumeo;
    cout << "\ngia tri pcumeo: " << pcumeo;
    cout << "\ndia chi pmeo: "<< & pcumeo;
    chuot = 6;
```

```
    cout << "\ngia tri chuot: " << chuot;
    cout << "\ngia tri *pmeo: "<<*pmeo;
    *pmeo = *pmeo + 1;
    cout << "\ngia tri chuot: " << chuot;
    cout << "\ngia tri *pmeo: "<<*pmeo;
    cout << "\n";
    pmeo = &ca;
    cout << "\ngia tri ca: " << ca;
    cout << "\ngia tri *pmeo: "<<*pmeo;
    cout << "\ndia chi ca: " << &ca;
    cout << "\ngia tri pmeo: "<<pmeo;
    cout << "\ndia chi pmeo: "<< & pmeo;
}
```

Nhận xét: Con trỏ pmeo có thể thay đổi địa chỉ mà nó trỏ tới.

Con trỏ của một kiểu dữ liệu nào đó chỉ có thể trỏ tới địa chỉ vùng nhớ chứa giá trị của kiểu dữ liệu đó.

Ví dụ 7.7:

```
int *pmeo;
float * pt;
int chuot = 12;
float diem = 4.5;
pmeo = & chuot;
pmeo = &diem; // Câu lệnh này sẽ gây ra lỗi type mismach không thể
                //chuyển đổi  giá trị float * sang int *
```

Địa chỉ của một biến không phải là mộ số nguyên. Ta không thể thực hiện phép gán con trỏ với một số nguyên nào đó.

Câu lệnh sau **không** hợp lệ:

```
pmeo = 0x0012FF78;
```

Những câu lệnh sau hợp lệ:

```
int *pmeo;
pmeo = (int *)0x0012FF78;
pmeo = pmeo + 1;
```

4.5.2. Con trỏ và toán tử new

Toán tử new cho phép chúng ta lấy địa chỉ một vùng nhớ với kích thước phù hợp với kiểu dữ liệu mà ta khai báo trước.Vùng nhớ được cấp phát trên gọi là vùng nhớ cấp phát động chỉ có thể truy xuất thông qua một con trỏ. Khi bộ nhớ

đầy, toán tử new trả về địa chỉ 0 hay còn gọi là con trỏ null. Trình biên dịch sẽ nhận biết vùng nhớ này đang có "nhiệm vụ" lưu giữ giá trị của một biến nào đó và sẽ không cung cấp cho một biến khác.

Cú pháp khai báo:

dữ_liệu tên_con_trỏ;

tên_con_trỏ = new **dữ_liệu**;

hoặc

dữ_liệu tên_con_trỏ = new **dữ_liệu**;

Ví dụ 7.8:

```
int *pmeo;
pmeo = new int;
```

hoặc

```
int * pmeo = new int;
```

Vậy pmeo đã được chỉ định đến một vùng nhớ mới lưu trữ giá trị thuộc kiểu số nguyên. Đây là vùng nhớ mà trình biên dịch mới cung cấp, nó không có tên đại diện và chưa có giá trị khởi tạo. Muốn truy xuất đến vùng nhớ này ta phải thông qua con trỏ.

Ví dụ 7.9:

```
#include <iostream.h>
void main()
{
        int *pmeo;
        pmeo = new int;
        *pmeo  = 1;
        cout << "dia chi vung nho pmeo tro den: "<<pmeo;
        cout << "\ngia tri cua vung nho: "<<*pmeo;
}
```

Kết quả:

```
dia chi vung nho pmeo tro den: 0x00441DF0
ngia tri cua vung nho: 1
```

4.5.3. Con trỏ và toán tử delete

C++ cung cấp toán tử delete để giải phóng vùng nhớ này. Sau khi được giải phóng, vùng nhớ này có thể được cấp phát cho một biến khác.

Cú pháp:

```
delete      con_trỏ;
```

Biến con_trỏ phải được chỉ định vùng nhớ mà ta muốn giải phóng. Chỉ những con trỏ chỉ đến vùng nhớ được cấp phát động mới có thể dùng toán tử delete, ngược lại sẽ dẫn đến lỗi khi chương trình hoạt động. Toán tử delete này vô tình giải phóng cho những vùng nhớ cố định đang dành cho một biến khác hoặc một đoạn lệnh nào đó.

Ví dụ 7.10:

```
#include <iostream.h>
void main()
{
int *pmeo;
    pmeo = new int;
    *pmeo = 1;
    cout << "dia chi vung nho pmeo tro den: "<<pmeo;
    cout << "\ngia tri cua vung nho: "<<*pmeo;
    delete pmeo;
    cout << "\ndia chi vung nho pmeo tro den: "<<pmeo;
    cout << "\ngia tri cua vung nho: "<<*pmeo;
    int chuot = 6;
    pmeo = &chuot;
    cout << "\ndia chi vung nho pmeo tro den: "<<pmeo;
    cout << "\ngia tri cua vung nho: "<<*pmeo;
}
```

Kết quả:

```
dia chi vung nho pmeo tro den: 0x00441DF0
gia tri cua vung nho: 1
dia chi vung nho pmeo tro den: 0x00441DF0
gia tri cua vung nho: -572662307
dia chi vung nho pmeo tro den: 0x0012FF78
gia tri cua vung nho: 6
```

4.5.4. Mảng động

Ta có thể sử dụng toán tử new để tạo ra một mảng mà số phần tử xác định trong lúc chương trình đang hoạt động.

Cú pháp:

dữ_liệu con_trỏ = new kiểu_**dữ_liệu** [*số_phần_tử*];

Ví dụ 7.11:

```
int *  psonguyen = new int[3];
```

số_phần_tử là một giá trị nguyên có thể là một hằng (ở ví dụ trên là số 3) cũng có thể là một biến lưu giữ một giá trị nào đó.

Ví dụ 7.12:

```
int n = 5;
int *  psonguyen = new int[n];
```

Khi ta không muốn sử dụng mảng trên nữa, ta cũng có thể giải phóng vùng nhớ trên bằng toán tử delete.

Ví dụ 7.13:

```
delete  [] psonguyen;
```

Sau khi sử dụng toán tử new để cấp phát một vùng nhớ để tạo ra mảng động, địa chỉ mà con trỏ lưu giữ sẽ là địa chỉ của phần tử đầu tiên trong mảng. Để truy xuất đến phần tử thứ n thì ta phải xác định được địa chỉ của phần tử này.

Cú pháp truy xuất đến phần tử thứ n - 1:

```
* (con_trỏ + n )
```

Ví dụ 7.14:

```
#include <iostream.h>
void main()
{
    int *pmeo;
    pmeo = new int[3];
    *pmeo = 1;
    pmeo += 1;
    *pmeo = 2;
    *(pmeo+1) = 3;
    pmeo-=1;
    for( int i = 0; i< 3; i ++)
    {
        cout <<"\ngia tri phan tu "<< i <<": "<<*(pmeo + i);
        cout <<"\ndia chi phan tu "<< i <<": "<<pmeo + i;
    }
}
```

Kết quả:

```
gia tri phan tu 0: 1
dia chi phan tu 0: 0x00441DE0
gia tri phan tu 1: 2
```

```
dia chi phan tu 1: 0x00441DE4
gia tri phan tu 2: 3
dia chi phan tu 2: 0x00441DE8
```

Ngoài cách truy xuất như trên, ta có thể truy xuất như truy xuất phần tử của mảng tĩnh.

Cú pháp:

```
con_trỏ [chỉ_số_phần_tử]
```

Ví dụ 7.15:

```
#include <iostream.h>
void main()
{
    int *pmeo;
    pmeo = new int[3];
    pmeo[0] = 1;
    pmeo[1] = 2;
    pmeo[2] = 3;
    for( int i = 0; i< 3; i ++)
    {
        cout <<"\ngia tri phan tu "<< i <<": "<<pmeo [i];
        cout <<"\ndia chi phan tu "<< i <<": "<<&pmeo [i];
    }
}
```

Kết quả:

```
gia tri phan tu 0: 1
dia chi phan tu 0: 0x00441DE0
gia tri phan tu 1: 2
dia chi phan tu 1: 0x00441DE4
gia tri phan tu 2: 3
dia chi phan tu 2: 0x00441DE8
```

Chú ý:

Cách truy xuất một mảng tĩnh thông qua tên của mảng cũng giống như cách truy xuất một mảng động thông qua con trỏ. Lý do của sự giống nhau này là C++ thật sự quản lý mảng cũng bằng con trỏ. Tên của một mảng tĩnh chính là địa chỉ của phần tử đầu tiên của mảng, cũng giống như con trỏ trỏ vào phần tử đầu tiên của mảng vậy. Nhưng khác nhau là địa chỉ mà tên của mảng tĩnh không thay đổi được.

Câu lệnh sau là **không** hợp lệ:

```
int a[3];
a = a + 1;
```

Câu lệnh sau là hợp lệ:

```
int * p = new int[3];
p = p + 1;
int a[3];
*(a + 1) = 3;
p = a;
```

Chú ý:

- Mảng tĩnh không thể sử dụng toán tử delete.
- Đối với mảng động, khi dùng toán tử delete thì con trỏ phải chỉ phần tử đầu tiên của mảng.

4.6. CON TRỎ VÀ THAM SỐ CỦA HÀM

Tham số truyền vào hàm có hai cách, tham trị hoặc tham chiếu. Ngoài cách truyền biến tham chiếu như ta đã học ta có thể sử dụng con trỏ để truyền tham số theo kiểu tham chiếu.

4.6.1. Tham số là kiểu dữ liệu khác kiểu mảng

Khai báo prototype:

```
dữ_liệu   Tên_hàm (dữ_liệu_1  * con_trỏ_1,…,dữ_liệu_n * con_trỏ_n);
```

Cú pháp định nghĩa hàm:

```
dữ_liệu   Tên_hàm (dữ_liệu_1  * con_trỏ_1,…,dữ_liệu_n * con_trỏ_n)
{
    <lệnh>
}
```

Cú pháp gọi hàm:

```
Tên_hàm( & biến 1, … , & biến n);
```

Ý nghĩa:

Tham số là những con trỏ , những con trỏ này sẽ có địa chỉ vùng nhớ mà nó chỉ tới tương ứng là các địa chỉ của đối số truyền vào. Vậy mọi thao tác mà chúng ta xử lý đều thực hiện trên các đối số hay mọi sự thay đổi trên tham số truyền vào đều làm thay đổi các đối số tương ứng.

Ví dụ 7.20:

```
#include <iostream.h>
void doicho( int * a, int * b);
```

```
void main()
{
    int x, y;
    x = 5;
    y = 7;
    cout <<"x = " << x;
    cout <<"\ny = "<< y;
    doicho(&x,&y);
    cout <<"\nSau khi doi cho: ";
    cout <<"\nx = " << x;
    cout <<"\ny = "<< y;
}
void doicho(int * a, int * b)
{
    int t;
    t = * a;
    *a = * b;
    *b = t;
}
```

4.6.2. Tham số là kiểu dữ liệu mảng một chiều:

Cú pháp khai báo hàm:

dữ_liệu Tên_hàm (**dữ_liệu**_1 * con_trỏ_1,…);

Cú pháp định nghĩa hàm:

```
dữ_liệu   Tên_hàm (dữ_liệu_1  * con_trỏ_1,…)
{
    <lệnh>
}
```

Cú pháp gọi hàm:

Tên_hàm(biến_1, … , biến_n);

Ý nghĩa:

Tham số truyền vào là những con trỏ có cùng kiểu dữ liệu với các phần tử của màng dùng làm đối số.Những con trỏ này sẽ có địa chỉ vùng nhớ mà nó chỉ tới tương ứng là các địa chỉ của phần tử đầu tiên của đối số truyền vào. Vì tên của mảng là địa chỉ đầu tiên của mảng nên khi truyền vào một hàm, ta không thêm toán tử & trước tên đối số.

Ví dụ 7.22:

```
#include <iostream.h>
void xuat( int * a, int n);
void main()
{
    int mang [] = {1,2,3,5,6};
    xuat (mang,5);
}
void xuat( int * a, int n)
{
    int i = 0;
    for (; i < n; i++)
        cout << "\t" << a[i];
}
```

Kết quả:

```
1   2   3   5   6
```

Chú ý:

- Khi truyền vào một mảng ta dùng cách khai báo như trên hoặc như cách chúng ta đã học ở chương 3. Về bản chất, cả hai cách đều truyền địa chỉ của phần tử đầu tiên của mảng vào hàm.
- Đối với chuỗi cũng tương tự mảng một chiều. Tuy nhiên trong một vài trường hợp, chúng ta có thể không cần truyền vào độ dài của chuỗi vì độ dài của chuỗi ta có thể xác định được.

4.6.3. Tham số là kiểu dữ liệu mảng hai chiều

Cú pháp khai báo hàm:

dữ_liệu Tên_hàm (**dữ_liệu_1** (* con_trỏ_1)[*số_phần_tử*],…);

Cú pháp định nghĩa hàm:

dữ_liệu Tên_hàm (**dữ_liệu_1** (* con_trỏ_1)[*số_phần_tử*],…);
{
    <lệnh>
}

Cú pháp gọi hàm:

Tên_hàm(biến_1, … , biến_n);

Ý nghĩa:

Mảng hai chiều là mảng một chiều mà từng phần tử của nó cũng là mảng một chiều. Để truyền mảng hai chiều vào một hàm nào đó, ta cũng phải truyền vào

địa chỉ mảng, nhưng đồng thời ta cũng thông báo cho trình biên dịch ta đang truyền vào địa chỉ một mảng 2 chiều. Vì vậy tham số trong danh sách tham số của hàm phải là một con trỏ chỉ đến một mảng có n phần tử với n phải là hằng số.

Ví dụ 7.23:

```
#include <iostream.h>
void xuatm2c(int (*a)[2],int n);
void main()
{
    int mang [][2] = {{1,2},{3,5},{6,9}};
    xuatm2c (mang,3);
}
void xuatm2c(int (*a)[2],int n)
{
    int i,j;
    for (i = 0; i< n; i++)
    {
        for (j = 0; j< 2; j++)
            cout <<a[i][j] << "\t";
        cout << "\n";
    }
}
```

Dấu ngoặc đơn trước và sau *a là bắt buộc. Nếu không có dấu ngoặc đơn, trình biên dịch sẽ hiểu chúng ta truyền vào một mảng a có 2 phần tử là con trỏ **int**. Nhưng tham số không thể là một mảng, nên trình biên dịch sẽ báo lỗi. Chúng ta để dấu ngoặc đơn ở đó để trình biên dịch hiểu đây là con trỏ mảng 2 phần tử số nguyên.

4.7. CON TRỎ HÀM

4.7.1. Khai báo và sử dụng

Giống như tên của mảng, tên của hàm chính là địa chỉ của hàm. *Ví dụ* doigiatri(**int**&a, **int** &b) là một hàm, thì doigiatri là địa chỉ của hàm đó.

Cú pháp khai báo một con trỏ hàm:

Khai báo một hàm như sau:

Dữ_liệu Tên_hàm (<danh sách tham số>) ;

Trong một hàm nào đó, ta khai báo con trỏ hàm chỉ tới hàm trên:

Dữ_liệu (*con_trỏ) (<danh sách tham số>);

con_trỏ = Tên_hàm;

Danh sách tham số có thể theo các dạng sau:

Dữ_liệu_1 biến_1, **Dữ_liệu_2** biến_2, …, **Dữ_liệu_n** biến_n

hoặc

Dữ_liệu_1, **Dữ_liệu_2**, …, **Dữ_liệu_n**

Kiểu dữ liệu trả về, số lượng và kiểu dữ liệu của danh sách tham số của hàm và con trỏ hàm phải giống nhau, nếu có sử dụng tên biến thì tên biến có thể khác nhau. Danh sách tham số có thể là một, nhiều hoặc không có tham số nào.

Ví dụ 7.24:

Khai báo hàm:

int sosanh (**int** , **int**);

Khai báo con trỏ hàm:

int (*pham) (**int**, **int**);

pham = sosanh; // chỉ định pham chỉ tới hàm sosanh

Nhận xét:

- pham là con trỏ của hàm sosanh.
- Dấu ngoặc đơn trước và sau *pham là bắt buộc. Nếu không có dấu ngoặc đơn, trình biên dịch sẽ hiểu đây là một hàm có tên pham và *kết quả* trả về một con trỏ **int**.

Một con trỏ hàm có thể trỏ đến nhiều hàm có cùng dạng (có cùng kiểu dữ liệu trả về và danh sách tham số).

Ví dụ 7.25:

Khai báo hàm:

int sosanhlon(**int**, **int**);

int sosanhnho(**int**, **int**);

Khai báo con trỏ hàm:

int (*pHam) (**int**, **int**);

pHam = sosanhlon; // chỉ định pHam chỉ tới hàm sosanhlon

pHam = sosanhnho;// chỉ định pHam chỉ tới hàm sosanhnho

Thông thường hàm được gọi thực hiện thông qua tên của hàm. Nếu ta muốn gọi thực hiện hàm thông qua con trỏ hàm ta dùng toán tử * .

Cú pháp như sau:

(* con_trỏ) (<danh sách đối số>);

Ví dụ 7.26:

```
#include <iostream.h>
int sosanhlon(int, int);
int sosanhnho(int, int);
void main()
{
    int kqHam = 0,kqpHam = 0;
    int (*pHam) (int,int);
    pHam = sosanhlon;
    kqHam = sosanhlon(4,2);
    kqpHam = (*pHam)(4,2);
    cout << kqHam;
    cout <<"\n" << kqpHam;
    pHam = sosanhnho;
    kqHam = sosanhnho(4,2);
    kqpHam = (*pHam)(4,2);
    cout <<"\n" << kqHam;
    cout <<"\n" << kqpHam;
}
int sosanhlon (int a, int b)
{
    if (a > b)
        return 1;
    else
        return 0;
}
int sosanhnho (int a, int b)
{
    if (a < b)
        return 1;
    else
        return 0;
}
```

Kết quả:

```
1
1
0
```

0

4.7.2. Truyền hàm vào hàm

Truyền một hàm vào hàm khác chúng ta phải khai báo tham số truyền vào là một con trỏ hàm có cùng dạng với hàm mà chúng ta muốn truyền và đối số chính là tên hàm mà chúng ta muốn gọi.

Cú pháp khai báo prototype:

dữ_liệu Tên_hàm_A (<danh sách tham số>,

dữ_liệu_p (*con_trỏ_hàm) (<danh sách tham số>));

Cú pháp định nghĩa hàm:

dữ_liệu Tên_hàm_A (<danh sách tham số>,

dữ_liệu_p (*con_trỏ_hàm) (<danh sách tham số>));

{

<lệnh>

}

Cú pháp gọi thực hiện hàm:

Tên_hàm_A (<danh sách đối số>, tên_hàm_B);

Ví dụ 7.27::

```
#include <iostream.h>
int sosanhlon(int, int);
int sosanhnho(int, int);
void sapxep( int [],int, int (*pHam) (int,int));
void xuat( int * a, int n);
void main()
{
    int mangA[] = {1,3,7,2,9,5};
    int mangB[] = {5,3,7,1,8,2};
    cout << "Mang A sap xep tang:";
    sapxep(mangA,6,sosanhlon);
    xuat(mangA,6);
    cout << "Mang B sap xep giam:";
    sapxep(mangB,6,sosanhnho);
    xuat(mangB,6);
}
void sapxep( int a[],int n, int (*pHam) (int,int))
```

```
{
    int i, j, k;
    for ( i =0 ; i< n - 1; i++)
    {
        k = i;
        for ( j = i + 1; j< n; j++)
            if ( (*pHam)(a[k],a[j]) )
                k = j;
        if (k != i)
            doigiatri(&a[k],&a[i]);
    }
}
int sosanhlon (int a, int b)
{
    if (a > b)
        return 1;
    else
        return 0;
}
int sosanhnho (int a, int b)
{
    if (a < b)
        return 1;
    else
        return 0;
}
void xuat( int * a, int n)
{
    int i = 0;
    for (; i < n; i++)
        cout << "\t" << a[i];
    cout << "\n";
}
```

4.8. BÀI TẬP MINH HOẠ

1. Nhập vào 1 mảng n phần tử. Thực hiện các yêu cầu sau:

- Tính tổng của các phần tử mà chỉ số là các số chẳn, số lẽ, chia hết cho 5, 13, là số nguyên tố, số chính phương

- Sắp xếp mảng tăng giảm mà vẫn giữ nguyên vị trí chẳn lẽ
- Sắp xếp các số chẳn tăng dần nằm phía đầu mảng, các số lẻ giảm dần nằm phía cuối mảng
- Sắp xếp các số âm tăng dần ở đầu mảng , số dương giảm dần ở cuối mảng

2. Viết chương trình (dùng hàm) xữ lý các công việc sau:

- Nhập điểm môn toán của 20 học sinh
- Xuất ra điểm cao nhất, thấp nhất
- Xuất ra điểm cao nhất, 5 điểm thấp nhất
- Tính số lương học sinh từ có điểm từ 9 đến 10 điểm, từ 7 đến 9 điểm, từ 5 đến 7 điểm, và nhỏ hơn 5 điểm
- Xuất ra điểm trung bình của cả lớp
- Sắp xếp điểm theo thứ tự tăng dần, giảm dần
- Sắp xếp điểm lớn hơn 5 ở đầu danh sách theo thứ tự giảm dần, điểm nhỏ hơn 5 cuối danh sách theo thứ tự tăng dần

4. Làm chương trình công, trừ, nhân, chia trên số lớn (lớn hơn 50 chữ số)

5. Viết chương trình xữ lý một mảng 2 chiều số nguyên có 5 dòng 6 cột. Thực hiện các công việc sau:

- Nhập giá trị cho mảng 2 chiều
- Xuất mảng 2 chiều theo định dạng cột và hàng
- Tính tổng của từng hàng, từng cột
- Sắp xếp theo thứ tự tăng dần, giảm dần từng hàng của mảng
- Sắp xếp theo thứ tự tăng dần, giảm dần từng cột của mảng
- Tính số lượng các phần tử có giá trị chẳn, lẽ.

CHƯƠNG 5 KIỂU CHUỖI

5.1. KIỂU KÝ TỰ

Kiểu dữ liệu ký tự có miền giá trị là các ký tự trong bảng mã ASII:

A B C … Z

a b c … z

0 1 2 … 9

* & …

Cú pháp : **char** tên_biến;

Ví dụ 5.1 **char** ch;

Mỗi biến lưu giữ một ký tự. Để biểu diễn giá trị của biến kiểu **char** trong C++ ta đặt các ký tự trong dấu nháy đơn. Trong bộ nhớ mỗi ký tự được biểu diễn bởi 1 byte.

Thực chất kiểu ký tự **char** biểu diễn số nguyên từ 0 đến 255. Mỗi con số được quy ước tương ứng với một ký tự trong bảng mã ASII.

Ví dụ 5.2:

48 biểu diễn số 0

49 biểu diễn số 1

97 biểu diễn chữ a

Khi sử dụng kiểu **char** để tính toán như một số nguyên, ta có thêm các từ khoá signed hoặc unsigned để quy định số nguyên có dấu hoặc không có dấu tương tự như số nguyên kiểu **int**

signed **char**	-128 đến 127
unsigned **char**	0 đến 255

5.2. HẰNG KÝ TỰ - HẰNG CHUỖI

Hằng ký tự là một ký tự được đặt giữa 2 dấu nháy đơn. Ví dụ: ‘a’ ‘b’ …

Ngoài ra C++ cũng thêm một số hằng ký tự đặc biệt sau:

\b	Khoảng trắng
\n	Dòng mới
\r	Về đầu dòng

\t	một khoảng tab
\”	Dấu nháy kép
\’	Dấu nháy đơn
\0	Null
\\	Dấu \

Hằng chuỗi là một nhóm các ký tự được bỏ vào trong hai dấu nháy kép. Ví dụ: **“hang chuoi” .**

Ví dụ 5.3:

```
#include <iostream.h>
void main()
{
    ch = '0';
    cout <<ch;
    ch =  48;
    cout <<'\t'<< ch;
    cout << "xuong hang \n dau hang moi ";
}
```

Kết quả:

```
0 0xuonghang
dauhangmoi
```

5.3. KIỂU CHUỖI

Chuỗi là một dãy các ký tự liên tiếp nhau. Trong C++, chuỗi là mảng một chiều các ký tự mà phần tử cuối cùng của mảng là ký tự ‘\0’ (mã 0 trong bảng mã ASCII) để đánh dấu kết thúc chuỗi.

Ví dụ 5.4:

Chuỗi : “ngon ngu” là một dãy liên tiếp các ký tự

n	g	o	n	n	g	u	\0

Ví dụ 5.5:

```
char  dienthoai [5] = {‘n’ , ‘o’, ‘k’ , ‘i’ , ‘a’};   // không phải là chuỗi
char   chucnang[5] = {‘n’ , ‘g’, ‘h’, ‘e’ , ‘\0’};  // Là 1 chuỗi
```

Ngoài cách khởi tạo giá trị cho một chuỗi như cách khởi tạo giá trị cho một mảng một chiều thông thường, ta có thể khởi tạo chuỗi theo cách sau:

char ten_chuỗi[*số_ký_tự*]= "*chuỗi_hằng*";

hoặc

char tên_chuỗi[] = "*chuỗi_hằng*";

Ký tự '\0' được tự động thêm vào cuối mỗi chuỗi. Ở trường hợp 2, trình biên dịch tự đếm số ký tự trong chuỗi hằng và cộng thêm 1 cho ký tự '\0' kết thúc chuỗi và cung cấp đủ bộ nhớ cho chuỗi vừa khai báo.

Ví dụ 5.6:

char hinh[10] = "tam giac";

char yeucau[] = "dien tich";

Lưu trữ trong bộ nhớ như sau:

hinh

t	a	m	g	i	a	c	\0	\0	\0

yeucau

d	i	e	n		T	i	c	h	\0

Nhận xét:

- Nếu mảng các ký tự có số phần tử lớn hơn số ký tự của chuỗi hằng thì các ký tự '\0' tự động gán cho các ký tự còn lại.
- Khi chúng ta khai báo một mảng các ký tự lưu trữ một chuỗi, chúng ta phải khai báo số phần tử của mảng phải bao gồm cả ký tự '\0'.

Chỉ có thể khởi tạo giá trị đầu ngay sau khi khai báo biến. Dưới đây là cách khởi tạo không hợp lệ:

char hinh [10];

hinh = "tam giac"; // **không hợp lệ**

5.4. CÁC HÀM NHẬP, XUẤT CHUỖI

5.4.1. cout và toán tử <<

Toán tử << của **cout** được sử dụng để xuất ra một chuỗi. **cout** lần lượt đọc giá trị của các phần tử của chuỗi cho đến ký tự '\0' và xuất ra màn hình.

Ví dụ 5.7:

```
#include <iostream.h>
void main()
{
    char hinh[10] = "tam giac";
    cout << hinh;
```

```
}
```

Kết quả:

```
tam giac
```

5.4.2. cin và toán tử >>

Ngoài cách khởi tạo giá trị đầu cho chuỗi chúng ta có thể nhập giá trị chuỗi vào từ bàn phím. Toán tử >> của **cin** có thể đọc các ký tự nhập vào từ bàn phím và gán giá trị cho chuỗi được khai báo trong chương trình.

Ví dụ 5.8:

```
char ten[8];
cout << "nhap ten: " ;
cin >> ten;
cout << tên;
```

Nhận xét:

- Nếu nhập vào chuỗi **giang** *Kết quả* chuỗi xuất ra là **giang**.
- Nếu nhập vào chuỗi **huong giang** kết quả chuỗi xuất ra là **huong**.

cin sử dụng ký tự khoảng trắng, tab, dòng mới (newline) làm dấu hiệu kết thúc chuỗi nhập từ bàn phím. Có nghĩa là **cin** chỉ đọc vào một từ, sau đó tự động thêm ký tự kết thúc chuỗi và đưa giá trị vào chuỗi đã được khai báo. Vì vậy ten chỉ lưu giá trị từ đầu tiên **huong**.

Ngoài ra, nếu chuỗi mà ta khai báo có số phần tử nhỏ hơn số phần tử mà ta nhập vào bằng **cin** thì sẽ dẫn tới chuỗi trở thành mảng các ký tự và không có ký tự kết thúc chuỗi.

5.4.3. cin và getline()

Phương thức getline () được xây dựng trong đối tượng **cin**. Phương thức này giúp chúng ta kiểm soát được bao nhiêu ký tự đọc vào.

Phương thức **getline()** sẽ đọc toàn bộ một dòng cho đến gặp ký tự newline. Phương thức này có hai tham số, tham số đầu là mảng ký tự lưu giữ chuỗi, tham số thứ 2 là số ký tự nhập vào bao gồm cả ký tự '\0' kết thúc chuỗi (có nghĩa nếu ta truyền vào tham số thứ hai là n thì chỉ đọc n-1 ký tự). Hàm dừng đọc khi một trong hai điều kiện thỏa mãn: đọc đủ số ký tự mà chúng ta đưa vào hoặc gặp ký tự newline. Phương thức **Getline()** đọc ký tự newline và tự động chuyển đổi thành ký tự '\0' để lưu vào chuỗi.

Ví dụ 5.9:

```
#include <iostream.h>
void main ()
{
```

```
    char ten [15];
    char nghe[15];
    cout << "ten:";
    cin.getline(ten,15);
    cout << "\n nghe nghiep: ";
    cin.getline(nghe,15);
    cout << "\n Thong tin vua nhap ";
    cout << "\n Ten: " << ten;
    cout << "\n Nghe: " << nghe;
}
```

Kết quả:

ten: **huong giang**
nghe nghiep: **giao vien**
Thong tin vua nhap
Ten: **huong giang**
Nghe: **giao vien**

5.4.4. cin và get()

Phương thức get() của đối tượng của **cin** cũng tương tự về tham số cũng như cách thức hoạt động hàm **getline()**. Khác nhau ở chỗ hàm get() để lại ký tự newline trong bộ nhớ đệm.

Nếu ta sử dụng 2 hàm get() liên tiếp nhau như sau:

```
cin.get(ten,15);
cin.get(nghe,15);
```

Hàm get thứ hai được gọi nhưng không đọc được ký tự nào, nó không thể bỏ qua ký tự newline.

Tuy nhiên phương thức get() không có tham số của **cin** có thể đọc được 1 ký tự bất kỳ kể cả ký tự newline. Chúng ta có thể sử dụng hàm này đọc ký tự newline để đọc dòng tiếp theo.

```
cin.get(ten,15);
cin.get();
cin.get(nghe,15);
```

Ví dụ 5.10:

```
#include <iostream.h>
void main ()
{
    char ten [15];
    char nghe[15];
```

```
        cout << "ten:";
        cin.get(ten,15);
        cout << "\n nghe nghiep: ";
        cin.get();
        cin.get(nghe,15);
        cout << "\n Thong tin vua nhap ";
        cout << "\n Ten: " << ten;
        cout << "\n Nghe: " << nghe;
    }
```

Kết quả:

ten: **huong giang**
nghe nghiep: **giao vien**
Thong tin vua nhap
Ten: **huong giang**
Nghe: **giao vien**

Ngoài cách sử dụng hàm get không có đối số, ta có thể sử dụng hàm get có 1 đối số như sau:

```
char ch;
cin.get(ch);
```

Hàm get có một đối số có thể đọc được mọi ký tự kể cả ký tự newline. Nếu sử dụng phương thức get của **cin** ta có thể kiểm tra phương thức get đã đọc hết dòng hay chưa, có còn ký tự trong bộ nhớ đệm sau khi hàm get được gọi hay không. Điều này sẽ tránh được lỗi xảy ra khi người dùng nhập một chuỗi quá dài so với số ký tự mà ta cho phép nhập vào.

Ví dụ 5.11:

```
#include <iostream.h>
void main ()
{
    char ten [15];
    char nghe[15];
    char ch;
    cout << "ten:";
    cin.get(ten,5);
    cout << "\n nghe nghiep: ";
    ch = 0;
    while (ch != 10)
        cin.get(ch);
```

```
    cin.get(nghe,15);
    cout << "\n Thong tin vua nhap: ";
    cout << "\n Ten: " << ten;
    cout << "\n Nghe: " << nghe;
}
```

Kết quả:

ten: **huong giang**
nghe nghiep: **giao vien**
Thong tin vua nhap
Ten: **huon**
Nghe: **giao vien**

5.4.5. Nhập số và chuỗi

Khi chúng ta nhập số và chuỗi cùng nhau, lỗi rất dễ xảy ra.

Ví dụ 5.12:

```
#include <iostream.h>
void main()
{
    char ten [15];
    char nghe[15];
    int tuoi = 0;
    cout << "ten:";
    cin.getline(ten,15);
    cout << "tuoi: ";
    cin  >> tuoi;
    cout << "\nnghe nghiep: ";
    cin.get(nghe,15);
    cout << "\nThong tin vua nhap: ";
    cout << "\nTen: " << ten;
    cout << "\nTuoi: "<< tuoi;
    cout << "\nNghe: "<< nghe;
}
```

Kết quả:

ten: **huong giang**
tuoi:23

nghe nghiep:
Thong tin vua nhap

Ten: **huong giang**
Tuoi: 23
Nghe:

Chương trình viết lại như sau:

```
#include <iostream.h>
void main()
{
    char ten [15];
    char nghe[15];
    int tuoi = 0;
    cout << "ten:";
    cin.getline(ten,15);
    cout << "tuoi: ";
    cin  >> tuoi;
    cin.get();
    cout << "\nnghe nghiep: ";
    cin.get(nghe,15);
    cout << "\nThong tin vua nhap: ";
    cout << "\nTen: " << ten;
    cout << "\nTuoi: "<< tuoi;
    cout << "\nNghe: "<< nghe;
}
```

5.4.6. Một số hàm xử lý trên chuỗi

Các hàm xử lý trên chuỗi đã được định nghĩa trong thư viện string.h. Muốn sử dụng các hàm này, ta phải khai báo sử dụng thư viện này.

5.4.6.1. strlen

Cú pháp:

```
strlen(s);
```

Ý nghĩa: trả về giá trị là độ dài của chuỗi s.

Ví dụ 5.13:

```
#include <iostream.h>
#include <string.h>
void main()
{
    char ten [15] = "huong giang";
```

```
        int len ;
        len = strlen(ten);
        cout << len;
    }
```

Kết quả:

```
        11
```

5.4.6.2. strcpy

Cú pháp:

```
        strcpy (s1, s2);
```

Ý nghĩa: chép s2 cho s1.

Ví dụ 5.14:

```
    #include <iostream.h>
    #include <string.h>
    void main()
    {
        char ten [15] = "huong giang";
        char tennhap [15];
        strcpy (tennhap,ten);
        cout << tennhap;
    }
```

Kết quả:

```
        huong giang
```

Nhận xét:

strcpy đã copy giá trị của ten sang tennhap. Tuy nhiên, strcpy sẽ không kiểm tra số phần tử của chuỗi được nháp nhiều hay ít hơn số phần tử của chuỗi nguồn. Vì vậy, chúng ta phải chu ý sao cho chuỗi được chép luôn có số phần tử lớn hơn hoặc bằng chuỗi nguồn để tránh dữ liệu chép ra ngoài chuỗi và bị thay đổi ngoài ý muốn.

5.4.6.3. strcat

Cú pháp:

```
        strcat(s1,s2)
```

Ý nghĩa:

Nối chuỗi s2 vào cuối chuỗi s1.

Ví dụ 5.15:

```
#include <iostream.h>
#include <string.h>
void main()
{
    char ten [] = "huong giang";
    char ho [25] = “truong thi ”;
    strcpy (ho,ten);
    cout << ho;
}
```

Kết quả:

truong thi huong giang

Nhận xét:

Chuỗi được ghép với chuỗi nguồn phải có kích thước phần tử lơn hơn hoặc bằng tổng kích thước hai chuỗi.

5.4.6.4. strcmp

Cú pháp:

```
strcmp (s1, s2);
```

Ý nghĩa:

Trả về giá trị 0 nếu s1 và s2 như nhau, nhỏ hơn 0 nếu s1 nhỏ hơn s2, lớn hơn 0 nếu s1 lớn hơn s2.

Ví dụ 5.16:

```
#include <iostream.h>
#include <string.h>
void main()
{
    char ten [] = "huong giang";
    char ho [25] = “truong thi ”;
    int kq;
    kq = strcmp (ho,ten);
    cout << kq;
}
```

Kết quả

1

5.5. CON TRỎ VÀ CHUỖI

Mối quan hệ giữa con trỏ và chuỗi cũng tương tự con trỏ và mảng.

Cú pháp khai báo một con trỏ chỉ đến một chuỗi:

```
char   tên_chuôi[số_ký_tự];
char * con_trỏ;
con_trỏ = tên_chuôi;
```

Ví dụ 7.16:

```
#include <iostream.h>
void main()
{
    char hoa[20] = "hoa hong ";
    char *p = hoa;
    cout << hoa ;
    cout<< '\n' << p ;
}
```

Kết quả:

```
hoa hong
hoa hong
```

Cú pháp khai báo một con trỏ chỉ đến một chuỗi hằng:

```
char * con_trỏ = "chuỗi_hằng";
```

Ví dụ 7.17:

```
#include <iostream.h>
void main()
{
    char *mau = "mau hong";
    cout << mau ;
}
```

Kết quả:

```
mau hong
```

Để chắc chắn trong chương trình của chúng ta không thay đổi chuỗi hằng này, ta thêm từ khoá const. Nếu trong chương trình có sự thay đổi, trình biên dịch sẽ báo lỗi.

```
const char * mau= "mau hong";
```

Chú ý: Đối với **cout** và hầu hết các lệnh của C++, tên của một chuỗi, con trỏ **char** hoặc chuỗi hằng được hiểu là địa chỉ của phần tử đầu tiên của chuỗi.

Xét ví dụ sau:

```
#include <iostream.h>
```

```
void main()
{
    char hoa[20] = "hoa hong ";
    char *mau = "mau hong";
    cout <<hoa << mau << " rat dep";
}
```

Kết quả:

hoa hong mau hong rat dep

Nhận xét:

- Tên của mảng là địa chỉ của phần tử đầu tiên, nên **hoa** trong câu lệnh **cout** là địa chỉ phần tử đầu tiên của chuỗi (phần tử có chứa ký tự r). Do hoa là địa chỉ của một chuỗi nên **cout** sẽ tiếp tục đọc các phần tử kế tiếp in ra màn hình cho đến khi gặp ký tự null **(\0)**.
- Con trỏ **mau** chỉ tới địa chỉ một chuỗi hằng. Trong câu lệnh **cout**, **mau** cũng là địa chỉ phần tử đầu tiên của chuỗi hằng này.
- "rat dep" là một chuỗi hằng, "rat dep" đóng vai trò như là tên của một mảng char, nó truyền cho **cout** địa chỉ đầu tiên của chuỗi.

Khi truy xuất nội dung một chuỗi của một biến con trỏ kiểu char, ta chỉ nên dùng những biến đã được chỉ định địa chỉ của chuỗi mà nó trỏ tới hoặc đã sử dụng toán tử new khởi tạo một chuỗi cho con trỏ.

Ví dụ 7.18:

```
#include <iostream.h>
#include <string.h>
void main()
{
    char hoa[20];
    char * tam;
    cout << tam;
    cout <<"\nhoa ma ban thich";
    cin.getline(hoa);
    int len;
    len = strlen(hoa);
    tam = new char[len];
    strcpy(len,hoa);
    cout << hoa;
    cout << tam;
}
```

5.6. BÀI TẬP MINH HOẠ

1. Viết các hàm (không sử dụng hàm thư viện) thực hiện các yêu cầu sau:
   - Đổi chữ thường thành chữ hoa
   - Tính chiều dài của chuỗi
   - Đổi chuỗi thường thành chuỗi hoa
   - Đếm số từ trong chuỗi với dấu hiệu kết thúc từ là khoảng trắng
   - Tìm vị trí của một ký tự (chữ không phụ thuộc chữ hoa hay thường).
   - Kiểm tra chuỗi thứ nhất có thuộc chuỗi thứ hai không
   - Chép nội dung của chuỗi A vào chuỗi B với số ký tự cho trước
   - Đếm số lượng ký tự trong chuỗi bằng với một ký tự cho trước
2. Nhập vào danh sách 5 học sinh và thực hiện các yêu cầu sau:
   - Sắp xếp theo thứ tự của họ
   - Sắp xếp theo thứ tự của tên
   - Kiểm tra trong mỗi tên (họ và tên) xóa hết các khoảng trắng dư
   - Thêm các ký số 1,2 … cho các tên (họ và tên) trùng nhau
3. Viết chương trình quản lý tiệm bán hoa với các yêu cầu như sau:
   - Nhập vào danh sách tên 5 loại hoa với đơn giá bán (sử dụng 2 mảng)
   - Tìm giá bán của một loại hoa do người dùng nhập vào từ bàn phím.
   - Tìm loại hoa có giá bán cao nhất, thấp nhất.
   - Tìm giá bán trung bình của 5 loại hoa.
   - Xuất ra tất cả loại hoa có giá trị lớn hơn giá trị trung bình

CHƯƠNG 6 KIỂU DỮ LIỆU TỰ ĐỊNH NGHĨA

C++ cho phép bạn tạo ra các kiểu dữ liệu tuỳ ý bằng nhiều cách khác nhau.

6.1. KIỂU CẤU TRÚC

6.1.1. Cú pháp khai báo

Kiểu cấu trúc là kiểu dữ liệu do người dùng tự định nghĩa, cho phép gom nhóm nhiều kiểu dữ liệu khác nhau thành một kiểu dữ liệu mới để lưu giữ các thông tin liên quan với nhau. Các thông tin này được gọi là các trường.

Cú pháp:

```
struct  tên_cấu_trúc
{
    dữ_liệu_1     tên_trường_1;
    dữ_liệu_2     tên_trường_2;
    …
    dữ_liệu_n     tên_trường_n;
};
```

Ví dụ 6.1:

Khi lưu giữ thông tin của một cầu thủ bóng đá ta lưu giữ những thông tin sau: Tên, năm sinh, nơi sinh, câu lạc bộ.

```
struct  cau_thu
{
    char   ten[20];
    int namsinh;
    char   noisinh[20];
    char   clb[20];
};
```

Cú pháp khai báo một biến thuộc kiểu dữ liệu cấu trúc:

```
struct  tên_cấu_trúc
{
    dữ_liệu_1     tên_trường_1;
    dữ_liệu_2     tên_trường_2;
    …
    dữ_liệu_n     tên_trường_n;
} tên_biến_1, … , tên_biến_k;
```

Hoặc

tên_dữ_liệu_cấu_trúc tên_biến_1, … , tên_biến_k;

Ví dụ 6.2:

```
struct cau_thu
{
    char   ten[20];
    int namsinh;
    char   noisinh[20];
    char   clb[20];
} a,b;
```

Hoặc

```
cau_thu    a,b;
```

Một biến thuộc kiểu dữ liệu cấu trúc có kích thước tối thiểu bằng tổng kích thước của các trường của kiểu dữ liệu đó. Máy tính lưu trữ các trường này trên những ô nhớ liên tiếp nhau.

Ví dụ 6.3:

Biến a có kích thước là 1 x 20 + 2 x 1 + 1 x 20 + 1 x 20 = 62 byte

Chú ý:

Nếu khai báo kiểu dữ liệu cấu trúc ở ngoài hàm main và ở phần khai báo prototype gọi là khai báo toàn cục. Mọi hàm đều có thể sử dụng.

Cấu trúc và mảng đều là kiểu dữ liệu tạo thành từ các kiểu dữ liệu khác và có khả năng lưu trữ nhiều giá trị khác nhau. Tuy nhiên cấu trúc và mảng có những đặc điểm khác nhau như sau:

- Mảng có khả năng lưu trữ n phần tử khác nhau có cùng một kiểu dữ liệu.
- Cấu trúc có khả năng lưu trữ n trường khác nhau có thể cùng hoặc khác kiểu dữ liệu.
- Mảng được xây dựng để biểu diễn những thông tin (giá trị) có tính chất tương tự nhau. Mỗi thông tin là của một đối tượng (sự vật, sự việc) khác nhau.
- Cấu trúc được xây dựng để biểu diễn những thông tin (giá trị) có tính chất khác nhau để cùng mô tả một đối tượng (sự vật, sự việc) nào đó.

Ví dụ 6.4:

Tại một cửa hàng bán hoa, người ta muốn lưu giữ thông tin về các loại hoa và số lượng hoa bán được theo từng loại hoa.

Ta sẽ khai báo một kiểu dữ liệu cấu trúc mô tả loại hoa như sau:

```
struct loai_hoa
```

```
{
    char   ten[20];
    char   xuatxu[20];
    int giaban;
};
```

Để lưu giữ thông tin số lượng hoa bán ta sẽ sử dụng mảng các số nguyên.

6.1.2. Khởi tạo

Tương tự như các loại dữ liệu khác, khởi tạo giá trị đầu chỉ thực hiện ngay sau khi khai báo biến.

Cú pháp:

```
tên_cấu_trúc     tên_biến = {
    giá_trị_1 ,
    giá_trị_2,
    …
    giá_trị_n
};
```

Ví dụ 6.5:

```
loai_hoa   hoa =
        {
            "hoa hong nhung",
            "Da Lat",
            500
        };
```

6.1.3. Cách truy xuất các trường

Để truy xuất đến các trường của một cấu trúc ta sử dụng toán tử **.** (dấu chấm).

Cú pháp:

tên_cấu_trúc.*tên_trường*

Ví dụ 6.6:

```
#include <iostream.h>
#include <string.h>
struct loai_hoa
{
    char   ten[20];
```

```
    char   xuatxu[20];
    int giaban;
};
void main()
{
    loai_hoa   hoa =
        {
            "hoa hong nhung",
            "Da Lat",
            500
        };
    cout <<"\nHoa ten: "<< hoa.ten;
    cout <<"\nXuat xu: "<< hoa.xuatxu;
    cout <<"\nGia ban: "<< hoa.giaban;
}
```

Kết quả:

```
Hoa ten: hoa hong nhung
Xuat xu: Da Lat
Gia ban: 500
```

6.1.4. Phép gán trên cấu trúc

C++ cho phép bạn thực hiện phép gán trên cấu trúc. Lần lượt các trường của cấu trúc này được gán cho trường của cấu trúc kia, ngay cả khi trường là một kiểu dữ liệu chuỗi hay mảng.

Ví dụ 6.7:

```
#include <iostream.h>
#include <string.h>
struct loai_hoa
{
    char   ten[20];
    char   xuatxu[20];
    int giaban;
};
void main()
{
    loai_hoa   hoa,hoa1;
    strcpy(hoa.ten,"hoa hong nhung");
    strcpy(hoa.xuatxu,"Da Lat");
```

```
        hoa.giaban = 500;
        cout <<"\nHoa ten: "<< hoa.ten;
        cout <<"\nXuat xu: "<< hoa.xuatxu;
        cout <<"\nGia ban: "<< hoa.giaban;
        hoa1 = hoa;
        cout <<"\nHoa ten: "<< hoa1.ten;
        cout <<"\nXuat xu: "<< hoa1.xuatxu;
        cout <<"\nGia ban: "<< hoa1.giaban;
    }
```

Kết quả:

```
    Hoa ten: hoa hong nhung
    Xuat xu: Da Lat
    Gia ban: 500
    Hoa ten: hoa hong nhung
    Xuat xu: Da Lat
    Gia ban: 500
```

6.2. CON TRỎ VÀ KIỂU CẤU TRÚC

Tương tự như các kiểu dữ liệu cơ bản khác, ta có thể khai báo một con trỏ thuộc kiểu dữ liệu **struct** hay có thể tạo một mảng động kiểu **struct**.

Tuy nhiên đối với một biến bình thường kiểu **struct**, chúng ta dùng toán tử dấu chấm (.) để truy xuất đến các trường. Đối với một biến con trỏ thuộc kiểu **struct**, C++ cung cấp cho chúng ta toán tử -> .

Cú pháp:

```
    Con_trỏ -> tên_trường
```

Ví dụ 7.19:

```
#include <iostream.h>
#include <string.h>
struct loai_hoa
{
    char   ten[20];
    char   xuatxu[20];
    int gia[2];
};
void main ()
{
    loai_hoa * phoa;
    loai_hoa  hoa;
```

```
    phoa = &hoa;
    hoa.gia[0] = 800;
    phoa->gia[1] = 1000;
    strcpy(hoa.ten,"tuylip");
    strcpy(phoa->xuatxu ,"Ha Lan");
    cout <<"\nTen hoa: " <<phoa->ten;
    cout <<"\nXuat xu: " <<phoa->xuatxu;
    cout <<"\nGia mua: " <<phoa->gia[0];
    cout <<"\nGia ban: " <<phoa->gia[1];
}
```

Kết quả:

```
Ten hoa: tuylip
Xuat xu: Ha Lan
Gia mua: 800
Gia ban: 1000
```

Ngoài ra, như chúng ta đã biết, nếu phoa là con trỏ, thì *phoa là vùng nhớ lưu trữ giá trị biến đó hay nói cách khác *phoa là một biến kiểu **struct** bình thường nhưng không có tên riêng. Vậy khi truy suất đến các trường ta vẫn sử dụng toán tử dấu chấm (.) như sau: (*phoa).gia[0], (*phoa).ten … Lưu ý bắt buộc phải có dấu ngoặc đơn trước và sau *phoa.

6.3. TRUYỀN THAM SỐ KIỂU CẤU TRÚC

Tương tự như các kiểu dữ liệu cơ bản khác, một tham số thuộc kiểu dữ liệu cấu trúc vào hàm theo kiểu tham trị và kiểu tham chiếu. Khác với mảng, tham số truyền vào chỉ là tên của mảng (địa chỉ của mảng), đối với kiểu cấu trúc nếu tham số truyền vào theo kiểu tham trị thì hàm sẽ xử lý trên biến là bản sao của tham số, ngay cả khi các trường của cấu trúc thuộc kiểu mảng.

Ví dụ 6.8:

```
#include <iostream.h>
#include <string.h>
struct loai_hoa
{
    char   ten[20];
    char   xuatxu[20];
    int gia[2];
};
void taogiaban(loai_hoa h);
void main()
{
```

```
        loai_hoa   hoa =
            {"hoa hong nhung", "Da Lat", 500, 0 };
        taogiaban(hoa);
        cout <<"\nHoa ten: "<< hoa.ten;
        cout <<"\nXuat xu: "<< hoa.xuatxu;
        cout <<"\nGia mua: "<< hoa.gia[0];
        cout <<"\nGia ban: "<< hoa.gia[1];
    }
    void taogiaban(loai_hoa h)
    {
        h.gia[1] = h.gia[0] + 50;
    }
```

Kết quả:

Hoa ten: hoa hong nhung
Xuat xu: Da Lat
Gia mua: 500
Gia ban: 0

Nhận xét:

Giá bán vẫn bằng 0 và không có sự thay đổi. Chứng tỏ hàm taogiaban chỉ làm việc trên bản sao của hoa.

Chương trình sử dụng tham chiếu như sau:

```
    #include <iostream.h>
    #include <string.h>
    struct loai_hoa
    {
        char   ten[20];
        char   xuatxu[20];
        int gia[2];
    };
    void taogiaban(loai_hoa &h);
    void main()
    {
    loai_hoa   hoa =
            {"hoa hong nhung", "Da Lat", 500,     0 };
        taogiaban(hoa);
        cout <<"\nHoa ten: "<< hoa.ten;
        cout <<"\nXuat xu: "<< hoa.xuatxu;
```

```
        cout <<"\nGia mua: "<< hoa.gia[0];
        cout <<"\nGia ban: "<< hoa.gia[1];
    }
    void taogiaban(loai_hoa & h)
    {
        h.gia[1] = h.gia[0] + 50;
    }
```

Kết quả:

```
    Hoa ten: hoa hong nhung
    Xuat xu: Da Lat
    Gia mua: 500
    Gia ban: 550
```

Chú ý:

- Các trường của một cấu trúc có thể thuộc một kiểu dữ liệu cấu trúc khác.
- Cần phải xem xét khi nào cần thiết phải truyền theo kiểu tham trị. Vì mỗi lần truyền tham số kiểu tham trị, cần phải cung cấp thêm một vùng nhớ để lưu trữ biến mới là bản sao của tham số.

6.4. UNION

Một **Union** là một vùng nhớ được chia sẻ bởi hai hay nhiều biến khác nhau, thường là có kiểu khác nhau, tại những thời điểm khác nhau.

Cú pháp:

```
Union ten_Union {
    dữ_liệu_1    tên_biến_1;
    dữ_liệu_2    tên_biến_2;
    …
    dữ_liệu_n    tên_biến_n;
};
```

Ví dụ 6.9:

```
    Union    sobiendoi{
        int    nguyen;
        float    thuc;
        double    thuclon;
    };
```

Tại một thời điểm chúng ta sử dụng một biến thuộc một kiểu dữ liệu duy nhất.

Ví dụ 6.10:

```
void main()
{
Union     sobiendoi{
        int     nguyen;
        char    kytu;
        float   thuc;
    };
    sobiendoi soa;
    soa.nguyen = 98;
    cout <<"so nguyen: "<< soa.nguyen;
    cout <<"\nky tu: "<< soa.kytu;
    cout <<"\nso thuc: "<< soa.thuc;
    soa.thuc = 2.33;
    cout <<"\nso thuc: "<< soa.thuc;
    cout <<"\nnguyen: "<< soa.nguyen;
}
```

Kết quả:

```
so nguyen: 98
ky tu: B
so thuc: 1.3732 e – 043
so thuc: 2.32
nguyen: 1075125944
```

Nhận xét:

- Tại một thời điểm ta chỉ sử dụng một biến thuộc **Union** sobiendoi. Khi ta thay đổi giá trị một biến nào đó thì tất cả đều thay đổi vì chúng chia sẻ một vùng nhớ.
- Kích thước của một **Union** sẽ là kích thước của trường có kích thước lớn nhất.
- Ta thường sử dụng **Union** khi ta khai báo một biến mà có thể sử dụng theo nhiều kiểu dữ liệu khác nhau và các giá trị này không bao giờ sử dụng đồng thời. Như vậy sẽ tiết kiệm bộ nhớ.
- Nếu **Union** không được đặt tên thì thành viên của nó trở thành các biến chia sẻ với nhau một vùng nhớ, và tại mỗi thời điểm chỉ sử dụng 1 biến.

Ví dụ 6.11:

```
#include <iostream.h>
void main()
```

```
{
    Union {
        int nguyen;
        char    kytu;
        };
    nguyen = 23;
    cout <<nguyen;
    kytu = 'a';
    cout <<'\n'<< kytu;
}
```

Kết quả:

```
23
a
```

6.5. ENUMERATION

Một enum là một tập hợp của các hằng nguyên được đặt tên. Các hằng nguyên này là miền giá trị hợp lệ của biến thuộc kiểu dữ liệu enum được định nghĩa.

Cú pháp:

enum tên_enum {giá_trị_1, giá_trị_2, … , giá_trị_n};

tên_enum tên_biến;

Ví dụ 6.12:

Giá trị của tháng là từ tháng một đến tháng mười hai.

enum thang{mot, hai, ba, bon, nam, sau, bay, tam, chin, muoi, muoi_mot,muoi_hai};

thang t;

Nhận xét:

Biến t thuộc kiểu dữ liệu enum. Miền giá trị của biến t chỉ là mot, hai, ba,…, muoi_hai.

Các phát biểu sau là hợp lệ:

```
t = mot;
if ( t == hai)
    cout << "thang hai";
```

Các phát biểu sau không hợp lệ:

```
t = 2;
```

```
t = muoi_ba;
```

Mỗi giá trị của một enum tượng trưng cho một số nguyên. Giá trị bắt đầu thường có giá trị 0, mỗi giá trị đứng sau lớn hơn giá trị đứng trước 1 đơn vị.

Ví dụ 6.13:

```
#include <iostream.h>
void main()
{
enum  thang{mot, hai, ba, bon, nam, sau, bay, tam, chin, muoi,
muoi_mot,muoi_hai};
thang  t;
    t = mot;
    cout << '\n'<<t;
    t = hai;
    cout << '\n'<< t;
    t = muoi_hai;
    cout << '\n'<< t;
}
```

Kết quả:

```
0
1
11
```

Tuy nhiên ta có thể chỉ định giá trị nguyên cho mỗi giá trị của enum.

Ví dụ 6.14:

```
enum  doluong {mm = 1, cm = 10, dm = 100,m = 1000};
```

Chương trình:

```
#include <iostream.h>
void main()
{
    enum  doluong {mm = 1, cm = 10, dm = 100,m = 1000};
    cout << "\n" << mm;
    cout << "\n" << cm;
    cout << "\n" << dm;
    cout << "\n" << m;
}
```

Kết quả:

```
1
```

```
10
100
1000
```

Ta có thể có khởi tạo giá trị cho một biến enum như sau:

```
tên_biến   tên_enum(số_nguyên);
```

số_nguyên phải là giá trị số nguyên tương ứng với giá trị của enum. Nếu không có giá trị enum nào có giá trị nguyên tương ứng thì giá trị trả về cho biến enum là một giá trị không xác định.

Ví dụ 6.15:

```
#include <iostream.h>
void main()
{
    enum  doluong {mm = 1, cm = 10, dm = 100,m = 1000};
    doluong   donvi;
    donvi = doluong(10);
    if (donvi == cm)
        cout << donvi;
}
```

Kết quả:

```
10
```

Tuy nhiên ta không thể thực hiện các phép toán trên các biến thuộc kiểu dữ liệu enum như trên các số nguyên.

Những phát biểu sau không hợp lệ:

```
thang t;
t = mot * 2;
t = mot + hai;
```

Những phát biểu sau hợp lệ:

```
thang t;
int n;
t = mot;
n = mot + 2;
```

Người ta thường sử dụng enum không tên để định nghĩa một số hằng số.

Ví dụ 6.16:

```
enum { min = 10, max = 100, mid = 40};
```

Chương trình:

```
#include <iostream.h>
```

```
void main()
{
    enum { min = 10, max = 100, mid = 40};
    int n;
    cout << "nhap n: ";
    cin >> n;
    if ( n < min)
        cout << "so nho hon yeu cau toi thieu";
    else
    {
        if ( n < mid)
            cout << "so nho hon yeu cau trung binh";
        else
        {
            if ( n < max)
                cout << "so lon hon yeu cau trung binh";
            else
                cout << "so lon hon yeu cau toi da";
        }
    }
}
```

6.6. TYPEDEF

Từ khoá **typedef** cho phép định nghĩa một cách rõ ràng một tên kiểu dữ liệu mới. Một kiểu dữ liệu mới không thực sự được tạo ra mà chỉ là định nghĩa một tên mới cho một kiểu dữ liệu đã có sẵn. Ta có thể dùng **typedef** để mô tả lại các kiểu dữ liệu chuẩn một cách rõ ràng hơn, giúp chúng ta dễ sử dụng. Ngoài ra quá trình này còn giúp tạo ra một chương trình độc lập với máy tức không phụ thuộc vào kiểu dữ liệu chuẩn của máy. Mỗi lần thay đổi ta chỉ cần thay đổi tại khai báo **typedef**.

*Cách sử dụng **typedef**:*

typedef kiểu_dữ_liệu_cũ kiểu_dữ_liệu_mới;

Ví dụ 6.17:

```
typedef float sothuc;
```

Nhận xét:

Câu lệnh trên có tác dụng thông báo cho trình biên dịch biết sothuc là một tên khác của kiểu **float**. Chúng ta có thể khai báo một biến nào đó thuộc kiểu sothuc hay **float** ngay sau đó và chúng có cùng một kiểu dữ liệu.

Những câu lệnh sau là hợp lệ:

```
typedef float sothuc;
sothuc a;
float b;
a = 5.2;
b = a;
```

6.7. BÀI TẬP MINH HOẠ

1. Biểu diễn dữ liệu cho các đối tương sau: điểm, hình vuông, hình chữ nhât, hình tròn.
2. Nhập vào tọa độ của một điểm kiểm tra xem vị trí tương đối của điểm đó so với một hình vuông, hình chữ nhật, hình tròn đã biết trước.
3. Biễu diễn thông tin cho các đối tượng sau: thời gian, ngày tháng. Viết chương trình xử lý các yêu cầu sau:
   - Tính hiệu thời gian (nhập theo định dạng hh:mm:ss) theo đơn vị giây.
   - Tính hiệu ngày tháng (nhập theo định dạng dd/mm/yyyy) theo đơn vị ngày.
   - Viết ứng dụng tạo ra đồng điện tử, chương trình kết thúc khi người dùng bấm phím Esc.
4.Viết chương trình quản lý tiêm bán hoa với yêu cầu sau:
   - Nhập vào danh sách 5 loại hoa và lưu trữ các thông tin sau: Tên hoa, xuất xứ, giá mua, giá bán, số lượng mua, số lượng bán, số ngày bảo quản tối đa
   - Sắp xếp danh sách giảm dần theo thứ tự giá bán, giá mua, số lượng.
   - Tìm loại hoa có số ngày bảo quản tối đa
   - Tính tổng lượng hoa đã mua, lượng hoa đã bán
   - Tính lượng hoa chưa bán của mỗi loại hoa
   - Tính tổng tiền lãi bán của mỗi loại hoa
   - Tính tổng tiền lãi bán của tất cả loại hoa
5. Viết chương trình quản lý học sinh:
   - Nhập vào danh sách 5 học sinh và lưu trữ các thông tin sau: Họ và tên, ngày sinh, nơi sinh, lớp, điểm trung bình cuối học kỳ.
   - Sắp xếp danh sách học sinh theo tên, lớp.
   - Tính số tuổi của mỗi học sinh.
   - Đánh giá xếp loại mỗi học sinh biết:
     - 9 <= dtb < 10 : xuất sắc
     - 8 <= dtb < 9: giỏi
     - 7 <= dtb < 8: khá
     - 5 <= dtb < 7: trung bình
     - 0 <= dtb < 5: yếu
   - Tính tổng số học sinh theo từng loại của tất cả các lớp.
   - Tính số lương học sinh theo từng bậc xếploại của mỗi lớp.

CHƯƠNG 8 TỆP – FILE

8.1. KHÁI NIỆM TỆP

Tệp hay Tệp dữ liệu là một tập hợp các dữ liệu có liên quan với nhau được nhóm lại với nhau tạo thành một dãy. Chúng thường được chứa trong một thiết bị nhớ ngoài của máy tính (đĩa mềm, đĩa cứng,…) với một cái tên nào đó.

Hay nói cách khác, tệp là một đoạn byte được lưu trên các thiết bị bộ nhớ ngoài. Thông thường hệ điều hành sẽ quản lý các tệp về các thông tin như địa chỉ, kích thước, ngày tạo, ngày cập nhật…

Tên của tệp thông thường có hai phần. Phần tên chính và phần mở rộng phân cách nhau bởi dấu chấm. Kích thước phần tên chính có số lượng ký tự phụ thuộc theo hệ điều hành. Tên mở rộng thông thường có 3 ký tự.

Ví dụ 8.1:

vanban.txt , hoa.dat, nam.**int** …

Có hai cách giao tiếp với tệp, giao tiếp với tệp theo kiểu tệp văn bản (text file) và giao tiếp với tệp theo kiểu nhị phân (text binary).

Tệp được lưu theo kiểu tệp văn bản có nghĩa là mọi giá trị đều lưu dưới dạng ký tự kể cả số. *Ví dụ* ta lưu số 1024 vào tệp theo kiểu văn bản có nghĩa là ta lưu 4 ký tự để biểu diễn số trên. Tức là trước khi lưu trữ, số trên sẽ được chuyển đổi sang một dãy các ký tự rồi mới lưu xuống tệp văn bản. Chúng ta sẽ lưu 4 ký tự trên bằng 4 byte như sau:

0011 0001	0011 0000	0011 0010	0011 0100
(49)	(48)	(50)	(52)

Mỗi byte lưu một mã ký tự trong bảng mã ASII. Như ở *Ví dụ* trên, số 1234 được lưu bởi 4 byte. Byte đầu tiên biểu diễn số 49 là mã của số 1 trong bảng mã ASII. Tương tự byte thứ 2, 3, 4 lần lượt lưu mã 50, 51 và 52. Nếu chúng ta mở tệp lên bằng các chương trình soạn thảo như winword, notepad ta có thể thấy được nội dung văn bản là 1234.

Tệp được lưu theo kiểu nhị phân có nghĩa là mọi giá trị đều lưu đúng như sự biểu diễn của giá trị đó trong máy tính. Sẽ không có sự chuyển đổi các giá trị thành các ký tự. *Ví dụ* số 1024 trong máy tính được biểu diễn bằng 2 byte lưu xuống tệp nhị phân cũng phải sử dụng 2 byte để biểu diễn như sau:

0000 0100 0000 0000

Nếu chúng ta mở tệp bằng các chương trình soạn thảo ta sẽ không thấy nội dung là 1024 mà sẽ thấy hai ký tự đặc biệt mà ASII của chúng là 8 và 0.

Chú ý:

Khi giao tiếp với tệp theo kiểu tệp văn bản người ta gọi là tệp văn bản, khi giao tiếp với tệp theo kiểu nhị phân người ta gọi tệp là tệp nhị phân. Thực chất cả hai tệp này giống nhau, là một dãy các byte lưu trên đĩa.

8.2. CÁC KỸ THUẬT TRÊN TẬP TIN

Trong nhập xuất chuẩn, chúng ta sử dụng hai đối tượng đã có sẵn là **cin**, **cout** để thực hiện nhập xuất. Tương tự, nhập xuất trên tệp chúng ta cũng phải tạo ra các đối tượng giống **cin** và **cout** của lớp **ifstream** và **ofstream** để thực hiện nhập xuất trên tệp.

8.2.1. Ghi dữ liệu

Các thao tác cần thiết để ghi dữ liệu lên tệp là: tạo tệp (nếu tệp chưa tồn tại), kết nối với tệp, ghi nội dung lên tệp và đóng tệp. Để thực hiện được các thao tác trên ta phải khai báo một đối tượng thuộc lớp **ofstream** như sau:

ofstream tên_đối_tượng;

Ví dụ 8.2:

ofstream fout;

Thao tác tạo và kết nối với tệp được thực hiện thông qua phương thức open() của đối tượng thuộc lớp **ofstream**. Cách gọi thực hiện:

tên_đối_tượng.open(tên_tệp, tham_số mode);

Giải thích:

- tên_tệp: là tên của văn bản mà chúng ta muốn tạo (nếu chưa tồn tại) và kết nối.
- tham_số_mode: Là tham số chọn các kiểu mở tệp của chúng ta. Bao gồm các loại tham số sau:

Tham số	*Ý nghĩa*
ios::out	Mở file để ghi tệp nếu tệp đã có thì nó bị xoá
ios::app	Mở tệp và ghi bổ sung vào cuối tệp
ios::ate	chuyển con trỏ tệp đến cuối tệp sau khi mở tệp
ios::nocreate	Nếu tệp chưa có thì không làm gì
ios::noreplace	Nếu tệp đã có thì không làm gì
ios::binary	Mở file theo kiểu binary (nhị phân)

Tham số ios::out là bắt buộc để mở tệp và thực hiện việc ghi nội dung.

Ví dụ 8.3:

```
ofstream fout;
fout.open("C:\\vidu\\vanban.txt",ios::out);
```

Nhận xét:

Chúng ta vừa tạo ra một tệp có tên vanban.txt ở thư mục vidu ổ đĩa C. Nếu trong thư muc này đã tồn tại tệp có tên vanban.txt thì tệp này sẽ bị xoá nội dung, nếu chưa tồn tại thì tạo file mới. Tệp vừa mở là tệp văn bản.

Ví dụ 8.4:

```
ofstream fout;
fout.open("C:\\vidu\\nhiphan.txt", ios::out | ios::binary);
```

Nhận xét:

- Chúng ta vừa tạo ra một tệp theo kiểu nhị phân.
- Muốn kết hợp các kiểu mở tệp ở trên ta dùng toán tử bitwise | (or).

Ghi dữ liệu lên tệp văn bản bằng đối tượng thuộc lớp **ofstream** toán tử <<.

Ví dụ 8.5:

```
ofstream  fout;
int i = 1;
fout.open("C:\\vidu\\vanban.txt",ios::out);
fout << i;
fout << " day la van ban dau tien";
```

Nhận xét: Chúng ta vừa tạo ra nội dung văn bản là: **day la tep van ban dau tien** .

Ghi dữ liệu tên tệp nhị phân bằng đối tượng thuộc lớp **ofstream** và phương thức write ():

```
tên_đối_tương.write( (char *) đia_chỉ_biến, kích_thước_byte);
```

Ví dụ 8.6:

```
ofstream fout;
int i = 49;
fout.open("C:\\vidu\\nhiphan.txt", ios::out | ios::binary);
fout.write( (char*)&i, sizeof(int));
```

Nhận xét:

- Phương thức write() có nhiệm vụ chép một số lượng byte từ bộ nhớ vào tệp. Tham số thứ nhất truyền vào là con trỏ kiểu **char** là địa chỉ của byte đầu tiên, tham số thứ hai là kích thước truyền vào thuộc kiểu số nguyên. Như vậy hàm write xác định được kích thước vùng nhớ để chép xuống tệp.
- Toán tử sizeof trả về kích thước byte của một kiểu dữ liệu nào đó. *Cú pháp* sử dụng toán tử sizeof: sizeof(tên_**dữ_liệu**).
- Nếu ta mở trình soạn thảo để xem nội dung của nhiphan.txt ta thấy nội dung văn bản là số 1.

Sau khi đã ghi toàn bộ dữ liệu xuống file, chúng ta kết thúc kết nối với tệp bằng phương thức close().

Ví dụ 8.7:

Ghi xuống tệp văn bản:

```
ofstream  fout;
int i = 1;
fout.open("C:\\vidu\\vanban.txt",ios::out);
fout << i;
fout << " day la van ban dau tien";
fout.close();
```

Ghi xuống tệp nhị phân:

```
ofstream fout;
int i = 49;
fout.open("C:\\vidu\\nhiphan.txt", ios::out | ios::binary);
fout.write( (char*)&i, sizeof(int));
fout.close();
```

Sau khi kết thúc kết nối với một tệp, đối tượng của lơp **ofstream** có thể kết nối với tệp khác.

Ví dụ 8.8:

```
#include <iostream.h>
#include <fstream.h>
void main()
{
    ofstream  fout;
    int i = 1;
    fout.open("C:\\vidu\\vanban1.txt",ios::out);
    fout << i;
    fout << " day la van ban dau tien";
    fout.close();
    i++;
    fout.open("C:\\vidu\\vanban2.txt",ios::out);
    fout << i;
    fout << " day la van ban thu hai";
    fout.close();
}
```

Một đối tượng của lớp **ofstream** chỉ thực hiện kết nối tới một tệp tại một thời điểm. Một tệp tại một thời điểm cũng chỉ nên kết nối với một đối tượng của lớp **ofstream**.

8.2.2. Đọc dữ liệu

Các thao tác để đọc dữ liệu: kết nối với tệp , đọc tệp và kết thúc kết nối với tệp. Để khai báo và sử dụng đối tượng thuộc lớp **ifstream** ta phải khai báo sử dụng thư viện **ifstream**.h.

```
#include <ifstream.h>
ifstream  tên_đối_tương;
```

Ví dụ 8.9:

```
#include <ifstream.h>
ifstream  fin;
```

Thao tác tạo và kết nối với tệp được thực hiện thông qua phương thức open() của đối tương thuộc lớp **ifstream**. Cách gọi thực hiện:

```
tên_đối_tượng.open( tên_tệp, tham_số mode );
```

Giải thích:

- tên_tệp: là tên của văn bản mà chúng ta muốn kết nối để thực hiện đọc nội dung. Tên_tệp thuộc kiểu dữ liệu chuỗi.
- tham_số_mode: Là tham số chọn các kiểu mở tệp của chúng ta. Bao gồm các loại tham số sau:

Tham số	*Ý nghĩa*
ios::in	Mở file
ios::ate	chuyển con trỏ tệp đến cuối tệp sau khi mở tệp
ios::binary	Mở file theo kiểu binary (nhị phân)

Tham số ios::in là bắt buộc để mở tệp để thực hiện đọc nội dung tệp.

Ví dụ 8.10:

```
ifstream  fin;
fin.open("C:\\vidu\\vanban.txt",ios::in);
```

Nhận xét:

Chúng ta vừa kết nối với tệp có tên vanban.txt ở thư mục vidu ở ổ đĩa C.

Mặc định của phương thức open của đối tượng lớp **ofstream** là mở đọc theo kiểu text. Để mở và đọc một file theo kiểu nhị phân ta phải kết hợp ios::in và ios::binary

Ví dụ 8.11:

```
ifstream fin;
fin.open("C:\\vidu\\nhiphan.txt",ios::in|ios::binary);
```

Đọc dữ liệu theo kiểu tệp văn bản bằng cách sử dụng đối tượng lớp **ifstream** và toán tử >> hoặc các phương thức **getline()** hoặc get(). Cách sử dụng toán tử >>, phương thức **getline()**, get() cũng tương tự như trong nhập xuất chuẩn (đối tượng **cin**).

Ví dụ 8.12:

```
#include <iostream.h>
#include <ofstream.h>
#include <ifstream.h>
void main()
{
    int i,j, temp;
    char str[50];
// ghi tep
    ofstream fout;
    fout.open("C:\\vidu\\vanban.txt",ios::out);
    fout<<"ma tran ban co: \n";
    for (i = 0; i< 5; i++)
    {
        for(j = 0; j< 5; j++)
            fout << i*5 + j <<'\t';
        fout <<"\n";
    }
    fout.close();
    //doc tep
    ifstream fin;
    fin.open("C:\\vidu\\vanban.txt",ios::in);
    fin.getline(str,50);
    cout << str<<'\n';
    for (i = 0; i< 5; i++)
    {
        for( j = 0; j< 5; j++)
        {
            fin >> temp;
            cout << temp << '\t';
        }
        cout <<"\n";
    }
    fin.close();
```

```
}
```

Đọc một tệp theo kiểu nhị phân cần một đối tượng lớp **ifstream** và phương thức read như sau:

tên_đối_tượng.read((**char***)tên_biến, kích_thước_byte);

Giải thích:

- tên_biến: tên biến mà sẽ lưu giữ nội dung đọc lên từ tệp.
- kích_thước_byte: là kích thước byte sẽ đọc từ tệp hay kích thước của biến.

Ví dụ 8.13:

```
#include <iostream.h>
#include <ofstream.h>
#include <ifstream.h>
void main()
{
int i,j, temp;
    char str[50] = "ma tran ban co: \n";
    ofstream fout;
    fout.open("C:\\vidu\\nhiphan.txt",ios::out|ios::binary);
    fout.write(str,50);
    for (i = 0; i< 5; i++)
    {
        for(j = 0; j< 5; j++)
        {
            temp = i*5 + j;
            fout.write((char*)&temp,sizeof(int));
        }
    }
    fout.close();
    ifstream fin;
    fin.open("C:\\vidu\\nhiphan.txt",ios::in|ios::binary);
    fin.read(str,50);
    cout << str<<'\n';
    for (i = 0; i< 5; i++)
    {
        for( j = 0; j< 5; j++)
        {
            fin.read((char*)&temp,sizeof(int));
            cout << temp << '\t';
```

```
            }
            cout <<"\n";
        }
        fin.close();
    }
```

Nhận xét:

- Để lưu giữ cùng một thông tin, nếu ta sử dụng tệp văn bản thì hoàn toàn khác với tệp nhị phân. Thông thường lưu trữ theo tệp nhị phân sẽ tiết kiệm bộ nhớ hơn.
- Để lấy được thông tin đã lưu trữ một cách chính xác thì văn bản đã được lưu trữ theo kiểu tệp nào ta phai đọc lên theo kiểu tệp ấy.
- Ta phải biết trước cấu trúc lưu trữ của tệp trước khi đọc.

8.3. MỘT SỐ PHƯƠNG THỨC CỦA LỚP IFSTREAM VÀ OFSTREAM

8.3.1. int is_open()

Phương thức này trả về 0 nếu tệp không kết nối được ngược lại trả về giá trị khác 0. Phương thức này được sử dụng thường xuyên sau mỗi lần sử dụng phương thức open. Phương thức này dùng chung cho việc nhập xuất và với cả hai loại tệp.

Ví dụ 8.14:

```
#include <iostream.h>
#include <ofstream.h>
#include <ifstream.h>
void main()
{
int i,j, temp;
    char str[50];
// ghi tep
    ofstream fout;
    fout.open("C:\\vidu\\vanban.txt",ios::out);
    if (!fout.is_open())
        cout << "khong mo duoc tep!";
    else
    {
        fout<<"ma tran ban co: \n";
        for (i = 0; i< 5; i++)
        {
            for(j = 0; j< 5; j++)
                fout << i*5 + j <<'\t';
```

```
            fout <<"\n";
        }
        fout.close();
    }
    //doc tep
    ifstream fin;
    fin.open("C:\\vidu\\vanban.txt",ios::in);
    if(fin.is_open())
    {
        fin.getline(str,50);
        cout << str<<'\n';
        for (i = 0; i< 5; i++)
        {
            for( j = 0; j< 5; j++)
            {
                fin >> temp;
                cout << temp << '\t';
            }
            cout <<"\n";
        }
        fin.close();
    }
    else
        cout << "khong mo duoc tep";
}
```

8.3.2. int eof()

Khi con trỏ tệp đã ở cuối tệp mà vẫn thực hiện một lệnh đọc thì phương thức eof sẽ trả về giá trị khác 0. Người ta thương sử dụng phương thức này để đọc một file khi không biết trước tệp có bao nhiêu phần tử.

Ví dụ 8.15

```
#include <iostream.h>
#include <ofstream.h>
#include <ifstream.h>
void main()
{
    ifstream fin;
    fin.open("C:\\vidu\\vanban.txt",ios::in);
```

```
        if(fin.is_open())
        {
            while (!fin.eof())
            {
                fin.get(ch);
                cout << ch;
            }

            fin.close();
        }
        else
            cout << "khong mo duoc tep";
    }
```

8.3.3. Phương thức di chuyển con trỏ tệp trên dòng xuất

➢ **ostream & seekp(long n);**

Cách sử dụng như sau:

tên_đối_tượng.seekp(n);

Giải thích:

- tên_đối_tương: là tên đối tượng **ofstream** mà ta đã khai báo và kết nối với tệp mà ta dang sử dụng.
- n: là vị trí byte thứ n nào đó trong tệp. Vị trí đầu tiên là 0.

Phương thức trên có ý nghĩa di chuyển con trỏ tệp tới vị trí (byte) thứ n nào đó.

➢ **ostream & seekp(long offset, seek_dir dir);**

Phương này sẽ chuyển con trỏ tệp tới vị trí n kể từ vị trí xuất phát.

Cách sử dụng như sau:

tên_đối_tượng.seekp(n , vị_trí);

Giải thích:

- tên_đối_tương: là tên đối tượng **ofstream** mà ta đã khai báo và kết nối với tệp mà ta đang sử dụng.
- vị_trí: Vị trí là một trong 3 kiểu sau:
    - ios::beg xuất phát từ đầu tệp
    - ios::end xuất phát từ cuối tệp
    - ios::cur xuất phát từ vị trí hiện tại của con trỏ tệp

- n: Là vị trí tính từ vị_trí con trỏ sẽ di chuyển đến. n có thể là số âm.

➢ **long tellp();**

Phương thức này cho biết vị trí hiện tại của tệp.

Phương thức di chuyển con trỏ tệp trên dòng nhập

➢ **istream & seekg(long n);**

Phương thức này di chuyển con trỏ tệp tới vị trí (byte) thứ n nào đó.

Cách sử dụng như sau:

tên_đối_tượng.seekp(n);

Giải thích:

- tên_đối_tương: là tên đối tượng **ifstream** mà ta đã khai báo và kết nối với tệp mà ta đang sử dụng.
- n: là vị trí byte thứ n nào đó trong tệp. Vị trí đầu tiên là 0.

➢ **istream & seekg(long offset, seek_dir dir);**

Phương trên sẽ chuyển con trỏ tệp tới vị trí n kể từ vị trí xuất phát.

Cách sử dụng như sau:

tên_đối_tượng.seekp(n , vị_trí);

Giải thích:

- tên_đối_tương: là tên đối tượng **ifstream** mà ta đã khai báo và kết nối với tệp mà ta đang sử dụng.
- vị_trí: Vị trí là một trong 3 kiểu sau:
  - ios::beg xuất phát từ đầu tệp
  - ios::end xuất phát từ cuối tệp
  - ios::cur xuất phát từ vị trí hiện tại của con trỏ tệp
- n: Là vị trí tính từ vị_trí con trỏ sẽ di chuyển đến. n có thể là số âm.

➢ **long teelg();**

Phương thức này cho biết vị trí hiện tại của tệp.

8.3.4. Một số phương thức khác

➢ **int fail()**

Phương thức này trả về giá trị khác không nếu lần nhập xuất cuối cùng có lỗi, trái lại phương thức có giá trị bằng 0.

➢ **int bad()**

Phương thức bad() trả về giá tri khác không khi một phép nhập xuất không hợp lệ hoặc có lỗi mà chưa phát hiện được, trái lại phương thức có giá trị bằng 0.

- **int good()**

Phương thức good() trả về giá trị khác không nếu mọi việc đều tốt (không có lỗi xảy ra). Khi có một lỗi nào đó thì phương thức có giá trị bằng 0.

- **void clear()**

Dùng để tắt hết các bit lỗi.

8.4. ĐỌC GHI ĐỒNG THỜI TRÊN TỆP

Để đọc ghi đồng thời trên tệp chúng ta sử dụng lớp fstream. Các thao tác để thực hiện việc đọc ghi đồng thời là: khai báo một đối tượng nhập xuất của lớp fstream, dùng phương thức open để kết nối với tệp. Sau khi thực hiện việc đọc ghi trên tệp, dùng phương thức close để kết thúc kết nối. Sau đó đối tương này có thể kết nối với một tệp khác.

```
#include <fstream.h>
fstream    tên_đối_tượng;
```

Ví dụ 8.16:

```
fstream    f;
```

Sử dụng phương thức open để kết nối với một tệp:

```
tên_đối_tượng.open( tên_tệp, tham_số mode );
```

Giải thích:

- tên_tệp: là tên của văn bản mà chúng ta muốn kết nối để thực hiện đọc nội dung. Tên_tệp thuộc kiểu dữ liệu chuỗi.
- tham_số_mode: Là tham số chọn các kiểu mở . Bao gồm các loại tham số sau:

Tham số	*Ý nghĩa*
ios::in	Mở file
ios::out	ghi tệp, nếu tệp đã tồn tại thì bị xoá nội dung
ios::app	ghi bổ sung vào cuối tệp
ios::ate	chuyển con trỏ tệp đến cuối tệp sau khi mở tệp
ios::trunc	xoá nội dung nếu tệp đã tồn tại
ios::binary	Mở file theo kiểu binary (nhị phân)
ios::nocreate	nếu tệp chưa có thì không làm gì

ios::noreplace	nếu tệp đã có thì không làm gì

Chúng ta muốn thực hiện cả thao tác ghi và đọc cùng một lúc thì kết hợp ios::in và ios::out. Khi kết hợp hai tham số này lại thì nội dung của tệp không bị xoá khác với chỉ sử dụng một tham số ios::out.

Ví dụ 8.17:

```
fstream  f;
f.open("C:\\vidu\\vanban.txt",ios::in|ios::out);
fstream f2;
f2.open("C:\\vidu\\nhiphan.txt", ios::in|ios::out|ios::binary);
```

Nhận xét:

- f được gắn với tệp vanban.txt theo kiểu tệp văn bản.
- f2 được gắn kết với tệp nhiphan.txt theo kiểu tệp nhị phân.
- Chúng ta có thể thực hiện các thao tác đọc ghi trên cả hai tệp trên.

Việc đọc và ghi tương tự như khi sử dụng đối tượng thuộc lớp **ofstream** và **ifstream**. Đối với tệp văn bản, ta sử dụng toán << hoặc >> hoặc getline, get. Đối với tệp nhị phân, ta dùng <<, >>, write và read.

Để di chuyển con trỏ tệp trên dòng nhập xuất, ta sử dụng các phương thức : seekp, seekg.

Ví dụ 8.18:

Nhập xuất trên tệp văn bản

Chương trình dưới đây sẽ nhập danh sách các loài hoa và sau đó xuất ra màn hình.

```
#include <iostream.h>
#include <fstream.h>
struct loai_hoa
{
    char   ten[20];
    char   xuatxu[20];
    int gia[2];
};
void main()
{
    fstream f;
    loai_hoa hoa;
    char ch;
```

```
        f.open("C:\\vidu\\dshoa.txt",ios::in|ios::out);
        if(f.is_open())
        {
            f << "Hong nhung \tDa Lat \t500\t1000\n";
            f << "Cuc vang \tDaklak \t200\t400\n";
            f << "Ly \tDaLat \t4000\t8000\n";
            f << "Dong tien \tDakNong\t500\t1000\n";
            f << "Hue \tDaklak\t500\t1000\n";
            f << "Cam chuong \tDa Lat\t2000\t3000\n";
            f.seekp(0,ios::beg);
            f.get(hoa.ten,20,'\t');
            f.get(ch);
            f.get(hoa.xuatxu,20,'\t');
            f.get(ch);
            f >> hoa.gia[0];
            f >> hoa.gia[1];
            f.get(ch);
            while (!f.eof())
            {
                cout << hoa.ten << '\t';
                cout << hoa.xuatxu <<'\t';
                cout << hoa.gia[0] << '\t';
                cout << hoa.gia[1] << '\t' <<'\n';
                f.get(hoa.ten,20,'\t');
                f.get(ch);
                f.get(hoa.xuatxu,20,'\t');
                f.get(ch);
                f >> hoa.gia[0];
                f >> hoa.gia[1];
                f.get(ch);
            }
            f.close();
        }
        else
            cout << "khong mo duoc tep";
    }
```

Kết quả

Hong nhung	Da Lat	500	1000
Cuc vang	Daklak	200	400
Ly DaLat		4000	8000
Dong tien	DakNong	500	1000
Hue	Daklak	500	1000
Cam chuong	Da Lat	2000	3000

Nhập xuất trên tệp nhị phân

Chương trình sau sẽ tạo một tệp nhị phân lưu trữ danh sách các loại hoa, xuât xứ, giá mua, giá bán. Thực hiện chỉnh sửa nội dung của một phần tử trong tệp.

```
#include <iostream.h>
#include <fstream.h>
#include <string.h>
#include <iomanip.h>
struct loai_hoa
{
    char   ten[20];
    char   xuatxu[20];
    int gia[2];
};
void khoitao(loai_hoa & h,char ten[], char xuatxu[], int giamua, int
giaban);
void xuat(loai_hoa hoa);
void main()
{
    fstream f;
    loai_hoa hoa;
    char ch;
    f.open("C:\\vidu\\dshoa.txt",ios::in|ios::out|ios::binary);
    if(f.is_open())
    {
        khoitao(hoa,"Hong nhung","Da Lat",500,1000);
        f.write((char*)&hoa,sizeof(loai_hoa));
        khoitao(hoa,"Cuc vang","DakLak",200,400);
        f.write((char*)&hoa,sizeof(loai_hoa));
        khoitao(hoa,"Dong tien","DakNong",500,1000);
        f.write((char*)&hoa,sizeof(loai_hoa));
        khoitao(hoa,"Hue","DakLak",1000,2000);
        f.write((char*)&hoa,sizeof(loai_hoa));
```

```
        khoitao(hoa,"Cam chuong","Da Lat",2000,3000);
        f.write((char*)&hoa,sizeof(loai_hoa));
        f.seekp(0,ios::beg);
        cout.setf(ios::left);
        cout<<setw(20)<<"TEN HOA"<<setw(20)<<"XUAT XU"
<<setw(10)<<"GIA MUA"<<setw(10)<<"GIA BAN" <<'\n';
        f.read((char*)&hoa,sizeof(loai_hoa));
        while (!f.eof())
        {
            xuat(hoa);
            f.read((char*)&hoa,sizeof(loai_hoa));
        }
        f.clear();
        cout << "\n\n";
        f.seekp(0,ios::beg);
        while (!f.eof())
        {
            f.read((char*)&hoa,sizeof(loai_hoa));
            if( strcmp(hoa.ten,"Hue") == 0)
            {
                f.seekp(-sizeof(loai_hoa),ios::cur);
                hoa.gia[1] = 1500;
                f.write((char*)&hoa,sizeof(loai_hoa));
            }
        }
        f.clear();
        f.seekp(0,ios::beg);
        f.read((char*)&hoa,sizeof(loai_hoa));
        cout<<setw(20)<<"TEN HOA"<<setw(20)<<"XUAT XU"
            <<setw(10)<<"GIA MUA"<<setw(10)<<"GIA BAN" <<'\n';
        while (!f.eof())
        {
            xuat(hoa);
            f.read((char*)&hoa,sizeof(loai_hoa));
        }
        f.close();
    }
    else
```

```
            cout << "khong mo duoc tep";
    }
    void xuat(loai_hoa hoa)
    {
        cout.setf(ios::left);
        cout <<setw(20)<< hoa.ten ;
        cout <<setw(20)<< hoa.xuatxu ;
        cout <<setw(10)<< hoa.gia[0] ;
        cout <<setw(10)<< hoa.gia[1] <<'\n';
    }
    void khoitao(loai_hoa &h,char ten[], char xuatxu[], int giamua, int
    giaban)
    {
        strcpy(h.ten,ten);
        strcpy(h.xuatxu,xuatxu);
        h.gia[0] = giamua;
        h.gia[1] = giaban;
    }
```

Kết quả:

```
TEN HOA          XUAT XU          GIAMUA    GIABAN
Hong nhung       Da Lat           500       1000
Cuc vang         DakLak           200       400
Dong tien        DakNong          50        1000
Hue              DakLak           1000      2000
Cam chuong       Da Lat           2000      3000

TEN HOA          XUAT XU          GIAMUA    GIABAN
Hong nhung       Da Lat           500       1000
Cuc vang         DakLak           200       400
Dong tien        DakNong          500       1000
Hue              DakLak           1000      1500
Cam chuong       Da Lat           2000      3000
```

8.5. BÀI TẬP ỨNG DỤNG

1. Viết nội dụng bài thơ 5 điều bác Hồ dạy vào một file văn bản.

- Đọc và xuất ra màn hình
- Thêm các từ “bao” sau mỗi từ “day”.
- Xóa tất cả các từ “day” trong file văn bản.

2. Viết chương trình nhập danh sách học sinh, tên ngày sinh, lớp, điểm 3 môn.

- Lưu danh sách học sinh xuống file nhị phân
- Đọc danh sách học sinh và xuất ra màn hình.
- Thêm học sinh vào file
- Xóa học sinh ra khỏi file theo tên được nhập vào bàn phím
- Tạo các file văn bản với nội dung là danh sách học sinh của tưng lớp đã được sắp xếp tăng dần theo tên.
- Viết chương trình đọc các file văn bản trên ra màn hình
- Tạo các file văn bản với nội dung là bảng xếp loại học sinh của từng lớp với cách tính điểm như sau:
    - Dtb = (điểm m1 + điểm m2 + điểm m3) /3
    - Dtb >= 9: xuất xắc
    - 8 <= dtb < 9: giỏi
    - 7 <= dtb < 8: khá
    - 5 <= dtb <7: trung bình
    - 0 <= dtb <5: yếu